

Webscrapping v2.27

June 1, 2026

```
[30]: # ===== WEBSCRAPING 2.25 - CELDA 1/5: IMPORTS + CONSTANTES +  
      ↪ AUXILIARES =====  
# Qué hace:  
# - Imports mínimos del wrapper (WS 2.25)  
# - Constantes de salida  
# - parsear_preio_mxn() para asegurar Precio_num al cargar  
  
import os  
import re  
import numpy as np  
import pandas as pd  
  
RAW_CSV = "Casas_raw.csv"  
LIMPIO_CSV = "Casas_limpio.csv"  
  
TIPO_CAMBIO_USD_MXN = 18.0  
  
def parsear_preio_mxn(texto, tipo_cambio: float = TIPO_CAMBIO_USD_MXN):  
    if texto is None or (isinstance(texto, float) and np.isnan(texto)):  
        return np.nan  
    s = str(texto).lower()  
  
    moneda = "MXN"  
    if ("usd" in s or "us$" in s or "u$s" in s or "dólar" in s or "dolar" in s):  
        moneda = "USD"  
  
    m = re.search(r"(\d[\d,\.]*)", str(texto))  
    if not m:  
        return np.nan  
  
    try:  
        valor = float(m.group(1).replace(",", ""))  
    except ValueError:  
        return np.nan  
  
    if moneda == "USD":  
        valor *= tipo_cambio
```

```
return valor
```

```
[31]: # ===== WEBSCRAPING 2.25 - CELDA 2/5: EJECUTAR scraper_runner.
      ↪ ipynb EN ESTE KERNEL =====
      # Qué hace:
      # - Corre scraper_runner.ipynb con %run -i (mismo kernel)
      # - Evita que SystemExit tumbe el kernel
      # - Evita doble ejecución: SOLO llama main() si el runner NO generó/actualizó
      ↪ CSVs

      import os
      import traceback
      from IPython import get_ipython

      SCRAPER_NOTEBOOK = os.path.join(os.getcwd(), "scraper_runner.ipynb")
      if not os.path.exists(SCRAPER_NOTEBOOK):
          raise FileNotFoundError(f"No existe: {SCRAPER_NOTEBOOK}")

      def _mtime(path: str):
          return os.path.getmtime(path) if os.path.exists(path) else None

      raw_before = _mtime(RAW_CSV)
      limpio_before = _mtime(LIMPIO_CSV)

      print("Ejecutando:", SCRAPER_NOTEBOOK)

      try:
          get_ipython().run_line_magic("run", f'-i "{SCRAPER_NOTEBOOK}"')
      except SystemExit as e:
          print(f"[WARN] SystemExit atrapado al ejecutar el runner: {e}")
      except Exception:
          print("[ERROR] Fallo ejecutando el runner.")
          traceback.print_exc()
          raise

      raw_after = _mtime(RAW_CSV)
      limpio_after = _mtime(LIMPIO_CSV)

      runner_ya_genero_outputs = (
          (raw_after is not None and raw_before != raw_after) or
          (limpio_after is not None and limpio_before != limpio_after)
      )

      if runner_ya_genero_outputs:
          print("El runner YA generó/actualizó CSVs. No se llama main() (evitar doble
          ↪ ejecución).")
```

```

else:
    try:
        if "main" in globals() and callable(globals()["main"]):
            ret = globals()["main]()
            print("Scraping terminado. main() retornó:", ret)
        else:
            print("No existe main() en globals() y el runner no generó outputs.␣
↪Revisa runner.")
    except SystemExit as e:
        print(f"[WARN] SystemExit atrapado dentro de main(): {e}")
    except Exception:
        print("[ERROR] Fallo dentro de main().")
        traceback.print_exc()
        raise

```

Ejecutando: C:\Users\yasc1\Documents\Ingenieria
financiera\Web scraping2\Web scraping v2.27\scraper_runner.ipynb
URL inicial: https://inmuebles.mercadolibre.com.mx/casas/venta/distrito-
federal/_DisplayType_M
Sesión OK / listado cargado (no se pidió login).
Abriendo: https://inmuebles.mercadolibre.com.mx/casas/venta/distrito-
federal/_DisplayType_M
[CP] Nominatim: requests=300 cache_hits=356 max=300
Guardado Casas_raw.csv
Guardado Casas_raw.xlsx
C:\Users\yasc1\AppData\Local\Temp\ipykernel_14684\3791896421.py:48: UserWarning:
DataFrame columns are not unique, some columns will be omitted.
df.to_xml(f"{base_name}.xml", index=False)
Guardado Casas_raw.xml
Guardado Casas_limpio.csv
Guardado Casas_limpio.xlsx
C:\Users\yasc1\AppData\Local\Temp\ipykernel_14684\3791896421.py:48: UserWarning:
DataFrame columns are not unique, some columns will be omitted.
df.to_xml(f"{base_name}.xml", index=False)
Guardado Casas_limpio.xml
Scraping terminado. main() retornó: {'ok': True, 'raw': 3992, 'limpio': 3992,
'tiempo': 95300.84680247307}

[32]: # ===== WEBSCRAPING 2.25 - CELDA 3/5: CARGA + VALIDACIÓN +␣
↪ASEGURAR Precio_num =====
Qué hace:
- Verifica que existan los 2 CSV
- Carga df_raw y df_limpio
- Asegura columna Precio_num (si no existe, la crea desde Precio)

```

import os
import pandas as pd
import numpy as np

if not (os.path.exists(RAW_CSV) and os.path.exists(LIMPIO_CSV)):
    raise FileNotFoundError(
        f"No encuentro {RAW_CSV} o {LIMPIO_CSV}. Revisa la salida de la CELDA 2.
        ↪"
    )

df_raw = pd.read_csv(RAW_CSV, encoding="utf-8-sig")
df_limpio = pd.read_csv(LIMPIO_CSV, encoding="utf-8-sig")

if "Precio_num" not in df_limpio.columns:
    if "Precio" in df_limpio.columns:
        df_limpio["Precio_num"] = df_limpio["Precio"].apply(parsear_precio_mxn)
    else:
        df_limpio["Precio_num"] = np.nan

print("raw:", df_raw.shape, "| limpio:", df_limpio.shape)
df_limpio.head()

```

raw: (3992, 203) | limpio: (3992, 204)

[32]:

Link \

```

0 https://casa.mercadolibre.com.mx/MLM-257886171...
1 https://casa.mercadolibre.com.mx/MLM-257886171...
2 https://casa.mercadolibre.com.mx/MLM-254901132...
3 https://casa.mercadolibre.com.mx/MLM-254901132...
4 https://casa.mercadolibre.com.mx/MLM-247728102...

```

Direccion \

```

0 Reserva del sur
1 Reserva del sur
2 Vitant Santa Fe By Be Grand
3 Vitant Santa Fe By Be Grand
4 Magnifica Casa Multifuncional.

```

Ubicacion \

```

0 Conrado Pelayo 67, Cp. 13200, Col. Miguel Hida...
1 Conrado Pelayo 67, Cp. 13200, Col. Miguel Hida...
2 Av. Santa Fe 578, Lomas De Santa Fe, Contadero...
3 Av. Santa Fe 578, Lomas De Santa Fe, Contadero...
4 R. López Velarde, Xalpa, Ciudad De México, San...

```

Maps_url Lat Lng \

```

0 https://www.google.com/maps?q=19.296073,-99.04... 19.296073 -99.044325

```

1	https://www.google.com/maps?q=19.296073,-99.04...	19.296073	-99.044325
2	https://www.google.com/maps?q=19.3560228,-99.2...	19.356023	-99.275232
3	https://www.google.com/maps?q=19.3560228,-99.2...	19.356023	-99.275232
4	https://www.google.com/maps?q=19.3357756,-99.0...	19.335776	-99.022957

	POI_Botones	\
0	Transporte Educación Áreas verdes Comerc...	
1	Transporte Educación Áreas verdes Comerc...	
2	Transporte Educación Áreas verdes Comerc...	
3	Transporte Educación Áreas verdes Comerc...	
4	Transporte Educación Áreas verdes Comerc...	

	Transporte_txt_all	\
0	Estaciones de metro Nopalera 6 mins - 458 ...	
1	Estaciones de metro Nopalera 6 mins - 458 ...	
2	Estaciones de metro "Estación "Santa Fé\" ...	
3	Estaciones de metro "Estación "Santa Fé\" ...	
4	Paraderos Palmitas 7 mins - 532 metros\nPa...	

	Escuelas_txt_all	\
0	Escuelas Colegio de Culturas de México 11 ...	
1	Escuelas Colegio de Culturas de México 11 ...	
2	Jardines de niños Jardin de Niños Carmen Gon...	
3	Jardines de niños Jardin de Niños Carmen Gon...	
4	Escuelas Escuela Secundaria Xalpa 318 15 m...	

	AreasVerdes_txt_all	...	\
0	Plazas Parque y Gimnasios Urbanos 222 11 m... ..		
1	Plazas Parque y Gimnasios Urbanos 222 11 m... ..		
2	Plazas La Mexicana 3 mins - 261 metros\nPl... ..		
3	Plazas La Mexicana 3 mins - 261 metros\nPl... ..		
4	Plazas Parque El Mirador 15 mins - 1,143 m... ..		

	Transporte_prom_mins	Transporte_prom_metros	Escuelas_prom_mins	\
0	14.666667	1123.000000	12.0	
1	14.666667	1123.000000	12.0	
2	10.333333	822.666667	18.6	
3	10.333333	822.666667	18.6	
4	10.666667	813.333333	16.0	

	Escuelas_prom_metros	AreasVerdes_prom_mins	AreasVerdes_prom_metros	\
0	947.4	14.2	1105.4	
1	947.4	14.2	1105.4	
2	1450.0	9.6	758.2	
3	1450.0	9.6	758.2	
4	1249.0	18.2	1410.8	

	Comercios_prom_mins	Comercios_prom_metros	Salud_prom_mins	\
0	14.500000	1133.125	18.500000	
1	14.500000	1133.125	18.500000	
2	11.222222	987.875	20.333333	
3	11.222222	987.875	20.333333	
4	17.600000	1363.800	12.000000	

	Salud_prom_metros
0	1438.250000
1	1438.250000
2	1590.333333
3	1590.333333
4	901.000000

[5 rows x 204 columns]

```
[2]: # ===== WEBSCRAPING 2.30 - CELDA 4/5: CARGA para comprar y agregar
      ↪ el municipio =====
# Qué hace:
# - Revisa los datos de la base de datos, los organiza de una mejor manera
# - Revisa la BD de los municipios por código postal y hace el cruce de
      ↪ información

import pandas as pd
import re

# =====
# CONFIG
# =====
CASAS_IN = "Casas_limpio.csv"
CP_CAT_IN = "CodigosPostales_CDMX_sin_duplicados.xlsx"

OUT_CSV = "Casas_FINAL_sin_shapefile.csv"
OUT_XLSX = "Casas_FINAL_sin_shapefile.xlsx"

# =====
# Cargar datos
# =====
df = pd.read_csv(CASAS_IN, dtype=str)
df.columns = df.columns.astype(str).str.strip()

cp = pd.read_excel(CP_CAT_IN, dtype=str)
cp.columns = cp.columns.astype(str).str.strip()

# =====
# 1) Eliminar columnas *_txt_all (todas las variantes)
```

```

# =====
pat_drop = r'^(Transporte|Escuelas|AreasVerdes|Comercios|Salud)_txt_all(\..+)?$'
cols_drop = [c for c in df.columns if re.match(pat_drop, c)]
df = df.drop(columns=cols_drop, errors="ignore")

# =====
# 2) Reordenar columnas
# =====
orden_deseado = [
    "Link",
    "POI_Botones",
    "Direccion",
    "Ubicacion",
    "Maps_url",
    "Lat",
    "Lng",
    "CP",
    "Recamaras",
    "Superficie_m2",
    "Banos",
    "Unidades",
    "Precio",
    "Precio_num",
    "Transporte_POI",
    "Escuelas_POI",
    "AreasVerdes_POI",
    "Comercios_POI",
    "Salud_POI",
]
orden_deseado = [c for c in orden_deseado if c in df.columns]
resto = [c for c in df.columns if c not in orden_deseado]
df = df[orden_deseado + resto].copy()

# =====
# 3) Detectar columnas del catálogo (CP, Municipio, Asentamiento NOMBRE)
# =====
col_cp_cat = next(c for c in cp.columns if "postal" in c.lower() or c.lower()
    == "cp")
col_mun_cat = next(c for c in cp.columns if "municip" in c.lower() or "alcald"
    in c.lower())
col_asent_cat = next(c for c in cp.columns if "asentamiento" in c.lower() and
    "tipo" not in c.lower())

# Normalizar CP
df["CP"] = df["CP"].astype(str).str.strip().str.replace(r"\.0$", "",
    regex=True).str.zfill(5)

```

```

cp[col_cp_cat] = cp[col_cp_cat].astype(str).str.strip().str.replace(r"\.0$", "0",
↳", regex=True).str.zfill(5)

cp[col_mun_cat] = cp[col_mun_cat].astype(str).str.strip()
cp[col_asent_cat] = cp[col_asent_cat].astype(str).str.strip()

# =====
# 4) Construir mapeos sin duplicar filas
# =====
mun_map = (
    cp.dropna(subset=[col_cp_cat, col_mun_cat])
    .groupby(col_cp_cat)[col_mun_cat]
    .agg(lambda s: s.mode().iat[0] if not s.mode().empty else s.iloc[0])
    .reset_index()
    .rename(columns={col_cp_cat: "CP", col_mun_cat: "Municipio"})
)

asent_map = (
    cp.dropna(subset=[col_cp_cat, col_asent_cat])
    .groupby(col_cp_cat)[col_asent_cat]
    .agg(lambda s: s.mode().iat[0] if not s.mode().empty else s.iloc[0])
    .reset_index()
    .rename(columns={col_cp_cat: "CP", col_asent_cat: "Asentamiento"})
)

# =====
# 5) Merge e insertar después de CP
# =====
df2 = df.merge(mun_map, on="CP", how="left").merge(asent_map, on="CP",
↳how="left")

cols = df2.columns.tolist()
idx = cols.index("CP")
cols_final = (
    cols[idx+1:]
    + ["Municipio", "Asentamiento"]
    + [c for c in cols[idx+1:] if c not in ["Municipio", "Asentamiento"]]
)
df2 = df2[cols_final]

# =====
# 6) Guardar
# =====
df2.to_csv(OUT_CSV, index=False)
df2.to_excel(OUT_XLSX, index=False)

print("Listo.")

```



```

print("Salida:", OUT_CSV, "y", OUT_XLSX)
print("Municipios únicos:", df2["Municipio"].nunique())
print("Asentamientos no nulos:", df2["Asentamiento"].notna().sum(), "de",
      len(df2))

```

Listo.

Salida: Casas_FINAL_sin_shapefile.csv y Casas_FINAL_sin_shapefile.xlsx

Municipios únicos: 15

Asentamientos no nulos: 760 de 3992

```

[3]: # ===== WEBSCRAPING 2.30 - CELDA 5/5: CARGA para clcular el precio
      ↳ de las viviendas en base a los indices de los trimestres =====
# Qué hace:
# - Revisa los datos de la base de datos ya con unicipio
# - Revisa la base de datos de los indices de los precios por trimestre dentro
      ↳ de la cdmx
# - En base al indice por trimestre, ya sea del municipio (si es que esta en la
      ↳ BD y en el trimestre), o con el indice global
#   se calcula el precio de la vivienda en cada uno de los trimestres y se
      ↳ agregan las columnas de todos los precios por trimestre

import pandas as pd
import numpy as np

# =====
# CONFIG
# =====
CASAS_IN = "Casas_FINAL_sin_shapefile.csv"
SHF_IN   = "Indice_SHF_CDMX.xlsx"

OUT_CSV  = "Casas_FINAL_precios_historicos_SHF_comodin3.csv"
OUT_XLSX = "Casas_FINAL_precios_historicos_SHF_comodin3.xlsx"

# =====
# 1) Cargar
# =====
df = pd.read_csv(CASAS_IN, dtype=str)
shf = pd.read_excel(SHF_IN, dtype=str)

df.columns = df.columns.str.strip()
shf.columns = shf.columns.str.strip()

# =====
# 2) Preparar casas
# =====
df["Precio_num"] = pd.to_numeric(df["Precio_num"], errors="coerce")

```

```

# Normalizar Municipio (permitir vacíos)
df["Municipio"] = df["Municipio"].astype(str).str.strip()
df.loc[df["Municipio"].str.lower().isin(["nan", "none", ""]), "Municipio"] = np.
    ↳nan

# =====
# 3) Preparar SHF
# =====
shf["Año"] = shf["Año"].astype(int)
shf["Trimestre"] = shf["Trimestre"].astype(int)
shf["Indice"] = pd.to_numeric(shf["Indice"], errors="coerce")

shf["Municipio"] = shf["Municipio"].astype(str).str.strip()
shf.loc[shf["Municipio"].str.lower().isin(["nan", "none", ""]), "Municipio"] =
    ↳np.nan

shf = shf.dropna(subset=["Indice"])
shf["periodo"] = shf["Año"].astype(str) + "_T" + shf["Trimestre"].astype(str)

# =====
# 4) Separar índices: por municipio vs comodín (sin municipio)
# =====
shf_mun = shf.dropna(subset=["Municipio"]).copy()
shf_wild = shf[shf["Municipio"].isna()].copy()

# comodín por periodo (año-trimestre)
wild_series = (
    shf_wild.groupby("periodo")["Indice"]
    .agg(lambda s: s.mode().iat[0] if not s.mode().empty else s.iloc[0])
)

# matriz municipio x periodo
idx_pivot = shf_mun.pivot_table(index="Municipio", columns="periodo",
    ↳values="Indice", aggfunc="mean")

# rellenar faltantes dentro de municipios con comodín del mismo periodo
for p in idx_pivot.columns:
    if p in wild_series.index:
        idx_pivot[p] = idx_pivot[p].fillna(wild_series[p])

# =====
# 5) Periodo base = último disponible (max Año/Trimestre)
# =====
base_y = shf["Año"].max()
base_q = shf.loc[shf["Año"] == base_y, "Trimestre"].max()
base_periodo = f"{base_y}_T{base_q}"

```

```

# asegurar que exista base en pivot (si no, se creará)
if base_periodo not in idx_pivot.columns:
    idx_pivot[base_periodo] = np.nan

# rellenar base con comodín si existe
if base_periodo in wild_series.index:
    idx_pivot[base_periodo] = idx_pivot[base_periodo].
    ↪ fillna(wild_series[base_periodo])

# =====
# 6) Lookup de índices por vivienda
# - si Municipio es NaN -> usar comodín SIEMPRE
# - si Municipio no existe en el pivot -> comodín
# =====
idx_lookup = idx_pivot.reset_index()
df2 = df.merge(idx_lookup, on="Municipio", how="left")

period_cols = idx_pivot.columns.tolist()
df2[period_cols] = df2[period_cols].apply(pd.to_numeric, errors="coerce")

# Aplicar regla de comodín
for p in period_cols:
    if p in wild_series.index:
        df2[p] = df2[p].fillna(wild_series[p])

# =====
# 7) Calcular precios históricos en bloque
# =====
idx_base = df2[base_periodo]
precio = df2["Precio_num"]

factors = df2[period_cols].div(idx_base, axis=0)
precios_mat = factors.mul(precio, axis=0).rename(columns={p: f"precio_{p}" for_
    ↪ p in period_cols})

df_out = pd.concat(
    [df2.drop(columns=period_cols, errors="ignore"), precios_mat],
    axis=1
).copy()

# =====
# 8) Guardar
# =====
df_out.to_csv(OUT_CSV, index=False)
df_out.to_excel(OUT_XLSX, index=False)

print("Listo.")

```

```
print("Periodo base usado:", base_periodo)
print("Archivos:", OUT_CSV, "y", OUT_XLSX)
print("Casas sin Municipio:", int(df["Municipio"].isna().sum()))
print("Periodos (trimestres) generados:", len(period_cols))
```

Listo.

Periodo base usado: 2025_T3

Archivos: Casas_FINAL_precios_historicos_SHF_comodin3.csv y

Casas_FINAL_precios_historicos_SHF_comodin3.xlsx

Casas sin Municipio: 3232

Periodos (trimestres) generados: 83

```
[4]: print("Dimensión total del DataFrame final (filas, columnas):", df_out.shape)
```

Dimensión total del DataFrame final (filas, columnas): (3992, 279)

0.1 Modelo Econométrico 1: Modelo semi-log estructural (base)

0.1.1 Especificación del modelo

$$\ln(\text{Precio}_i) = \beta_0 + \beta_1 \text{Recamaras}_i + \beta_2 \text{Banos}_i + \beta_3 \ln(\text{Superficie}_i) + \beta_4 \text{Transporte_prom_mins}_i + \beta_5 \text{Transporte_prom_metros}_i + \beta_6 \text{Escuelas_prom_mins}_i + \beta_7 \text{Escuelas_prom_metros}_i + \beta_8 \text{Escuelas_prom_metros_log}_i + \beta_9 \text{AreasVerdes_prom_metros}_i + \beta_{10} \text{Comercios_prom_mins}_i + \beta_{11} \text{Comercios_prom_metros}_i + \beta_{12} \text{Salud_prom_metros}_i$$

0.1.2 Descripción e interpretación de los términos

- $\ln(\text{Precio}_i)$:
Logaritmo natural del precio de la vivienda i .
Esta transformación permite interpretar los coeficientes en términos porcentuales y reduce problemas de heterocedasticidad.
- β_0 (intercepto):
Nivel base del logaritmo del precio cuando todas las variables explicativas toman valor cero.
No tiene interpretación económica directa, pero es necesario para el ajuste del modelo.

0.1.3 Variables estructurales del inmueble

- Recamaras_i :
Número de recámaras de la vivienda i .
– β_1 representa el **cambio porcentual aproximado** en el precio ante una recámara adicional, manteniendo constantes las demás variables.
- Banos_i :
Número de baños de la vivienda i .
– β_2 mide el **efecto porcentual** de agregar un baño adicional al precio de la vivienda.

- $\ln(Superficie_i)$:
Logaritmo natural de la superficie en metros cuadrados.
- β_3 es una **elasticidad precio-superficie**:

$$\beta_3 = \frac{\% \Delta Precio}{\% \Delta Superficie}$$

Indica en cuánto porcentaje cambia el precio ante un aumento del 1% en la superficie.

0.1.4 Variables de accesibilidad urbana (tiempos)

- $Transporte_prom_mins_i$:
Tiempo promedio (en minutos) para llegar a transporte.
- $Escuelas_prom_mins_i$:
Tiempo promedio (en minutos) para llegar a escuelas.
- $AreasVerdes_prom_mins_i$:
Tiempo promedio (en minutos) para llegar a áreas verdes.
- $Comercios_prom_mins_i$:
Tiempo promedio (en minutos) para llegar a comercios.
- $Salud_prom_mins_i$:
Tiempo promedio (en minutos) para llegar a servicios de salud.

Para estas variables, los coeficientes $\beta_4, \beta_6, \beta_8, \beta_{10}, \beta_{12}$ representan el **cambio porcentual aproximado en el precio** ante un minuto adicional de traslado, manteniendo constantes las demás variables.

0.1.5 Variables de accesibilidad urbana (distancias)

- $Transporte_prom_metros_i$
- $Escuelas_prom_metros_i$
- $AreasVerdes_prom_metros_i$
- $Comercios_prom_metros_i$
- $Salud_prom_metros_i$

Estas variables miden la distancia promedio (en metros) a cada tipo de servicio.

Los coeficientes $\beta_5, \beta_7, \beta_9, \beta_{11}, \beta_{13}$ indican el **efecto porcentual aproximado en el precio** ante un aumento unitario en la distancia, ceteris paribus.

0.1.6 Término de error

- ε_i :
Término de error aleatorio que recoge todos los factores no observables que afectan el precio de la vivienda, como calidad constructiva, estado del inmueble, vistas, ruido, negociación y preferencias individuales.

```
[8]: import pandas as pd
import numpy as np
import statsmodels.api as sm

# =====
# Cargar
# =====
df = pd.read_csv("Casas_FINAL_precios_historicos_SHF_comodin3.csv", dtype=str)
df.columns = df.columns.astype(str).str.strip()

# =====
# Variables (14)
# =====
TARGET = "Precio_num"

FEATURES = [
    "Recamaras",
    "Banos",
    "Superficie_m2",
    "Transporte_prom_mins",
    "Transporte_prom_metros",
    "Escuelas_prom_mins",
    "Escuelas_prom_metros",
    "AreasVerdes_prom_mins",
    "AreasVerdes_prom_metros",
    "Comercios_prom_mins",
    "Comercios_prom_metros",
    "Salud_prom_mins",
    "Salud_prom_metros",
]

# convertir a numérico
for c in [TARGET] + FEATURES:
    if c not in df.columns:
        raise KeyError(f"Falta columna requerida: {c}")
    df[c] = pd.to_numeric(df[c], errors="coerce")

df_model = df.dropna(subset=[TARGET] + FEATURES).copy()

# =====
# Transformaciones econométricas (mejor práctica)
```

```
# =====
# Log del precio (target)
df_model["log_precio"] = np.log(df_model[TARGET])

# Log de superficie (elasticidad de m2)
df_model["log_superficie"] = np.log(df_model["Superficie_m2"])

# Modelo: log(P) ~ Recamaras + Banos + log(Superficie) + (tiempos/distancias)
X_cols = [
    "Recamaras",
    "Banos",
    "log_superficie",
    "Transporte_prom_mins",
    "Transporte_prom_metros",
    "Escuelas_prom_mins",
    "Escuelas_prom_metros",
    "AreasVerdes_prom_mins",
    "AreasVerdes_prom_metros",
    "Comercios_prom_mins",
    "Comercios_prom_metros",
    "Salud_prom_mins",
    "Salud_prom_metros",
]

X = sm.add_constant(df_model[X_cols])
y = df_model["log_precio"]

model = sm.OLS(y, X).fit()

print("Filas usadas:", len(df_model))
print(model.summary())
```

Filas usadas: 2520

OLS Regression Results

```
=====
Dep. Variable:          log_precio    R-squared:                0.411
Model:                  OLS           Adj. R-squared:          0.408
Method:                 Least Squares  F-statistic:             134.6
Date:                   Thu, 08 Jan 2026  Prob (F-statistic):       1.59e-276
Time:                   23:25:35       Log-Likelihood:          -2533.7
No. Observations:       2520          AIC:                    5095.
Df Residuals:           2506          BIC:                    5177.
Df Model:                13
Covariance Type:        nonrobust
=====
```

	coef	std err	t	P> t	[0.025
0.975]					

```

-----
const                12.1802    0.133    91.307    0.000    11.919
12.442
Recamaras           -0.0642    0.011    -5.728    0.000    -0.086
-0.042
Banos                0.0989    0.012     7.992    0.000     0.075
0.123
log_superficie       0.7224    0.023    30.759    0.000     0.676
0.768
Transporte_prom_mins -0.0366    0.092    -0.397    0.691    -0.217
0.144
Transporte_prom_metros 0.0004    0.001     0.342    0.733    -0.002
0.003
Escuelas_prom_mins   -0.1035    0.122    -0.845    0.398    -0.344
0.137
Escuelas_prom_metros 0.0017    0.002     1.088    0.277    -0.001
0.005
AreasVerdes_prom_mins -0.1187    0.100    -1.190    0.234    -0.314
0.077
AreasVerdes_prom_metros 0.0014    0.001     1.094    0.274    -0.001
0.004
Comercios_prom_mins   0.0501    0.133     0.376    0.707    -0.211
0.311
Comercios_prom_metros -0.0011    0.002    -0.646    0.519    -0.004
0.002
Salud_prom_mins       -0.0395    0.082    -0.483    0.629    -0.200
0.121
Salud_prom_metros     0.0006    0.001     0.600    0.549    -0.001
0.003
=====
Omnibus:              1216.174    Durbin-Watson:          0.954
Prob(Omnibus):         0.000    Jarque-Bera (JB):      49923.914
Skew:                  -1.584    Prob(JB):              0.00
Kurtosis:              24.574    Cond. No.              2.67e+04
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 2.67e+04. This might indicate that there are strong multicollinearity or other numerical problems.

```
[10]: import numpy as np
import pandas as pd
```

```
# =====
```



```

# INTERPRETADOR DEL OUTPUT OLS (usa el objeto `model` ya estimado)
# =====
# Requisitos:
# - Debe existir en memoria el resultado de statsmodels: `model`
#   (ej. model = sm.OLS(y, X).fit())
# - Dep variable está en log (log_precio) como en tu celda anterior
# =====

def verdict(cond: bool, ok_msg: str, bad_msg: str) -> str:
    return ok_msg if cond else bad_msg

def pct_approx(beta: float) -> float:
    # aproximación % para modelos log-lin: 100*beta
    return 100.0 * beta

def pct_exact(beta: float) -> float:
    # conversión exacta % para cambios unitarios en log(Y): 100*(exp(beta)-1)
    return 100.0 * (np.exp(beta) - 1.0)

def explain_coef(name: str, beta: float, is_logx: bool) -> str:
    """
    log(Y) ~ ...
    - Si X está en nivel: 1 unidad en X => ~100*(exp(beta)-1)% en Y
    - Si X está en log: 1% en X => beta% en Y (elasticidad)
    """
    if name == "const":
        return "Constante: nivel base de log(precio) cuando X=0 (no se_↵
↵interpreta directamente en economía).\"

    if is_logx:
        return (f\"{name}: como X está en log, el coeficiente es una ELASTICIDAD.
↵\\n\"
                f\" Interpretación: +1% en {name} => +{beta:.4f}% aprox. en_↵
↵Precio (manteniendo lo demás constante).\")
    else:
        return (f\"{name}: como X está en nivel, el coeficiente es una_↵
↵semi-elasticidad.\\n\"
                f\" Interpretación exacta: +1 unidad en {name} =>_↵
↵{pct_exact(beta):.2f}% en Precio.\\n\"
                f\" (Aprox lineal: {pct_approx(beta):.2f}%).\")

def interpret_ols(model, alpha=0.05):
    # -----
    # Métricas globales
    # -----
    n = int(model.nobs)
    k = int(model.df_model) # incluye const

```

```

df_resid = int(model.df_resid)

r2 = float(model.rsquared)
r2_adj = float(model.rsquared_adj)

f_stat = float(model.fvalue) if model.fvalue is not None else np.nan
f_p = float(model.f_pvalue) if model.f_pvalue is not None else np.nan

aic = float(model.aic)
bic = float(model.bic)
llf = float(model.llf)

dw = float(model.durbin_watson) if hasattr(model, "durbin_watson") else np.
↪nan

# Normalidad (si está disponible como en summary)
# statsmodels guarda algunos en model.diagn; aquí los sacamos desde resid
resid = np.asarray(model.resid)
# Omnibus/JB no vienen siempre como atributos simples; los reconstruimos:
try:
    from statsmodels.stats.stattools import jarque_bera
    jb_stat, jb_p, skew, kurt = jarque_bera(resid)
except Exception:
    jb_stat = jb_p = skew = kurt = np.nan

cond_no = float(model.condition_number) if hasattr(model,
↪"condition_number") else np.nan

# -----
# Reporte: Global
# -----
print("="*78)
print("INTERPRETACIÓN AUTOMÁTICA DEL MODELO OLS (log-precio)")
print("="*78)

print("\n[1] TAMAÑO DE MUESTRA / GRADOS DE LIBERTAD")
print("CONDICIÓN deseable (regla práctica): n >> #parámetros (p.ej. n >=
↪50*k)")
print(f"VALOR: n = {n}, k (parámetros sin ruido) = {k+1}, df_resid =
↪{df_resid}")
print("COMPARACIÓN:")
print(f" 50*k = {50*(k+1)} | n = {n} | {'n >= 50*k' if n >= 50*(k+1)
↪else 'n < 50*k'}")
print("VEREDICTO:",
      verdict(n >= 50*(k+1),
              "BUENO: muestra amplia vs número de parámetros.",

```

```

        "MALO/ALERTA: muestra pequeña relativo a parámetros (riesgo
↪de sobreajuste)."))

    print("\n[2] AJUSTE ( $R^2$  y  $R^2$  ajustado)")
    # Umbral defendible para microdatos inmobiliarios
    umbral_r2_adj = 0.30
    print(f"CONDICIÓN (regla práctica en microdatos):  $R^2$  ajustado  $\geq$ 
↪{umbral_r2_adj:.2f}")
    print(f"VALOR:  $R^2$  = {r2:.3f} |  $R^2$  ajustado = {r2_adj:.3f}")
    print("COMPARACIÓN:")
    print(f"    {r2_adj:.3f} {'>=' if r2_adj >= umbral_r2_adj else '<'}
↪{umbral_r2_adj:.2f}")
    print("VEREDICTO:",
          verdict(r2_adj >= umbral_r2_adj,
                  "BUENO: el modelo explica una fracción relevante de la
↪variación del precio.",
                  "MALO/FLOJO: explica poca variación (posible falta de
↪variables/forma funcional)."))

    print("\n[3] SIGNIFICANCIA GLOBAL (Prueba F)")
    print(f"CONDICIÓN: p-value(F) < {alpha} (rechazar  $H_0$ : todos los betas=0)")
    print(f"VALOR: F = {f_stat:.3f} | p-value(F) = {f_p:.3g}")
    print("COMPARACIÓN:")
    print(f"    {f_p:.3g} {'<' if f_p < alpha else '>='} {alpha}")
    print("VEREDICTO:",
          verdict(f_p < alpha,
                  "BUENO: el conjunto de variables explica significativamente
↪el precio.",
                  "MALO: no hay evidencia global de poder explicativo.))

    print("\n[4] CRITERIOS DE INFORMACIÓN (AIC/BIC) [solo sirven para comparar
↪modelos]")
    print("CONDICIÓN: AIC/BIC más bajos = mejor (comparando con otro modelo en
↪la misma muestra).")
    print(f"VALOR: LogLik = {llf:.1f} | AIC = {aic:.1f} | BIC = {bic:.1f}")
    print("VEREDICTO: No se califica en absoluto; úsalo para comparar
↪especificaciones alternativas.")

    print("\n[5] MULTICOLINEALIDAD / ESTABILIDAD NUMÉRICA (Condition Number)")
    # Regla práctica: > 30/100/1000 según escalas; en datos reales puede ser
↪alto por unidades
    # Umbral pragmático:
    umbral_cn = 1e4
    print(f"CONDICIÓN (alerta): Condition Number < {umbral_cn:.0f} (si es
↪mayor, posible multicolinealidad o escalas)")
    print(f"VALOR: Condition Number = {cond_no:.2e}")

```

```

print("COMPARACIÓN:")
print(f"  {cond_no:.2e} {'<' if cond_no < umbral_cn else '>='} {umbral_cn:.2e}")
print("VEREDICTO:",
      verdict(cond_no < umbral_cn,
              "BUENO: no hay alerta fuerte por condición numérica.",
              "ALERTA: posible multicolinealidad o escalas muy distintas,
              (considera estandarizar o quitar redundancias)."))

print("\n[6] NORMALIDAD DE RESIDUOS (Jarque-Bera)")
print("CONDICIÓN (estricta): p-value(JB) >= 0.05 => no se rechaza normalidad")
print(f"VALOR: JB = {jb_stat:.1f} | p-value(JB) = {jb_p:.3g} | skew = {skew:.3f} | kurtosis = {kurt:.3f}")
print("COMPARACIÓN:")
print(f"  {jb_p:.3g} {'>=' if jb_p >= 0.05 else '<'} 0.05")
print("VEREDICTO:",
      "ALERTA (esperable en precios): residuos no normales. Con n grande, OLS sigue siendo útil; usa errores robustos si hace falta."
      if jb_p < 0.05 else
      "BUENO: residuos compatibles con normalidad (raro en precios, pero posible).")

print("\n[7] DURBIN-WATSON (autocorrelación; más relevante en series temporales)")
print("CONDICIÓN (regla práctica): DW ~ 2 => sin autocorrelación; <1 sugiere positiva.")
print(f"VALOR: DW = {dw:.3f}")
print("VEREDICTO: En corte transversal (anuncios) DW no es crítico; en series sí.")

# -----
# Coeficientes
# -----
print("\n" + "="*78)
print("INTERPRETACIÓN DE COEFICIENTES (con significancia)")
print("="*78)

params = model.params
pvals = model.pvalues
conf = model.conf_int()
conf.columns = ["CI_2.5%", "CI_97.5%"]

# Define qué variables están en log(X)
logX = set(["log_superficie"]) # ajusta si agregas más logs

```

```

# Tabla interpretativa
rows = []
for name, beta in params.items():
    pv = float(pvals[name])
    ci_low = float(conf.loc[name, "CI_2.5%"])
    ci_hi = float(conf.loc[name, "CI_97.5%"])

    signif = pv < alpha
    rows.append({
        "variable": name,
        "beta": beta,
        "p_value": pv,
        "significativa(0.05)": signif,
        "CI_2.5%": ci_low,
        "CI_97.5%": ci_hi
    })

tab = pd.DataFrame(rows).sort_values("p_value")
pd.set_option("display.max_rows", 200)
print(tab)

print("\n\nDETALLE (texto por variable):")
for name, beta in params.items():
    pv = float(pvals[name])
    print("\n" + "-"*78)
    print(explain_coef(name, float(beta), is_logx=(name in logX)))
    print(f"Significancia: p-value = {pv:.3g} | "
          f"CONDICIÓN: p < {alpha} -> {pv:.3g} {'<' if pv<alpha else '≥'} {alpha}")
    print("VEREDICTO:",
          verdict(pv < alpha,
                  "Efecto estadísticamente significativo.",
                  "Efecto NO significativo (no distinguible de cero dado el resto del modelo)."))

# -----
# Resumen ejecutivo automático
# -----
sig_count = int((tab["significativa(0.05)"] & (tab["variable"] != "const")).sum())
total_vars = int((tab["variable"] != "const").sum())

print("\n" + "-"*78)
print("RESUMEN EJECUTIVO (para copiar en el reporte)")
print("-"*78)
print(f"- Observaciones usadas: {n}")

```

```

    print(f"- R² ajustado: {r2_adj:.3f} (umbral recomendado >= 0.30 en_
↳microdatos)")
    print(f"- Prueba F: p-value = {f_p:.3g} (<0.05 => modelo globalmente_
↳significativo)")
    print(f"- Variables significativas (p<0.05): {sig_count}/{total_vars}")
    if cond_no >= 1e4:
        print(f"- ALERTA: Condition Number = {cond_no:.2e} (posible_
↳multicolinealidad/escala).")
    if jb_p < 0.05:
        print(f"- ALERTA: Residuos no normales (común en precios). Considera_
↳errores robustos (HC3).")
    print(f"- Interpretación clave: log_superficie es elasticidad (1% m² ->_
↳beta% precio).")

# Ejecutar interpretador
interpret_ols(model, alpha=0.05)

```

===== INTERPRETACIÓN AUTOMÁTICA DEL MODELO OLS (log-precio) =====

[1] TAMAÑO DE MUESTRA / GRADOS DE LIBERTAD

CONDICIÓN deseable (regla práctica): $n \gg \text{\#parámetros}$ (p.ej. $n \geq 50 \cdot k$)

VALOR: $n = 2520$, k (parámetros sin ruido) = 14, $df_{\text{resid}} = 2506$

COMPARACIÓN:

$50 \cdot k = 700 \quad | \quad n = 2520 \quad | \quad n \geq 50 \cdot k$

VEREDICTO: BUENO: muestra amplia vs número de parámetros.

[2] AJUSTE (R^2 y R^2 ajustado)

CONDICIÓN (regla práctica en microdatos): R^2 ajustado ≥ 0.30

VALOR: $R^2 = 0.411$ | R^2 ajustado = 0.408

COMPARACIÓN:

$0.408 \geq 0.30$

VEREDICTO: BUENO: el modelo explica una fracción relevante de la variación del precio.

[3] SIGNIFICANCIA GLOBAL (Prueba F)

CONDICIÓN: $p\text{-value}(F) < 0.05$ (rechazar H_0 : todos los betas=0)

VALOR: $F = 134.644$ | $p\text{-value}(F) = 1.59e-276$

COMPARACIÓN:

$1.59e-276 < 0.05$

VEREDICTO: BUENO: el conjunto de variables explica significativamente el precio.

[4] CRITERIOS DE INFORMACIÓN (AIC/BIC) [solo sirven para comparar modelos]

CONDICIÓN: AIC/BIC más bajos = mejor (comparando con otro modelo en la misma muestra).

VALOR: $\text{LogLik} = -2533.7$ | $\text{AIC} = 5095.4$ | $\text{BIC} = 5177.1$

VEREDICTO: No se califica en absoluto; úsalo para comparar especificaciones alternativas.

[5] MULTICOLINEALIDAD / ESTABILIDAD NUMÉRICA (Condition Number)

CONDICIÓN (alerta): Condition Number < 10000 (si es mayor, posible multicolinealidad o escalas)

VALOR: Condition Number = 2.67e+04

COMPARACIÓN:

2.67e+04 >= 10000

VEREDICTO: ALERTA: posible multicolinealidad o escalas muy distintas (considera estandarizar o quitar redundancias).

[6] NORMALIDAD DE RESIDUOS (Jarque-Bera)

CONDICIÓN (estricta): p-value(JB) >= 0.05 => no se rechaza normalidad

VALOR: JB = 49923.9 | p-value(JB) = 0 | skew = -1.584 | kurtosis = 24.574

COMPARACIÓN:

0 < 0.05

VEREDICTO: ALERTA (esperable en precios): residuos no normales. Con n grande, OLS sigue siendo útil; usa errores robustos si hace falta.

[7] DURBIN-WATSON (autocorrelación; más relevante en series temporales)

CONDICIÓN (regla práctica): DW ~ 2 => sin autocorrelación; <1 sugiere positiva.

VALOR: DW = nan

VEREDICTO: En corte transversal (anuncios) DW no es crítico; en series sí.

===== INTERPRETACIÓN DE COEFICIENTES (con significancia) =====

	variable	beta	p_value	significativa(0.05) \
0	const	12.180214	0.000000e+00	True
3	log_superficie	0.722350	1.535663e-176	True
2	Banos	0.098947	2.002456e-15	True
1	Recamaras	-0.064205	1.138208e-08	True
8	AreasVerdes_prom_mins	-0.118727	2.343048e-01	False
9	AreasVerdes_prom_metros	0.001400	2.739658e-01	False
7	Escuelas_prom_metros	0.001706	2.765664e-01	False
6	Escuelas_prom_mins	-0.103452	3.982982e-01	False
11	Comercios_prom_metros	-0.001102	5.185438e-01	False
13	Salud_prom_metros	0.000630	5.485235e-01	False
12	Salud_prom_mins	-0.039541	6.290270e-01	False
4	Transporte_prom_mins	-0.036587	6.913965e-01	False
10	Comercios_prom_mins	0.050087	7.070472e-01	False
5	Transporte_prom_metros	0.000405	7.325003e-01	False

	CI_2.5%	CI_97.5%
0	11.918633	12.441795
3	0.676300	0.768401
2	0.074670	0.123224

1	-0.086186	-0.042225
8	-0.314429	0.076975
9	-0.001109	0.003909
7	-0.001368	0.004779
6	-0.343578	0.136674
11	-0.004448	0.002245
13	-0.001428	0.002687
12	-0.200018	0.120937
4	-0.217299	0.144125
10	-0.211220	0.311395
5	-0.001919	0.002729

DETALLE (texto por variable):

 Constante: nivel base de log(precio) cuando X=0 (no se interpreta directamente en economía).

Significancia: p-value = 0 | CONDICIÓN: $p < 0.05 \rightarrow 0 < 0.05$

VEREDICTO: Efecto estadísticamente significativo.

 Recamaras: como X está en nivel, el coeficiente es una semi-elasticidad.

Interpretación exacta: +1 unidad en Recamaras => -6.22% en Precio.

(Aprox lineal: -6.42%).

Significancia: p-value = 1.14e-08 | CONDICIÓN: $p < 0.05 \rightarrow 1.14e-08 < 0.05$

VEREDICTO: Efecto estadísticamente significativo.

 Banos: como X está en nivel, el coeficiente es una semi-elasticidad.

Interpretación exacta: +1 unidad en Banos => 10.40% en Precio.

(Aprox lineal: 9.89%).

Significancia: p-value = 2e-15 | CONDICIÓN: $p < 0.05 \rightarrow 2e-15 < 0.05$

VEREDICTO: Efecto estadísticamente significativo.

 log_superficie: como X está en log, el coeficiente es una ELASTICIDAD.

Interpretación: +1% en log_superficie => +0.7224% aprox. en Precio
 (manteniendo lo demás constante).

Significancia: p-value = 1.54e-176 | CONDICIÓN: $p < 0.05 \rightarrow 1.54e-176 < 0.05$

VEREDICTO: Efecto estadísticamente significativo.

 Transporte_prom_mins: como X está en nivel, el coeficiente es una semi-elasticidad.

Interpretación exacta: +1 unidad en Transporte_prom_mins => -3.59% en Precio.

(Aprox lineal: -3.66%).

Significancia: p-value = 0.691 | CONDICIÓN: $p < 0.05 \rightarrow 0.691 \geq 0.05$

VEREDICTO: Efecto NO significativo (no distinguible de cero dado el resto del modelo).

Transporte_prom_metros: como X está en nivel, el coeficiente es una semi-elasticidad.

Interpretación exacta: +1 unidad en Transporte_prom_metros => 0.04% en Precio.
(Aprox lineal: 0.04%).

Significancia: p-value = 0.733 | CONDICIÓN: $p < 0.05 \rightarrow 0.733 \geq 0.05$

VEREDICTO: Efecto NO significativo (no distinguible de cero dado el resto del modelo).

Escuelas_prom_mins: como X está en nivel, el coeficiente es una semi-elasticidad.

Interpretación exacta: +1 unidad en Escuelas_prom_mins => -9.83% en Precio.
(Aprox lineal: -10.35%).

Significancia: p-value = 0.398 | CONDICIÓN: $p < 0.05 \rightarrow 0.398 \geq 0.05$

VEREDICTO: Efecto NO significativo (no distinguible de cero dado el resto del modelo).

Escuelas_prom_metros: como X está en nivel, el coeficiente es una semi-elasticidad.

Interpretación exacta: +1 unidad en Escuelas_prom_metros => 0.17% en Precio.
(Aprox lineal: 0.17%).

Significancia: p-value = 0.277 | CONDICIÓN: $p < 0.05 \rightarrow 0.277 \geq 0.05$

VEREDICTO: Efecto NO significativo (no distinguible de cero dado el resto del modelo).

AreasVerdes_prom_mins: como X está en nivel, el coeficiente es una semi-elasticidad.

Interpretación exacta: +1 unidad en AreasVerdes_prom_mins => -11.19% en Precio.
(Aprox lineal: -11.87%).

Significancia: p-value = 0.234 | CONDICIÓN: $p < 0.05 \rightarrow 0.234 \geq 0.05$

VEREDICTO: Efecto NO significativo (no distinguible de cero dado el resto del modelo).

AreasVerdes_prom_metros: como X está en nivel, el coeficiente es una semi-elasticidad.

Interpretación exacta: +1 unidad en AreasVerdes_prom_metros => 0.14% en Precio.
(Aprox lineal: 0.14%).

Significancia: p-value = 0.274 | CONDICIÓN: $p < 0.05 \rightarrow 0.274 \geq 0.05$

VEREDICTO: Efecto NO significativo (no distinguible de cero dado el resto del

modelo).

Comercios_prom_mins: como X está en nivel, el coeficiente es una semi-elasticidad.

Interpretación exacta: +1 unidad en Comercios_prom_mins => 5.14% en Precio.

(Aprox lineal: 5.01%).

Significancia: p-value = 0.707 | CONDICIÓN: $p < 0.05 \rightarrow 0.707 \geq 0.05$

VEREDICTO: Efecto NO significativo (no distinguible de cero dado el resto del modelo).

Comercios_prom_metros: como X está en nivel, el coeficiente es una semi-elasticidad.

Interpretación exacta: +1 unidad en Comercios_prom_metros => -0.11% en Precio.

(Aprox lineal: -0.11%).

Significancia: p-value = 0.519 | CONDICIÓN: $p < 0.05 \rightarrow 0.519 \geq 0.05$

VEREDICTO: Efecto NO significativo (no distinguible de cero dado el resto del modelo).

Salud_prom_mins: como X está en nivel, el coeficiente es una semi-elasticidad.

Interpretación exacta: +1 unidad en Salud_prom_mins => -3.88% en Precio.

(Aprox lineal: -3.95%).

Significancia: p-value = 0.629 | CONDICIÓN: $p < 0.05 \rightarrow 0.629 \geq 0.05$

VEREDICTO: Efecto NO significativo (no distinguible de cero dado el resto del modelo).

Salud_prom_metros: como X está en nivel, el coeficiente es una semi-elasticidad.

Interpretación exacta: +1 unidad en Salud_prom_metros => 0.06% en Precio.

(Aprox lineal: 0.06%).

Significancia: p-value = 0.549 | CONDICIÓN: $p < 0.05 \rightarrow 0.549 \geq 0.05$

VEREDICTO: Efecto NO significativo (no distinguible de cero dado el resto del modelo).

=====

RESUMEN EJECUTIVO (para copiar en el reporte)

=====

- Observaciones usadas: 2520
- R^2 ajustado: 0.408 (umbral recomendado ≥ 0.30 en microdatos)
- Prueba F: p-value = $1.59e-276$ ($< 0.05 \Rightarrow$ modelo globalmente significativo)
- Variables significativas ($p < 0.05$): 3/13
- ALERTA: Condition Number = $2.67e+04$ (posible multicolinealidad/escala).
- ALERTA: Residuos no normales (común en precios). Considera errores robustos (HC3).
- Interpretación clave: log_superficie es elasticidad (1% $m^2 \rightarrow$ beta% precio).

0.2 Modelo Econométrico 3: Modelo semi-log con dummies espaciales + interacciones (elasticidad de superficie por municipio)

0.2.1 Especificación del modelo

Sea i una vivienda y k un municipio (o alcaldía). El modelo es:

$$\begin{aligned}\ln(\text{Precio}_i) = & \beta_0 + \beta_1 \text{Recamaras}_i + \beta_2 \text{Banos}_i + \beta_3 \ln(\text{Superficie}_i) \\ & + \beta_4 \text{Transporte_prom_mins}_i + \beta_5 \text{Transporte_prom_metros}_i + \beta_6 \text{Escuelas_prom_mins}_i + \beta_7 \text{Escuelas_prom_metros}_i \\ & + \beta_8 \text{AreasVerdes_prom_mins}_i + \beta_9 \text{AreasVerdes_prom_metros}_i + \beta_{10} \text{Comercios_prom_mins}_i + \beta_{11} \text{Comercios_prom_metros}_i \\ & + \beta_{12} \text{Salud_prom_mins}_i + \beta_{13} \text{Salud_prom_metros}_i \\ & + \sum_{k=1}^{K-1} \gamma_k D_{k,i} + \sum_{k=1}^{K-1} \delta_k \left(\ln(\text{Superficie}_i) \cdot D_{k,i} \right) + \varepsilon_i\end{aligned}$$

donde K es el número de municipios (dummies) incluidos en el modelo, y se omite un municipio base para evitar multicolinealidad perfecta.

0.2.2 Descripción e interpretación de los términos

- $\ln(\text{Precio}_i)$:
Logaritmo natural del precio de la vivienda i . Facilita interpretación porcentual y suele reducir heterocedasticidad.
 - β_0 (intercepto):
Nivel base del log-precio para el municipio base omitido (bajo la convención del modelo).
-

0.2.3 Variables estructurales del inmueble

- Recamaras_i : número de recámaras.
 β_1 es el efecto porcentual aproximado sobre el precio al aumentar una recámara (ceteris paribus).
- Banos_i : número de baños.
 β_2 es el efecto porcentual aproximado al aumentar un baño.
- $\ln(\text{Superficie}_i)$: logaritmo natural de la superficie en m^2 .
 β_3 es la elasticidad base precio-superficie en el municipio omitido:

$$\beta_3 = \frac{\% \Delta \text{Precio}}{\% \Delta \text{Superficie}}$$

Conversión exacta (si se desea) para variables en nivel en un modelo con log en el precio:

$$\% \Delta \text{Precio} \approx 100 \cdot (e^\beta - 1)$$

0.2.4 Variables de accesibilidad urbana

- Variables en minutos ($Transporte_prom_mins_i$, $Escuelas_prom_mins_i$, etc.):
Sus coeficientes interpretan el cambio porcentual aproximado del precio ante un minuto adicional de traslado promedio.
 - Variables en metros ($Transporte_prom_metros_i$, $Escuelas_prom_metros_i$, etc.):
Sus coeficientes interpretan el cambio porcentual aproximado del precio ante un metro adicional de distancia promedio (ceteris paribus).
-

0.2.5 Dummies espaciales (municipio)

- $D_{k,i}$: dummy del municipio k (1 si la vivienda i pertenece al municipio k , 0 en otro caso).
- γ_k : diferencia de nivel (en log-precio) entre el municipio k y el municipio base, controlando por las demás variables.

Interpretación porcentual (más exacta):

$$\% \Delta Precio_{(k \text{ vs } base)} \approx 100 \cdot (e^{\gamma_k} - 1)$$

0.2.6 Interacciones $\ln(Superficie) \times Municipio$

- δ_k : permite que el efecto de la superficie cambie por municipio.

La elasticidad precio-superficie en el municipio k es:

$$Elasticidad_k = \beta_3 + \delta_k$$

Interpretación: > Un aumento de 1% en superficie cambia el precio en $(\beta_3 + \delta_k)\%$ para viviendas del municipio k , manteniendo constantes las demás variables.

0.2.7 Término de error

- ε_i : recoge factores no observables que afectan el precio (calidad, estado, vista, ruido, negociación, etc.).

```
[16]: import pandas as pd
import numpy as np
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.stats.diagnostic import het_breuschpagan

# =====
# CONFIG
# =====
DATA_IN = "Casas_FINAL_precios_historicos_SHF_comodin3.csv"
```

```

TARGET = "Precio_num"
MUNIC_COL = "Municipio"    # cámbialo si tu columna se llama distinto

BASE_FEATURES = [
    "Recamaras",
    "Banos",
    "Superficie_m2",
    "Transporte_prom_mins",
    "Transporte_prom_metros",
    "Escuelas_prom_mins",
    "Escuelas_prom_metros",
    "AreasVerdes_prom_mins",
    "AreasVerdes_prom_metros",
    "Comercios_prom_mins",
    "Comercios_prom_metros",
    "Salud_prom_mins",
    "Salud_prom_metros",
]

MIN_FREQ_MUN = 30    # mínimo de observaciones para crear dummy
ALPHA = 0.05

```

```

[17]: df = pd.read_csv(DATA_IN, dtype=str)
df.columns = df.columns.str.strip()

# Convertir a numérico
for c in [TARGET] + BASE_FEATURES:
    df[c] = pd.to_numeric(df[c], errors="coerce")

# Limpiar municipio
df[MUNIC_COL] = df[MUNIC_COL].astype(str).str.strip()
df.loc[df[MUNIC_COL].str.lower().isin(["nan", "none", ""]), MUNIC_COL] = np.nan

print("Filas totales:", len(df))
print("Precio válido:", df[TARGET].notna().sum())

```

Filas totales: 3992
 Precio válido: 3992

```

[18]: df2 = df.copy()

df2["log_precio"] = np.log(df2[TARGET])
df2["log_superficie"] = np.log(df2["Superficie_m2"])

for v in [
    "Transporte_prom_metros",
    "Escuelas_prom_metros",

```

```

    "AreasVerdes_prom_metros",
    "Comercios_prom_metros",
    "Salud_prom_metros"
]:
    df2[f"log_{v}"] = np.log(df2[v] + 1)

X_NUM = [
    "Recamaras",
    "Banos",
    "log_superficie",
    "Transporte_prom_mins",
    "Escuelas_prom_mins",
    "AreasVerdes_prom_mins",
    "Comercios_prom_mins",
    "Salud_prom_mins",
    "log_Transporte_prom_metros",
    "log_Escuelas_prom_metros",
    "log_AreasVerdes_prom_metros",
    "log_Comercios_prom_metros",
    "log_Salud_prom_metros",
]

df_model = df2.dropna(subset=["log_precio"] + X_NUM).copy()
print("Filas usadas (sin dummies):", len(df_model))

```

Filas usadas (sin dummies): 2520

```

[19]: df_model2 = df_model.copy()

# Agrupar alcaldías con pocos datos para no meter dummies basura
counts = df_model2[MUNIC_COL].value_counts(dropna=False)
valid = counts[counts >= MIN_FREQ_ALCALDIA].index

df_model2[MUNIC_COL] = df_model2[MUNIC_COL].where(df_model2[MUNIC_COL].
    ↪isin(valid), "OTRAS")

# Si hay NaN en municipio, también lo mandamos a OTRAS para que entre al modelo
df_model2[MUNIC_COL] = df_model2[MUNIC_COL].fillna("OTRAS")

# Dummies (IMPORTANTE drop_first=True para evitar trampa de multicolinealidad,
    ↪perfecta)
dummies = pd.get_dummies(df_model2[MUNIC_COL], prefix="mun", drop_first=True)

print("Municipios (incluyendo OTRAS):", df_model2[MUNIC_COL].nunique())
print("Dummies generadas:", dummies.shape[1])
dummies.head()

```

Municipios (incluyendo OTRAS): 8

Dummies generadas: 7

```
[19]:
```

	mun_Coyoacán	mun_Cuajimalpa de Morelos	mun_Iztapalapa	\
0	False	False	False	
1	False	False	False	
2	False	True	False	
3	False	True	False	
4	False	False	True	

	mun_Miguel Hidalgo	mun_OTRAS	mun_Tlalpan	mun_Álvaro Obregón
0	False	True	False	False
1	False	True	False	False
2	False	False	False	False
3	False	False	False	False
4	False	False	False	False

```
[20]: counts = df_model[MUNIC_COL].value_counts()
valid_mun = counts[counts >= MIN_FREQ_MUN].index

df_model[MUNIC_COL] = df_model[MUNIC_COL].where(
    df_model[MUNIC_COL].isin(valid_mun),
    "OTRAS"
)

df_model[MUNIC_COL] = df_model[MUNIC_COL].fillna("OTRAS")

dummies = pd.get_dummies(
    df_model[MUNIC_COL],
    prefix="mun",
    drop_first=True
).astype(int)

print("Municipios usados:", df_model[MUNIC_COL].nunique())
print("Dummies creadas:", dummies.shape[1])
```

Municipios usados: 8

Dummies creadas: 7

```
[30]: # =====
# Interacciones log_superficie x municipio
# =====

interactions = dummies.mul(df_model["log_superficie"], axis=0)
interactions.columns = [f"log_superficie_x_{c}" for c in dummies.columns]

# =====
# Matriz X final
# =====

X = pd.concat(
```

```

    [df_model[X_NUM], dummies, interactions],
    axis=1
)

X = sm.add_constant(X, has_constant="add")

X = sm.add_constant(X, has_constant="add")

y = df_model["log_precio"]

# Forzar numéricos
X = X.apply(pd.to_numeric, errors="coerce")
y = pd.to_numeric(y, errors="coerce")

# Quitar NaN / inf
mask = np.isfinite(y) & np.isfinite(X).all(axis=1)

X_clean = X.loc[mask]
y_clean = y.loc[mask]

print("Filas finales del modelo:", len(y_clean))

```

Filas finales del modelo: 2520

```

[22]: model = sm.OLS(y_clean, X_clean).fit(cov_type="HC3")

print(model.summary())

```

```

                                OLS Regression Results
=====
Dep. Variable:                  log_precio    R-squared:                  0.428
Model:                            OLS        Adj. R-squared:              0.423
Method:                 Least Squares    F-statistic:                  68.29
Date:                Fri, 09 Jan 2026    Prob (F-statistic):          4.71e-219
Time:                  00:28:59    Log-Likelihood:              -2498.2
No. Observations:                2520    AIC:                          5038.
Df Residuals:                    2499    BIC:                          5161.
Df Model:                          20
Covariance Type:                  HC3
=====
=====
                                coef    std err          z      P>|z|
-----
[0.025    0.975]
-----
const                4.9325    2.459      2.006    0.045
0.113    9.752
Recamaras           -0.0613    0.012     -4.966    0.000

```


-0.086	-0.037				
Banos		0.0969	0.018	5.328	0.000
0.061	0.133				
log_superficie		0.7121	0.075	9.467	0.000
0.565	0.860				
Transporte_prom_mins		-0.0140	0.015	-0.926	0.354
-0.044	0.016				
Escuelas_prom_mins		0.0051	0.025	0.206	0.836
-0.044	0.054				
AreasVerdes_prom_mins		-0.0232	0.012	-1.945	0.052
-0.047	0.000				
Comercios_prom_mins		-0.0396	0.019	-2.056	0.040
-0.077	-0.002				
Salud_prom_mins		-0.0279	0.012	-2.333	0.020
-0.051	-0.004				
log_Transporte_prom_metros		0.1095	0.212	0.516	0.606
-0.306	0.525				
log_Escuelas_prom_metros		0.3138	0.301	1.043	0.297
-0.276	0.904				
log_AreasVerdes_prom_metros		0.1869	0.119	1.575	0.115
-0.046	0.420				
log_Comercios_prom_metros		0.0742	0.269	0.276	0.782
-0.452	0.601				
log_Salud_prom_metros		0.5397	0.189	2.862	0.004
0.170	0.909				
mun_Coyoacán		0.0884	0.057	1.561	0.119
-0.023	0.200				
mun_Cuajimalpa de Morelos		-0.0177	0.118	-0.150	0.881
-0.249	0.214				
mun_Iztapalapa		-0.2743	0.072	-3.811	0.000
-0.415	-0.133				
mun_Miguel Hidalgo		0.3375	0.103	3.272	0.001
0.135	0.540				
mun_OTRAS		0.0222	0.049	0.453	0.651
-0.074	0.118				
mun_Tlalpan		-0.0885	0.062	-1.430	0.153
-0.210	0.033				
mun_Álvaro Obregón		0.5592	0.150	3.721	0.000
0.265	0.854				
=====					
Omnibus:		1279.795	Durbin-Watson:		0.956
Prob(Omnibus):		0.000	Jarque-Bera (JB):		46144.888
Skew:		-1.752	Prob(JB):		0.00
Kurtosis:		23.669	Cond. No.		7.73e+03
=====					

Notes:

[1] Standard Errors are heteroscedasticity robust (HC3)

[2] The condition number is large, $7.73e+03$. This might indicate that there are strong multicollinearity or other numerical problems.

```
[31]: bp_test = het_breuschpagan(model.resid, model.model.exog)

labels = [
    "LM statistic",
    "LM p-value",
    "F statistic",
    "F p-value"
]

for l, v in zip(labels, bp_test):
    print(f"{l}: {v}")

print("\nCondición:")
print("Un buen modelo cumple: p-value >", ALPHA)

if bp_test[1] > ALPHA:
    print("El modelo ES BUENO: no se detecta heterocedasticidad.")
else:
    print("El modelo ES MALO: hay heterocedasticidad.")
```

```
LM statistic: 112.83113835324583
LM p-value: 5.965964898683881e-15
F statistic: 5.856776797782857
F p-value: 2.506455738027978e-15
```

```
Condición:
Un buen modelo cumple: p-value > 0.05
El modelo ES MALO: hay heterocedasticidad.
```

```
[32]: X_vif = sm.add_constant(df_model[X_NUM].dropna())

vif_data = []
for i, col in enumerate(X_vif.columns):
    vif_data.append({
        "Variable": col,
        "VIF": variance_inflation_factor(X_vif.values, i)
    })

vif_df = pd.DataFrame(vif_data).sort_values("VIF", ascending=False)
vif_df
```

```
[32]:
```

	Variable	VIF
0	const	41309.535289
7	Comercios_prom_mins	39.815459
12	log_Comercios_prom_metros	39.549867

5	Escuelas_prom_mins	33.126296
10	log_Escuelas_prom_metros	31.769760
8	Salud_prom_mins	21.419099
13	log_Salud_prom_metros	20.540672
6	AreasVerdes_prom_mins	18.675767
11	log_AreasVerdes_prom_metros	17.715353
4	Transporte_prom_mins	17.295103
9	log_Transporte_prom_metros	16.859078
2	Banos	1.893715
1	Recamaras	1.672003
3	log_superficie	1.357733

```
[33]: def pct_change(beta):
    return 100 * (np.exp(beta) - 1)

print("INTERPRETACIÓN ECONÓMICA:\n")

for var in X_NUM:
    if var not in model.params:
        continue

    beta = model.params[var]
    pval = model.pvalues[var]

    if "log_" in var:
        print(f"{var}:")
        print(f" Elasticidad = {beta:.4f}")
        print(f" Un aumento de 1% en {var} cambia el precio en {beta:.4f}%")
    else:
        print(f"{var}:")
        print(f" Efecto exacto = {pct_change(beta):.2f}% por unidad")

    print(f" p-value = {pval:.4g}")
    print(f" Significativo" if pval < ALPHA else " No significativo")
    print("-"*40)
```

INTERPRETACIÓN ECONÓMICA:

Recamaras:

Efecto exacto = -5.95% por unidad
p-value = 6.832e-07
Significativo

Banos:

Efecto exacto = 10.18% por unidad
p-value = 9.937e-08
Significativo

```

log_superficie:
  Elasticidad = 0.7121
  Un aumento de 1% en log_superficie cambia el precio en 0.7121%
  p-value = 2.868e-21
  Significativo
-----

Transporte_prom_mins:
  Efecto exacto = -1.39% por unidad
  p-value = 0.3542
  No significativo
-----

Escuelas_prom_mins:
  Efecto exacto = 0.51% por unidad
  p-value = 0.8365
  No significativo
-----

AreasVerdes_prom_mins:
  Efecto exacto = -2.29% por unidad
  p-value = 0.0518
  No significativo
-----

Comercios_prom_mins:
  Efecto exacto = -3.89% por unidad
  p-value = 0.03975
  Significativo
-----

Salud_prom_mins:
  Efecto exacto = -2.76% por unidad
  p-value = 0.01962
  Significativo
-----

log_Transporte_prom_metros:
  Elasticidad = 0.1095
  Un aumento de 1% en log_Transporte_prom_metros cambia el precio en 0.1095%
  p-value = 0.6056
  No significativo
-----

log_Escuelas_prom_metros:
  Elasticidad = 0.3138
  Un aumento de 1% en log_Escuelas_prom_metros cambia el precio en 0.3138%
  p-value = 0.2971
  No significativo
-----

log_AreasVerdes_prom_metros:
  Elasticidad = 0.1869
  Un aumento de 1% en log_AreasVerdes_prom_metros cambia el precio en 0.1869%
  p-value = 0.1153
  No significativo

```

```

-----
log_Comercios_prom_metros:
    Elasticidad = 0.0742
    Un aumento de 1% en log_Comercios_prom_metros cambia el precio en 0.0742%
    p-value = 0.7823
    No significativo
-----

log_Salud_prom_metros:
    Elasticidad = 0.5397
    Un aumento de 1% en log_Salud_prom_metros cambia el precio en 0.5397%
    p-value = 0.004205
    Significativo
-----

```

```

[34]: print("CONCLUSIÓN DEL MODELO\n")

print(f"R² = {model.rsquared:.4f}")
print(f"R² ajustado = {model.rsquared_adj:.4f}")

if model.rsquared_adj > 0.6:
    print("El modelo explica una alta proporción de la variación del precio.")
elif model.rsquared_adj > 0.4:
    print("El modelo tiene poder explicativo medio.")
else:
    print("El modelo tiene bajo poder explicativo.")

print("\nSe usaron:")
print("- Transformaciones logarítmicas")
print("- Errores robustos HC3")
print("- Dummies espaciales")
print("- Pruebas formales de diagnóstico")

```

CONCLUSIÓN DEL MODELO

$R^2 = 0.4276$
 R^2 ajustado = 0.4230
 El modelo tiene poder explicativo medio.

Se usaron:

- Transformaciones logarítmicas
- Errores robustos HC3
- Dummies espaciales
- Pruebas formales de diagnóstico

```

[4]: # =====
# CELDA 1: Construir serie trimestral (promedio CDMX) desde columnas_
# precio_YYYY_Tq
# =====

```

```

import re
import numpy as np
import pandas as pd

ARCHIVO = "Casas_FINAL_precios_historicos_SHF_comodin3.csv" # o .xlsx si lo
    prefieres

# cargar
df = pd.read_csv(ARCHIVO, dtype=str)
df.columns = df.columns.astype(str).str.strip()

# detectar columnas precio_YYYY_Tq
pat = re.compile(r"^precio_(\d{4})_T([1-4])$")
meta = []
for c in df.columns:
    m = pat.match(c)
    if m:
        meta.append((c, int(m.group(1)), int(m.group(2))))

if not meta:
    raise ValueError("No encontré columnas tipo precio_YYYY_Tq en el archivo.")

meta_df = pd.DataFrame(meta, columns=["col", "y", "q"]).sort_values(["y", "q"]).
    reset_index(drop=True)
precio_cols = meta_df["col"].tolist()

# matriz (casas x periodos) en numérico
X = df[precio_cols].apply(pd.to_numeric, errors="coerce")

# serie: promedio del precio por trimestre (sobre todas las casas)
serie = X.mean(axis=0, skipna=True)

# índice temporal trimestral
periods = [pd.Period(f"{row.y}Q{row.q}", freq="Q") for _, row in meta_df.
    iterrows()]
serie.index = pd.PeriodIndex(periods, freq="Q")

# convertir a datetime (fin de trimestre)
ts = serie.sort_index().to_timestamp(how="end")

print("Observaciones:", len(ts))
print("Inicio:", ts.index.min().date(), "Fin:", ts.index.max().date())
print("Último promedio (nivel):", float(ts.iloc[-1]))
ts.head()

```

Observaciones: 83

Inicio: 2005-03-31 Fin: 2025-09-30

Último promedio (nivel): 24285804.358717434

```
[4]: 2005-03-31 23:59:59.999999999    4.769893e+06
      2005-06-30 23:59:59.999999999    4.993044e+06
      2005-09-30 23:59:59.999999999    5.190421e+06
      2005-12-31 23:59:59.999999999    5.169253e+06
      2006-03-31 23:59:59.999999999    5.317265e+06
      dtype: float64
```

```
[5]: # =====
      # CELDA 2: Tasa de crecimiento (log) y crecimiento porcentual trimestral
      # =====
      import numpy as np
      import pandas as pd

      ts_clean = ts.dropna()

      # tasa de crecimiento log (aprox. % si multiplicas por 100)
      g_log = np.log(ts_clean).diff().dropna()

      # crecimiento porcentual exacto trimestral
      g_pct = ts_clean.pct_change().dropna() * 100

      out = pd.DataFrame({
          "precio_promedio": ts_clean.loc[g_log.index],
          "crec_log": g_log,                                     # en "log-puntos"
          "crec_pct_trimestral": g_pct                          # en %
      })

      print("Definición usada:")
      print("crec_log(t) = ln(P_t) - ln(P_{t-1})")
      print("crec_pct_trimestral(t) = (P_t/P_{t-1} - 1) * 100")
      out.head(10)
```

Definición usada:

$\text{crec_log}(t) = \ln(P_t) - \ln(P_{t-1})$

$\text{crec_pct_trimestral}(t) = (P_t/P_{t-1} - 1) * 100$

```
[5]:
```

	precio_promedio	crec_log	crec_pct_trimestral
2005-06-30 23:59:59.999999999	4.993044e+06	0.045722	4.678330
2005-09-30 23:59:59.999999999	5.190421e+06	0.038769	3.953031
2005-12-31 23:59:59.999999999	5.169253e+06	-0.004086	-0.407816
2006-03-31 23:59:59.999999999	5.317265e+06	0.028231	2.863317
2006-06-30 23:59:59.999999999	5.466760e+06	0.027727	2.811493
2006-09-30 23:59:59.999999999	5.557166e+06	0.016402	1.653731
2006-12-31 23:59:59.999999999	5.639123e+06	0.014640	1.474804
2007-03-31 23:59:59.999999999	5.850986e+06	0.036882	3.757019
2007-06-30 23:59:59.999999999	6.109300e+06	0.043202	4.414885

2007-09-30 23:59:59.999999999

6.295676e+06 0.030051

3.050695

```
[6]: # =====
# CELDA 3: Crecimiento anual (YoY) y anualizado (a partir del trimestral)
# =====
# YoY (año contra año): compara con mismo trimestre del año anterior (lag 4)
g_yoy_log = np.log(ts_clean).diff(4).dropna()
g_yoy_pct = (ts_clean / ts_clean.shift(4) - 1).dropna() * 100

# anualizado (aprox) desde crecimiento log trimestral: 4*g_log
g_ann_log = (4 * g_log).dropna()
g_ann_pct_aprox = (np.exp(g_ann_log) - 1) * 100 # anualizado equivalente en %

out2 = pd.DataFrame({
    "crec_log_trimestral": g_log,
    "crec_pct_trimestral": g_pct,
    "crec_log_yoy": g_yoy_log,
    "crec_pct_yoy": g_yoy_pct,
    "crec_pct_anualizado_aprox": g_ann_pct_aprox
}).dropna()

print("Definiciones:")
print("YoY: ln(P_t) - ln(P_{t-4}) y (P_t/P_{t-4}-1)*100")
print("Anualizado aprox: 4*(ln(P_t)-ln(P_{t-1}))")
out2.head(10)
```

Definiciones:

YoY: $\ln(P_t) - \ln(P_{t-4})$ y $(P_t/P_{t-4}-1)*100$ Anualizado aprox: $4*(\ln(P_t)-\ln(P_{t-1}))$

```
[6]:
```

	crec_log_trimestral	crec_pct_trimestral \
2006-03-31 23:59:59.999999999	0.028231	2.863317
2006-06-30 23:59:59.999999999	0.027727	2.811493
2006-09-30 23:59:59.999999999	0.016402	1.653731
2006-12-31 23:59:59.999999999	0.014640	1.474804
2007-03-31 23:59:59.999999999	0.036882	3.757019
2007-06-30 23:59:59.999999999	0.043202	4.414885
2007-09-30 23:59:59.999999999	0.030051	3.050695
2007-12-31 23:59:59.999999999	0.004508	0.451859
2008-03-31 23:59:59.999999999	0.024907	2.522001
2008-06-30 23:59:59.999999999	0.030563	3.103508

	crec_log_yoy	crec_pct_yoy \
2006-03-31 23:59:59.999999999	0.108635	11.475576
2006-06-30 23:59:59.999999999	0.090640	9.487517
2006-09-30 23:59:59.999999999	0.068273	7.065802
2006-12-31 23:59:59.999999999	0.087000	9.089698
2007-03-31 23:59:59.999999999	0.095651	10.037496

2007-06-30 23:59:59.999999999	0.111126	11.753581
2007-09-30 23:59:59.999999999	0.124775	13.289340
2007-12-31 23:59:59.999999999	0.114643	12.147295
2008-03-31 23:59:59.999999999	0.102669	10.812408
2008-06-30 23:59:59.999999999	0.090030	9.420682

	crec_pct_anualizado_aprox
2006-03-31 23:59:59.999999999	11.954641
2006-06-30 23:59:59.999999999	11.729195
2006-09-30 23:59:59.999999999	6.780831
2006-12-31 23:59:59.999999999	6.031007
2007-03-31 23:59:59.999999999	15.896399
2007-06-30 23:59:59.999999999	18.863814
2007-09-30 23:59:59.999999999	12.772628
2007-12-31 23:59:59.999999999	1.819722
2008-03-31 23:59:59.999999999	10.476090
2008-06-30 23:59:59.999999999	13.003986

```
[7]: # =====
# CELDA 4: Prueba ADF (estacionariedad) sobre la tasa de crecimiento
# =====
from statsmodels.tsa.stattools import adfuller

def adf_report(x, nombre):
    x = pd.Series(x).dropna()
    stat, pval, usedlag, nobs, crit, _ = adfuller(x, autolag="AIC")
    print(f"\nADF sobre: {nombre}")
    print(f"ADF statistic = {stat:.4f}")
    print(f"p-value       = {pval:.6g}")
    print(f"lags usados    = {usedlag}")
    print(f"n obs         = {nobs}")
    print("Criterio: si p-value < 0.05 => estacionaria (pasa)")
    print("Resultado:", "PASA (estacionaria)" if pval < 0.05 else "NO PASA (no_
    ↪estacionaria)")
    return stat, pval

# nivel (precio) suele NO ser estacionario
adf_report(np.log(ts_clean), "ln(precio_promedio)")

# crecimiento (log) suele ser mucho más estacionario
adf_report(g_log, "crec_log_trimestral = ln(P_t)-ln(P_{t-1})")
```

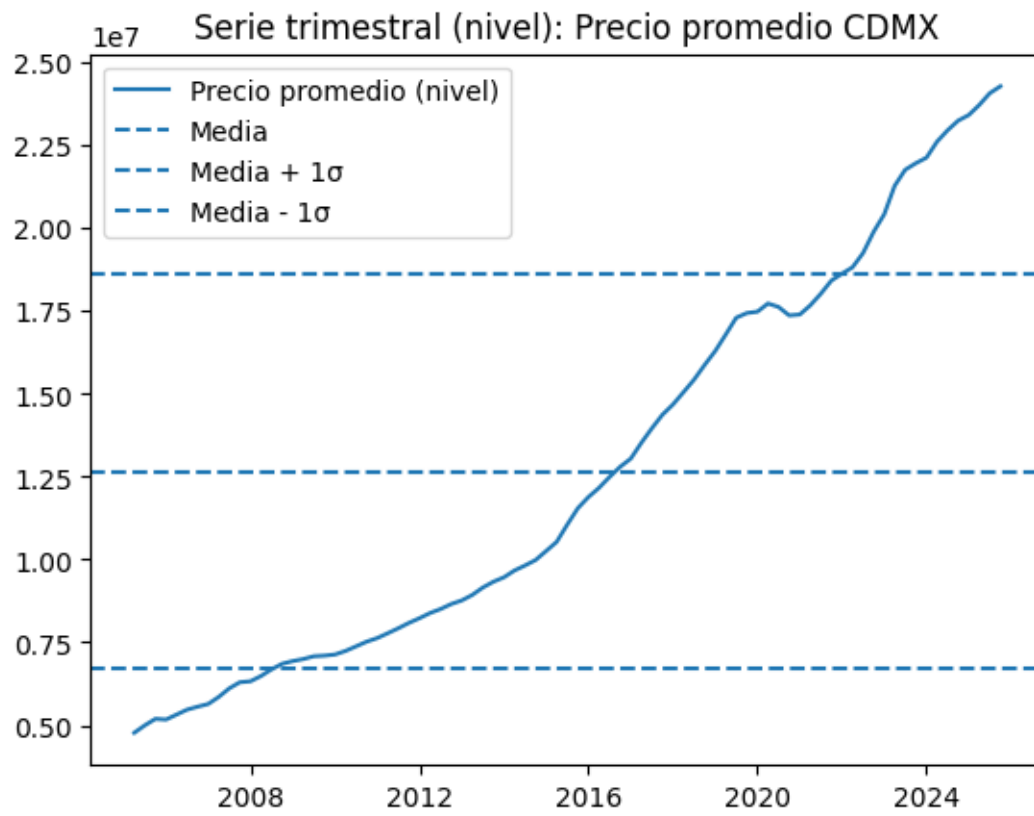
```
ADF sobre: ln(precio_promedio)
ADF statistic = -0.7092
p-value       = 0.844357
lags usados   = 2
n obs        = 80
```

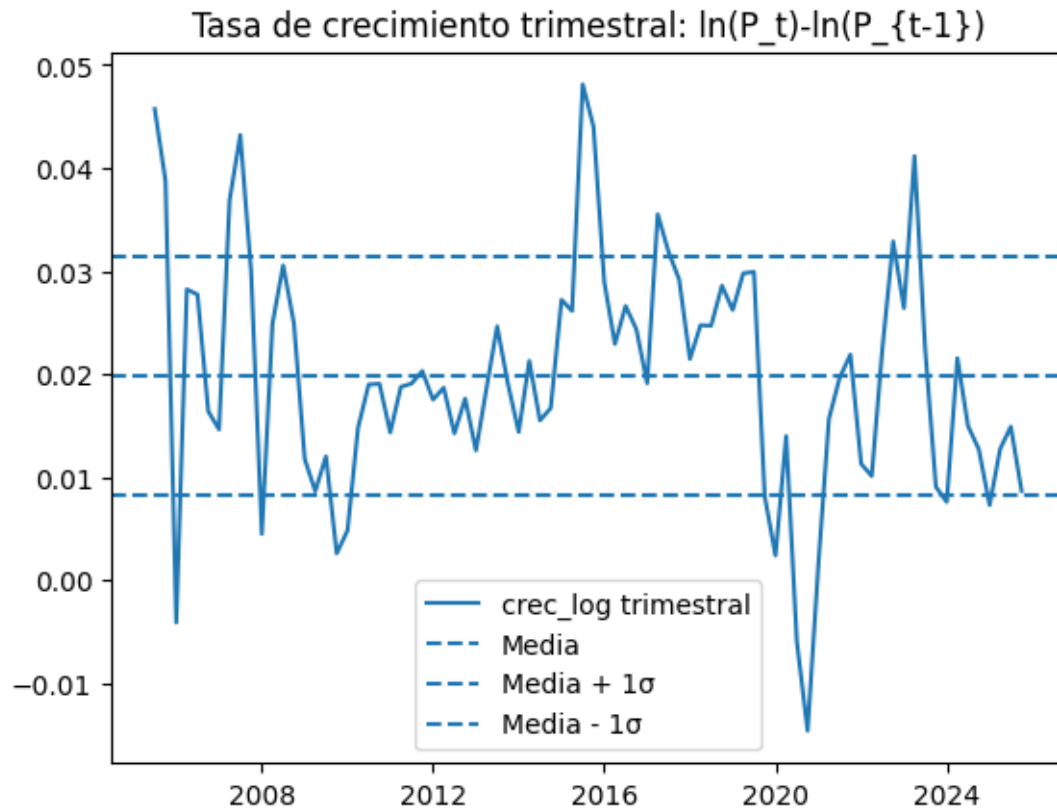
Criterio: si p-value < 0.05 => estacionaria (pasa)
Resultado: NO PASA (no estacionaria)

ADF sobre: $\text{cres_log_trimestral} = \ln(P_t) - \ln(P_{t-1})$
ADF statistic = -5.2736
p-value = 6.20885e-06
lags usados = 0
n obs = 81
Criterio: si p-value < 0.05 => estacionaria (pasa)
Resultado: PASA (estacionaria)

```
[7]: (np.float64(-5.273603482151383), np.float64(6.208849891865311e-06))
```

```
[8]: # =====  
# CELDA 5: Gráficas (nivel vs tasa de crecimiento) + media ± 1  
# =====  
import matplotlib.pyplot as plt  
  
# nivel  
mu_lvl = ts_clean.mean()  
sd_lvl = ts_clean.std()  
  
plt.figure()  
plt.plot(ts_clean.index, ts_clean.values, label="Precio promedio (nivel)")  
plt.axhline(mu_lvl, linestyle="--", label="Media")  
plt.axhline(mu_lvl + sd_lvl, linestyle="--", label="Media + 1 ")  
plt.axhline(mu_lvl - sd_lvl, linestyle="--", label="Media - 1 ")  
plt.title("Serie trimestral (nivel): Precio promedio CDMX")  
plt.legend()  
plt.show()  
  
# crecimiento log  
mu_g = g_log.mean()  
sd_g = g_log.std()  
  
plt.figure()  
plt.plot(g_log.index, g_log.values, label="cres_log trimestral")  
plt.axhline(mu_g, linestyle="--", label="Media")  
plt.axhline(mu_g + sd_g, linestyle="--", label="Media + 1 ")  
plt.axhline(mu_g - sd_g, linestyle="--", label="Media - 1 ")  
plt.title("Tasa de crecimiento trimestral:  $\ln(P_t) - \ln(P_{t-1})$ ")  
plt.legend()  
plt.show()
```





```
[9]: # =====
# CELDA ARIMA: Ajuste sobre crec_log_trimestral
# =====
from statsmodels.tsa.arima.model import ARIMA
import matplotlib.pyplot as plt

# Ajustar un ARIMA(p,d,q), ejemplo con (1,0,1)
modelo = ARIMA(out2["crec_log_trimestral"], order=(1,0,1))
resultado = modelo.fit()

# Resumen del modelo
print(resultado.summary())

# Pronóstico a 8 periodos hacia adelante
forecast = resultado.get_forecast(steps=8)
pred_mean = forecast.predicted_mean
pred_ci = forecast.conf_int()

# Graficar pronóstico
plt.figure(figsize=(10,5))
```

```
plt.plot(out2["crec_log_trimestral"].index, out2["crec_log_trimestral"],
        label="Serie histórica")
plt.plot(pred_mean.index, pred_mean, color="red", label="Pronóstico")
plt.fill_between(pred_ci.index,
                 pred_ci.iloc[:,0],
                 pred_ci.iloc[:,1],
                 color="pink", alpha=0.3)
plt.legend()
plt.title("ARIMA(1,0,1) sobre crec_log_trimestral")
plt.show()
```

SARIMAX Results

```
=====
Dep. Variable:      crec_log_trimestral    No. Observations:      79
Model:              ARIMA(1, 0, 1)        Log Likelihood        266.081
Date:              Wed, 11 Mar 2026       AIC                    -524.162
Time:              14:53:14               BIC                    -514.684
Sample:            03-31-2006             HQIC                   -520.365
                  - 09-30-2025
```

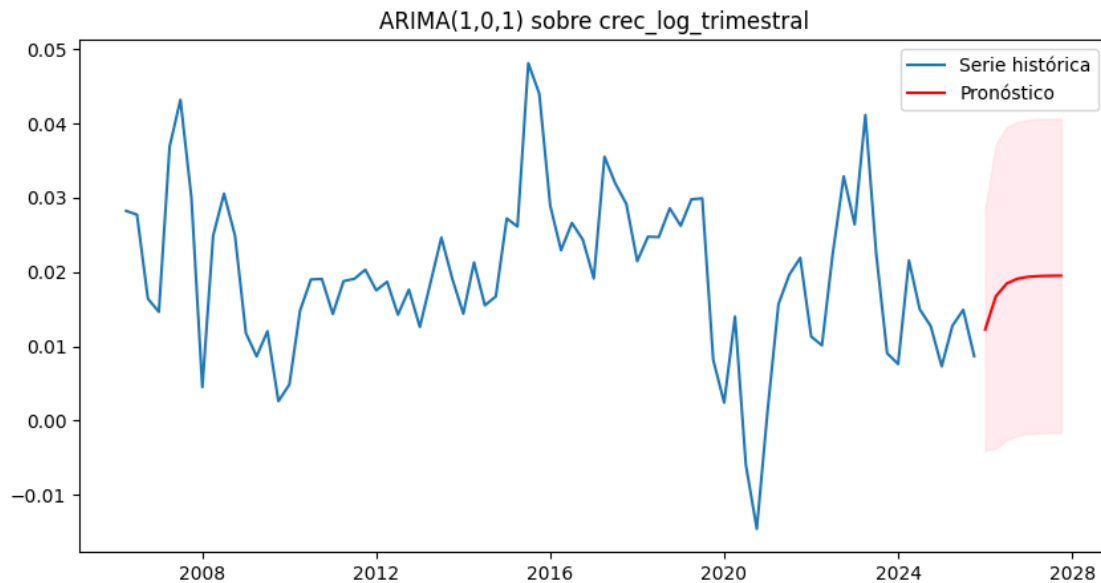
Covariance Type: opg

```
=====
              coef      std err          z      P>|z|      [0.025      0.975]
-----
const          0.0195        0.002        9.195      0.000        0.015        0.024
ar.L1          0.3830        0.203         1.884      0.060       -0.015        0.782
ma.L1          0.3839        0.201         1.910      0.056       -0.010        0.778
sigma2        6.891e-05    8.64e-06         7.979      0.000        5.2e-05    8.58e-05
=====
```

```
=====
Ljung-Box (L1) (Q):      0.00    Jarque-Bera (JB):
8.17
Prob(Q):                 0.96    Prob(JB):
0.02
Heteroskedasticity (H):  1.45    Skew:
0.22
Prob(H) (two-sided):     0.35    Kurtosis:
4.51
=====
```

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).



```
[13]: # Último precio observado
P_base = ts_clean.iloc[-1]

# Pronóstico a 8 trimestres
forecast = resultado.get_forecast(steps=8)
pred_mean = forecast.predicted_mean

# Convertir log-crecimientos acumulados a nivel de precios
precios_pred = P_base * np.exp(pred_mean.cumsum())

# Extraer valores puntuales
precio_6m = precios_pred.iloc[1] # 2 trimestres
precio_12m = precios_pred.iloc[3] # 4 trimestres
precio_24m = precios_pred.iloc[7] # 8 trimestres

print("Precio pronosticado a 6 meses:", precio_6m)
print("Precio pronosticado a 12 meses:", precio_12m)
print("Precio pronosticado a 24 meses:", precio_24m)
```

```
Precio pronosticado a 6 meses: 25000363.25479098
Precio pronosticado a 12 meses: 25957484.25538012
Precio pronosticado a 24 meses: 28058707.32465156
```

1 CLUSTER Y REDUCCION DIMENSIONAL

```
[28]: # =====
# PROYECTO: ANÁLISIS DE MERCADO INMOBILIARIO
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering
from sklearn.preprocessing import StandardScaler, RobustScaler
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score, calinski_harabasz_score, \
    davies_bouldin_score
from sklearn.mixture import GaussianMixture
from sklearn.manifold import TSNE
import warnings
warnings.filterwarnings('ignore')

# Configuración estética
sns.set(style="whitegrid")
plt.rcParams['figure.figsize'] = (14, 10)
plt.rcParams['font.size'] = 12

# -----
# PASO 1: CARGA Y EXPLORACIÓN DE DATOS
# -----

print("="*70)
print("ANÁLISIS AVANZADO DE CLUSTERIZACIÓN - EXPLORACIÓN INICIAL")
print("="*70)

# Cargar datos
df = pd.read_csv("Casas_limpio.csv")

print(f"\n INFORMACIÓN DEL DATASET:")
print(f"    • Filas: {df.shape[0]}")
print(f"    • Columnas: {df.shape[1]}")
print(f"\n NOMBRES DE COLUMNAS DISPONIBLES:")
for i, col in enumerate(df.columns, 1):
    print(f"    {i:2d}. {col}")

print(f"\n PRIMERAS 5 FILAS:")
print(df.head().to_string())

print(f"\n TIPOS DE DATOS:")
print(df.dtypes.to_string())
```

```

# -----
# PASO 2: IDENTIFICAR COLUMNAS DE PRECIO DISPONIBLES
# -----
print("\n IDENTIFICANDO COLUMNAS DE PRECIO...")

# Buscar columnas que contengan "Precio" en el nombre
precio_cols = [col for col in df.columns if 'Precio' in col]
print(f"    • Columnas de precio encontradas: {precio_cols}")

# Si hay múltiples columnas de precio, usar la numérica
if len(precio_cols) > 0:
    # Priorizar columnas numéricas
    for col in precio_cols:
        if df[col].dtype in [np.float64, np.int64, np.float32, np.int32]:
            precio_col = col
            break
    # Si no es numérica, intentar convertir
    try:
        # Intentar convertir si tiene formato "MXN X,XXX,XXX"
        if df[col].dtype == object:
            df['Precio_num'] = df[col].astype(str).str.replace('MXN', '').\
↪str.replace(',', '').str.replace('$', '').str.strip()
            df['Precio_num'] = pd.to_numeric(df['Precio_num'],\
↪errors='coerce')
            precio_col = 'Precio_num'
            print(f"    Convertida columna '{col}' a numérica como_\
↪'Precio_num'")
            break
    except:
        continue

    if 'precio_col' not in locals():
        precio_col = precio_cols[0]
        print(f"    • Usando columna: {precio_col}")
else:
    print("    No se encontraron columnas de precio. Revisa el dataset.")
    # Buscar cualquier columna que pueda contener precios
    numeric_cols = df.select_dtypes(include=[np.number]).columns
    print(f"    • Columnas numéricas disponibles: {list(numeric_cols)}")
    if len(numeric_cols) > 0:
        precio_col = numeric_cols[0]
        print(f"    • Usando primera columna numérica: {precio_col}")

# -----
# PASO 3: IDENTIFICAR COLUMNAS DE SUPERFICIE
# -----

```



```

print("\n IDENTIFICANDO COLUMNAS DE SUPERFICIE...")

# Buscar columnas que contengan "Superficie" o "m2" en el nombre
superficie_cols = [col for col in df.columns if any(term in col.lower() for
↳term in ['superficie', 'm2', 'metros', 'tamaño'])]
print(f"    • Columnas de superficie encontradas: {superficie_cols}")

if len(superficie_cols) > 0:
    superficie_col = superficie_cols[0]
    print(f"    • Usando columna: {superficie_col}")
else:
    # Buscar columnas numéricas que puedan ser superficie
    numeric_cols = df.select_dtypes(include=[np.number]).columns
    potencial_superficie = [col for col in numeric_cols if df[col].mean() > 30
↳and df[col].mean() < 500]
    if len(potencial_superficie) > 0:
        superficie_col = potencial_superficie[0]
        print(f"    • Usando columna potencial: {superficie_col}")
    else:
        print("    No se encontró columna de superficie clara")
        superficie_col = None

# -----
# PASO 4: LIMPIEZA BÁSICA DE DATOS
# -----
print("\n REALIZANDO LIMPIEZA DE DATOS...")

# Crear copia para trabajar
df_clean = df.copy()

# 4.1 Asegurar que tenemos columnas de precio y superficie
if 'precio_col' in locals() and precio_col in df_clean.columns:
    # Convertir a numérico si no lo es
    if df_clean[precio_col].dtype not in [np.float64, np.int64]:
        df_clean['Precio_num'] = pd.to_numeric(df_clean[precio_col],
↳errors='coerce')
        precio_col_actual = 'Precio_num'
    else:
        precio_col_actual = precio_col
        df_clean['Precio_num'] = df_clean[precio_col]
else:
    print("    No se pudo identificar columna de precio válida")
    # Crear columna dummy para continuar
    df_clean['Precio_num'] = np.nan

if superficie_col and superficie_col in df_clean.columns:
    # Convertir a numérico si no lo es

```

```

    if df_clean[superficie_col].dtype not in [np.float64, np.int64]:
        df_clean['Superficie_m2'] = pd.to_numeric(df_clean[superficie_col],
errors='coerce')
        superficie_col_actual = 'Superficie_m2'
    else:
        superficie_col_actual = superficie_col
        df_clean['Superficie_m2'] = df_clean[superficie_col]
else:
    print("    No se pudo identificar columna de superficie válida")
    # Crear columna dummy para continuar
    df_clean['Superficie_m2'] = np.nan

# 4.2 Calcular precio por m² si tenemos ambos
if 'Precio_num' in df_clean.columns and 'Superficie_m2' in df_clean.columns:
    df_clean['Precio_m2'] = df_clean['Precio_num'] / df_clean['Superficie_m2']
    print(f"    Calculado precio por m²")
else:
    df_clean['Precio_m2'] = np.nan
    print(f"    No se pudo calcular precio por m²")

# 4.3 Identificar columnas de ubicación
print("\n IDENTIFICANDO COLUMNAS DE UBICACIÓN...")
lat_cols = [col for col in df_clean.columns if 'lat' in col.lower()]
lng_cols = [col for col in df_clean.columns if any(term in col.lower() for term
in ['lng', 'lon', 'long'])]

if lat_cols:
    lat_col = lat_cols[0]
    df_clean['Lat'] = pd.to_numeric(df_clean[lat_col], errors='coerce')
    print(f"    • Latitud: {lat_col}")
else:
    df_clean['Lat'] = np.nan
    print(f"    No se encontró columna de latitud")

if lng_cols:
    lng_col = lng_cols[0]
    df_clean['Lng'] = pd.to_numeric(df_clean[lng_col], errors='coerce')
    print(f"    • Longitud: {lng_col}")
else:
    df_clean['Lng'] = np.nan
    print(f"    No se encontró columna de longitud")

# 4.4 Identificar columnas de recámaras
print("\n IDENTIFICANDO COLUMNAS DE RECÁMARAS...")
recamaras_cols = [col for col in df_clean.columns if any(term in col.lower()
for term in ['recamara', 'recámara', 'habitacion', 'habitación', 'room',
'bedroom'])]

```

```

if recamaras_cols:
    recamaras_col = recamaras_cols[0]
    df_clean['Recamaras'] = pd.to_numeric(df_clean[recamaras_col],
    ↪errors='coerce')
    print(f"    • Recámaras: {recamaras_col}")
else:
    df_clean['Recamaras'] = np.nan
    print(f"    No se encontró columna de recámaras")

# 4.5 Identificar columnas de baños
print("\n IDENTIFICANDO COLUMNAS DE BAÑOS...")
banos_cols = [col for col in df_clean.columns if any(term in col.lower() for
    ↪term in ['baño', 'baños', 'bano', 'banos', 'bath', 'wc'])]

if banos_cols:
    banos_col = banos_cols[0]
    df_clean['Banos'] = pd.to_numeric(df_clean[banos_col], errors='coerce')
    print(f"    • Baños: {banos_col}")
else:
    df_clean['Banos'] = np.nan
    print(f"    No se encontró columna de baños")

# 4.6 Identificar columnas de amenidades (tiempos de acceso)
print("\n IDENTIFICANDO COLUMNAS DE AMENIDADES...")
amenidad_patterns = {
    'Transporte': ['transporte', 'metro', 'estacion', 'estación', 'bus'],
    'Escuelas': ['escuela', 'colegio', 'school', 'educacion', 'educación'],
    'AreasVerdes': ['area', 'área', 'verde', 'parque', 'jardin', 'jardín',
    ↪'green'],
    'Comercios': ['comercio', 'super', 'mercado', 'tienda', 'shop', 'store'],
    'Salud': ['salud', 'hospital', 'clinica', 'clínica', 'farmacia', 'health']
}

amenidad_cols = {}
for tipo, patrones in amenidad_patterns.items():
    cols_tipo = []
    for col in df_clean.columns:
        if any(patron in col.lower() for patron in patrones):
            if 'min' in col.lower() or 'prom' in col.lower(): # Priorizar
    ↪columnas de tiempo
                cols_tipo.append(col)

    if cols_tipo:
        # Ordenar por relevancia (columnas con 'prom' o 'min' primero)
        cols_tipo.sort(key=lambda x: ('prom' in x.lower() or 'min' in x.
    ↪lower()), reverse=True)

```

```

    amenidad_cols[tipo] = cols_tipo[0]
    print(f"    • {tipo}: {cols_tipo[0]}")
else:
    amenidad_cols[tipo] = None

# 4.7 Eliminar filas con datos críticos faltantes
criticos = ['Precio_num', 'Superficie_m2']
criticos_disponibles = [col for col in criticos if col in df_clean.columns]

filas_antes = len(df_clean)
df_clean = df_clean.dropna(subset=criticos_disponibles)
filas_despues = len(df_clean)

print(f"\n RESUMEN LIMPIEZA:")
print(f"    • Filas originales: {filas_antes}")
print(f"    • Filas después de limpieza: {filas_despues}")
print(f"    • Filas eliminadas: {filas_antes - filas_despues}")

# -----
# PASO 5: ANÁLISIS EXPLORATORIO DE DATOS
# -----
print("\n ANÁLISIS EXPLORATORIO DE DATOS...")

# Estadísticas básicas de columnas numéricas
numeric_cols = df_clean.select_dtypes(include=[np.number]).columns
print(f"\n ESTADÍSTICAS DE COLUMNAS NUMÉRICAS:")
for col in numeric_cols[:10]: # Mostrar solo primeras 10
    if df_clean[col].notna().sum() > 0:
        print(f"\n    {col}:")
        print(f"        • No nulos: {df_clean[col].notna().sum()}")
        print(f"        • Media: {df_clean[col].mean():.2f}")
        print(f"        • Mediana: {df_clean[col].median():.2f}")
        print(f"        • Mín: {df_clean[col].min():.2f}")
        print(f"        • Máx: {df_clean[col].max():.2f}")

# Verificar correlaciones entre variables principales
if len(numeric_cols) > 3:
    print(f"\n CORRELACIONES ENTRE VARIABLES PRINCIPALES:")

    vars_principales = [col for col in ['Precio_num', 'Superficie_m2', '
↪ Precio_m2', 'Recamaras', 'Banos', 'Lat', 'Lng']
                        if col in numeric_cols]

    if len(vars_principales) > 2:
        corr_matrix = df_clean[vars_principales].corr()

    plt.figure(figsize=(10, 8))

```

```

sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='coolwarm',
             center=0, square=True, cbar_kws={'shrink': 0.8})
plt.title('Matriz de Correlación entre Variables Principales')
plt.tight_layout()
plt.show()

# -----
# PASO 6: SELECCIÓN DE VARIABLES PARA CLUSTERING
# -----
print("\n SELECCIÓN DE VARIABLES PARA CLUSTERING...")

# Variables base que debemos tener
variables_base = []

# 1. Precio y valor
if 'Precio_num' in df_clean.columns and df_clean['Precio_num'].notna().sum() > 0:
    variables_base.append('Precio_num')
    print(f"    Precio_num")

if 'Precio_m2' in df_clean.columns and df_clean['Precio_m2'].notna().sum() > 0:
    variables_base.append('Precio_m2')
    print(f"    Precio_m2")

# 2. Características físicas
if 'Superficie_m2' in df_clean.columns and df_clean['Superficie_m2'].notna().sum() > 0:
    variables_base.append('Superficie_m2')
    print(f"    Superficie_m2")

if 'Recamaras' in df_clean.columns and df_clean['Recamaras'].notna().sum() > 0:
    variables_base.append('Recamaras')
    print(f"    Recamaras")

if 'Banos' in df_clean.columns and df_clean['Banos'].notna().sum() > 0:
    variables_base.append('Banos')
    print(f"    Banos")

# 3. Ubicación
if 'Lat' in df_clean.columns and df_clean['Lat'].notna().sum() > 0:
    variables_base.append('Lat')
    print(f"    Lat")

if 'Lng' in df_clean.columns and df_clean['Lng'].notna().sum() > 0:
    variables_base.append('Lng')
    print(f"    Lng")

```

```

# 4. Amenidades (añadir las mejores disponibles)
for tipo, col in amenidad_cols.items():
    if col and col in df_clean.columns and df_clean[col].notna().sum() > 0:
        variables_base.append(col)
        print(f"    {col} ({tipo})")

print(f"\n VARIABLES SELECCIONADAS: {len(variables_base)}")
print(f"    • Lista: {variables_base}")

# -----
# PASO 7: PREPARACIÓN DE DATOS PARA CLUSTERING
# -----
print("\n PREPARANDO DATOS PARA CLUSTERING...")

# Verificar que tenemos suficientes variables
if len(variables_base) < 3:
    print(f"    Insuficientes variables para clustering ({len(variables_base)}).
↪")
    print(f"    Se necesitan al menos 3 variables numéricas.")

    # Agregar más variables numéricas si es necesario
    numeric_vars = df_clean.select_dtypes(include=[np.number]).columns.tolist()
    additional_vars = [col for col in numeric_vars if col not in variables_base]

    if additional_vars:
        print(f"    Añadiendo variables adicionales...")
        for col in additional_vars[:5]: # Añadir hasta 5 variables adicionales
            if df_clean[col].notna().sum() > len(df_clean) * 0.5: # Al menos
↪50% de datos
                variables_base.append(col)
                print(f"    {col} (adicional)")

        print(f"    Total variables ahora: {len(variables_base)}")

# Crear dataset para clustering
X = df_clean[variables_base].copy()

# Manejar valores nulos (rellenar con mediana)
for col in X.columns:
    X[col] = X[col].fillna(X[col].median())

print(f"    • Dataset preparado: {X.shape}")
print(f"    • Variables: {X.shape[1]}")
print(f"    • Propiedades: {X.shape[0]}")

# -----
# PASO 8: ESCALADO Y REDUCCIÓN DE DIMENSIONALIDAD

```

```

# -----
print("\n ESCALANDO Y REDUCIENDO DIMENSIONALIDAD...")

# Escalar datos
scaler = RobustScaler()
X_scaled = scaler.fit_transform(X)

# Reducción de dimensionalidad para visualización
if X_scaled.shape[1] > 3:
    print(f"    • Aplicando PCA (reduciendo de {X_scaled.shape[1]} a 3_
    ↪dimensiones)...")
    pca = PCA(n_components=3)
    X_pca = pca.fit_transform(X_scaled)

    print(f"    • Varianza explicada por PCA: {pca.explained_variance_ratio_
    ↪sum():.2%}")
    for i, var in enumerate(pca.explained_variance_ratio_, 1):
        print(f"    • Componente {i}: {var:.2%}")
else:
    X_pca = X_scaled
    print(f"    • No se aplica PCA (solo {X_scaled.shape[1]} dimensiones)")

# -----
# PASO 9: DETERMINAR NÚMERO ÓPTIMO DE CLUSTERS
# -----
print("\n DETERMINANDO NÚMERO ÓPTIMO DE CLUSTERS...")

def determinar_k_optimo(X_scaled, max_k=10):
    """Determina el número óptimo de clusters."""

    inertias = []
    silhouette_scores = []
    K_range = range(2, max_k + 1)

    for k in K_range:
        print(f"    • Probando k={k}...", end='\r')

        kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
        labels = kmeans.fit_predict(X_scaled)

        inertias.append(kmeans.inertia_)

        if len(set(labels)) > 1:
            score = silhouette_score(X_scaled, labels)
            silhouette_scores.append(score)
        else:
            silhouette_scores.append(0)

```

```

print(f"{' ' * 50}") # Limpiar línea

# Método del codo
diff = np.diff(inertias)
diff2 = np.diff(diff)
if len(diff2) > 0:
    elbow_point = np.argmax(diff2) + 3
else:
    elbow_point = 3

# Mejor según silueta
if silhouette_scores:
    best_silhouette_k = K_range[np.argmax(silhouette_scores)]
    best_silhouette_score = max(silhouette_scores)
else:
    best_silhouette_k = 3
    best_silhouette_score = 0

# Graficar resultados
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

# Método del codo
axes[0].plot(K_range, inertias, 'bo-')
axes[0].axvline(x=elbow_point, color='r', linestyle='--', alpha=0.5,
↳label=f'Codo: k={elbow_point}')
axes[0].set_xlabel('Número de Clusters (k)')
axes[0].set_ylabel('Inercia')
axes[0].set_title('Método del Codo')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Coeficiente de silueta
axes[1].plot(K_range, silhouette_scores, 'go-')
axes[1].axvline(x=best_silhouette_k, color='r', linestyle='--', alpha=0.5,
↳label=f'Mejor: k={best_silhouette_k}')
↳(score={best_silhouette_score:.3f})')
axes[1].set_xlabel('Número de Clusters (k)')
axes[1].set_ylabel('Coeficiente de Silueta')
axes[1].set_title('Coeficiente de Silueta')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.suptitle('Análisis de Número Óptimo de Clusters', fontsize=14)
plt.tight_layout()
plt.show()

```



```

# Decidir k óptimo
# Si el score de silueta es bueno (>0.5) usar esa recomendación
# Si no, usar el método del codo
if best_silhouette_score > 0.5:
    optimal_k = best_silhouette_k
    metodo = 'silueta'
else:
    optimal_k = elbow_point
    metodo = 'codo'

# Asegurar que tenemos al menos 3 clusters
if optimal_k < 3:
    optimal_k = 3
    metodo = 'mínimo'

# No más de 8 clusters para evitar sobre-segmentación
if optimal_k > 8:
    optimal_k = 8
    metodo = 'máximo'

return optimal_k, metodo, best_silhouette_score

# Ejecutar análisis
optimal_k, metodo, sil_score = determinar_k_optimo(X_scaled, max_k=min(10,
↪len(df_clean)//20))

print(f"\n    • K óptimo recomendado: {optimal_k} (método: {metodo})")
print(f"    • Score de silueta esperado: {sil_score:.3f}")

# -----
# PASO 10: APLICAR CLUSTERING
# -----
print(f"\n APLICANDO CLUSTERING CON k={optimal_k}...")

# Aplicar K-Means
kmeans = KMeans(n_clusters=optimal_k, random_state=42, n_init=20)
df_clean['Cluster'] = kmeans.fit_predict(X_scaled)

# Calcular métricas de calidad
silhouette_avg = silhouette_score(X_scaled, df_clean['Cluster'])
calinski_score = calinski_harabasz_score(X_scaled, df_clean['Cluster'])

print(f"    • Coeficiente de silueta: {silhouette_avg:.3f}")
print(f"    • Índice Calinski-Harabasz: {calinski_score:.0f}")

# Mostrar distribución de clusters
print(f"\n DISTRIBUCIÓN DE CLUSTERS:")

```

```

cluster_counts = df_clean['Cluster'].value_counts().sort_index()
for cluster_id, count in cluster_counts.items():
    percentage = (count / len(df_clean)) * 100
    print(f"    • Cluster {cluster_id}: {count} propiedades ({percentage:.1f}%)")

# -----
# PASO 11: ANÁLISIS DE CLUSTERS
# -----
print("\n ANALIZANDO PERFILES DE CLUSTERS...")

def crear_perfiles_clusters(df, cluster_col, variables_analisis):
    """Crea perfiles descriptivos para cada cluster."""

    perfiles = []

    for cluster_id in sorted(df[cluster_col].unique()):
        cluster_data = df[df[cluster_col] == cluster_id]

        perfil = {
            'Cluster': cluster_id,
            'N_Propiedades': len(cluster_data)
        }

        # Estadísticas básicas para cada variable
        for var in variables_analisis:
            if var in cluster_data.columns and cluster_data[var].notna().sum() > 0:
                perfil[f'{var}_mean'] = cluster_data[var].mean()
                perfil[f'{var}_median'] = cluster_data[var].median()
                perfil[f'{var}_std'] = cluster_data[var].std()

        perfiles.append(perfil)

    return pd.DataFrame(perfiles)

# Variables para análisis de perfiles
variables_perfil = ['Precio_num', 'Precio_m2', 'Superficie_m2', 'Recamaras',
                    'Banos']
variables_perfil = [v for v in variables_perfil if v in df_clean.columns]

perfiles_df = crear_perfiles_clusters(df_clean, 'Cluster', variables_perfil)

print(f"\n PERFILES DE CLUSTERS (resumen):")
print("-" * 80)

for idx, row in perfiles_df.iterrows():
    cluster_id = int(row['Cluster'])

```

```

n_prop = int(row['N_Propiedades'])

print(f"\n CLUSTER {cluster_id} ({n_prop} propiedades):")

if 'Precio_num_mean' in row:
    precio = row['Precio_num_mean']
    print(f"    Precio promedio: ${precio:,.0f} MXN")

if 'Precio_m2_mean' in row:
    precio_m2 = row['Precio_m2_mean']
    print(f"    Precio/m²: ${precio_m2:,.0f} MXN")

if 'Superficie_m2_mean' in row:
    superficie = row['Superficie_m2_mean']
    print(f"    Superficie promedio: {superficie:.0f} m²")

if 'Recamaras_mean' in row:
    recamaras = row['Recamaras_mean']
    print(f"    Recámaras promedio: {recamaras:.1f}")

if 'Banos_mean' in row:
    banos = row['Banos_mean']
    print(f"    Baños promedio: {banos:.1f}")

# -----
# PASO 12: VISUALIZACIÓN DE CLUSTERS
# -----
print("\n GENERANDO VISUALIZACIONES...")

def visualizar_clusters_basico(df, cluster_col, X_pca):
    """Visualización básica de clusters."""

    n_clusters = len(df[cluster_col].unique())
    colors = plt.cm.tab20(np.linspace(0, 1, n_clusters))

    fig = plt.figure(figsize=(18, 12))

    # 1. Mapa geográfico
    ax1 = plt.subplot(2, 3, 1)
    for i, cluster_id in enumerate(sorted(df[cluster_col].unique())):
        cluster_data = df[df[cluster_col] == cluster_id]
        ax1.scatter(cluster_data['Lng'], cluster_data['Lat'],
                    color=colors[i], s=30, alpha=0.6,
                    label=f'C{cluster_id}')

    ax1.set_xlabel('Longitud')
    ax1.set_ylabel('Latitud')

```

```

ax1.set_title('Distribución Geográfica')
ax1.legend(loc='best', fontsize=8)

# 2. PCA 2D
ax2 = plt.subplot(2, 3, 2)
for i, cluster_id in enumerate(sorted(df[cluster_col].unique())):
    cluster_mask = df[cluster_col] == cluster_id
    ax2.scatter(X_pca[cluster_mask, 0], X_pca[cluster_mask, 1],
                color=colors[i], s=30, alpha=0.6,
                label=f'C{cluster_id}')

ax2.set_xlabel('PCA Component 1')
ax2.set_ylabel('PCA Component 2')
ax2.set_title('Clusters en Espacio PCA')
ax2.grid(True, alpha=0.3)

# 3. Precio vs Superficie
ax3 = plt.subplot(2, 3, 3)
for i, cluster_id in enumerate(sorted(df[cluster_col].unique())):
    cluster_data = df[df[cluster_col] == cluster_id]
    ax3.scatter(cluster_data['Superficie_m2'],
                cluster_data['Precio_num']/1e6,
                color=colors[i], s=30, alpha=0.6,
                label=f'C{cluster_id}')

ax3.set_xlabel('Superficie (m²)')
ax3.set_ylabel('Precio (Millones MXN)')
ax3.set_title('Precio vs Superficie')
ax3.grid(True, alpha=0.3)

# 4. Boxplot de Precio/m²
ax4 = plt.subplot(2, 3, 4)
price_data = []
labels = []
for cluster_id in sorted(df[cluster_col].unique()):
    cluster_data = df[df[cluster_col] == cluster_id]
    price_data.append(cluster_data['Precio_m2'].values)
    labels.append(f'C{cluster_id}')

bp = ax4.boxplot(price_data, labels=labels, patch_artist=True)
for patch, color in zip(bp['boxes'], colors):
    patch.set_facecolor(color)
    patch.set_alpha(0.7)

ax4.set_ylabel('Precio por m² (MXN)')
ax4.set_title('Distribución de Precio/m²')
ax4.grid(True, alpha=0.3, axis='y')

```

```

# 5. Distribución de recámaras
ax5 = plt.subplot(2, 3, 5)
if 'Recamaras' in df.columns:
    recamaras_counts = pd.crosstab(df[cluster_col], df['Recamaras'])
    recamaras_counts.plot(kind='bar', stacked=True, ax=ax5,
                          colormap='viridis')
    ax5.set_xlabel('Cluster')
    ax5.set_ylabel('Número de Propiedades')
    ax5.set_title('Recámaras por Cluster')
    ax5.legend(title='Recámaras', bbox_to_anchor=(1.05, 1), loc='upper_
↳left')

# 6. Barras de distribución
ax6 = plt.subplot(2, 3, 6)
cluster_counts = df[cluster_col].value_counts().sort_index()
bars = ax6.bar(range(len(cluster_counts)), cluster_counts.values,
↳color=colors)

ax6.set_xlabel('Cluster')
ax6.set_ylabel('Número de Propiedades')
ax6.set_title('Distribución de Propiedades')
ax6.set_xticks(range(len(cluster_counts)))
ax6.set_xticklabels([f'C{i}' for i in cluster_counts.index])

# Añadir porcentajes
total = cluster_counts.sum()
for i, (bar, count) in enumerate(zip(bars, cluster_counts.values)):
    height = bar.get_height()
    ax6.text(bar.get_x() + bar.get_width()/2., height + 0.5,
             f'{count}\n({count/total*100:.1f}%)',
             ha='center', va='bottom', fontsize=9)

plt.suptitle(f'Análisis de Clusters Inmobiliarios (k={n_clusters})',
             fontsize=16, y=1.02)
plt.tight_layout()
plt.show()

# Generar visualizaciones
visualizar_clusters_basico(df_clean, 'Cluster', X_pca)

# -----
# PASO 13: ANÁLISIS DE CARACTERÍSTICAS POR CLUSTER
# -----
print("\n ANALIZANDO CARACTERÍSTICAS POR CLUSTER...")

# Crear heatmap de características

```

```

if len(variables_base) > 3:
    # Seleccionar variables clave para heatmap
    vars_heatmap = []
    for var in ['Precio_m2', 'Superficie_m2', 'Recamaras', 'Banos']:
        if var in df_clean.columns:
            vars_heatmap.append(var)

    # Añadir algunas amenidades si existen
    for tipo in ['Transporte', 'Escuelas', 'AreasVerdes']:
        if amenidad_cols.get(tipo) and amenidad_cols[tipo] in df_clean.columns:
            vars_heatmap.append(amenidad_cols[tipo])

    if len(vars_heatmap) >= 3:
        # Calcular promedios por cluster
        heatmap_data = df_clean.groupby('Cluster')[vars_heatmap].mean()

        # Normalizar para mejor visualización
        heatmap_data_normalized = heatmap_data.apply(lambda x: (x - x.min()) /
        ↪(x.max() - x.min()))

        plt.figure(figsize=(12, 8))
        sns.heatmap(heatmap_data_normalized.T, annot=True, fmt='.2f',
                    cmap='YlOrRd', cbar_kws={'label': 'Valor Normalizado (0-1)'})
        plt.title('Características Promedio por Cluster (Normalizadas)')
        plt.xlabel('Cluster')
        plt.ylabel('Características')
        plt.tight_layout()
        plt.show()

# -----
# PASO 14: CREAR ETIQUETAS DESCRIPTIVAS PARA CLUSTERS
# -----
print("\n CREANDO ETIQUETAS DESCRIPTIVAS PARA CLUSTERS...")

def crear_etiquetas_inteligentes(df, cluster_col, perfiles_df):
    """Crea etiquetas descriptivas inteligentes para cada cluster."""

    etiquetas = {}

    # Calcular percentiles generales
    if 'Precio_m2' in df.columns:
        precio_q1 = df['Precio_m2'].quantile(0.25)
        precio_q3 = df['Precio_m2'].quantile(0.75)

    if 'Superficie_m2' in df.columns:
        superficie_q1 = df['Superficie_m2'].quantile(0.25)
        superficie_q3 = df['Superficie_m2'].quantile(0.75)

```

```

for idx, row in perfiles_df.iterrows():
    cluster_id = int(row['Cluster'])

    # Determinar nivel de precio
    if 'Precio_m2_mean' in row:
        precio_m2 = row['Precio_m2_mean']
        if precio_m2 > precio_q3:
            nivel_precio = "ALTO"
        elif precio_m2 < precio_q1:
            nivel_precio = "BAJO"
        else:
            nivel_precio = "MEDIO"
    else:
        nivel_precio = ""

    # Determinar tamaño
    if 'Superficie_m2_mean' in row:
        tamano = row['Superficie_m2_mean']
        if tamano > superficie_q3:
            tamano_desc = "GRANDES"
        elif tamano < superficie_q1:
            tamano_desc = "COMPACTAS"
        else:
            tamano_desc = "MEDIANAS"
    else:
        tamano_desc = ""

    # Determinar densidad
    n_prop = row['N_Propiedades']
    total_prop = perfiles_df['N_Propiedades'].sum()
    porcentaje = (n_prop / total_prop) * 100

    if porcentaje > 20:
        densidad = "CONCENTRADO"
    elif porcentaje < 10:
        densidad = "DISPERSO"
    else:
        densidad = "MODERADO"

    # Construir etiqueta
    partes = []
    if nivel_precio:
        partes.append(nivel_precio)
    if tamano_desc:
        partes.append(tamano_desc)
    if densidad:

```

```

        partes.append(densidad)

    etiqueta = " - ".join(partes) if partes else f"Cluster {cluster_id}"
    etiquetas[cluster_id] = etiqueta

    return etiquetas

# Crear etiquetas
etiquetas_clusters = crear_etiquetas_inteligentes(df_clean, 'Cluster',
    ↪perfiles_df)

print(f"\n ETIQUETAS ASIGNADAS:")
for cluster_id, etiqueta in etiquetas_clusters.items():
    n_prop = perfiles_df[perfiles_df['Cluster'] == cluster_id]['N_Propiedades'].
    ↪values[0]
    print(f"    • Cluster {cluster_id}: {etiqueta} ({n_prop} propiedades)")

# -----
# PASO 15: EXPORTAR RESULTADOS
# -----
print("\n EXPORTANDO RESULTADOS...")

def exportar_resultados_simples(df, cluster_col, etiquetas,
    ↪filename="resultados_clusters.csv"):
    """Exporta resultados simples a CSV."""

    # Crear DataFrame para exportación
    df_export = df.copy()

    # Añadir etiqueta de cluster
    df_export['Cluster_Etiqueta'] = df_export[cluster_col].map(etiquetas)

    # Seleccionar columnas importantes
    export_cols = [cluster_col, 'Cluster_Etiqueta', 'Direccion', 'Ubicacion']

    # Añadir columnas numéricas importantes
    numeric_important = ['Precio_num', 'Precio_m2', 'Superficie_m2',
    ↪'Recamaras', 'Banos', 'Lat', 'Lng']
    for col in numeric_important:
        if col in df_export.columns:
            export_cols.append(col)

    # Añadir amenidades si existen
    for tipo, col in amenidad_cols.items():
        if col and col in df_export.columns:
            export_cols.append(col)

```



```

# Ordenar por cluster
df_export = df_export.sort_values([cluster_col, 'Precio_m2'],
↪ascending=[True, False])

# Exportar
df_export[export_cols].to_csv(filename, index=False, encoding='utf-8')

# Crear resumen de clusters
resumen_cols = ['Cluster', 'Etiqueta', 'N_Propiedades']
if 'Precio_num_mean' in perfiles_df.columns:
    resumen_cols.append('Precio_Promedio')
if 'Precio_m2_mean' in perfiles_df.columns:
    resumen_cols.append('Precio_m2_Promedio')
if 'Superficie_m2_mean' in perfiles_df.columns:
    resumen_cols.append('Superficie_Promedio')

resumen_data = []
for idx, row in perfiles_df.iterrows():
    cluster_id = int(row['Cluster'])
    resumen_row = {
        'Cluster': cluster_id,
        'Etiqueta': etiquetas[cluster_id],
        'N_Propiedades': int(row['N_Propiedades'])
    }

    if 'Precio_num_mean' in row:
        resumen_row['Precio_Promedio'] = row['Precio_num_mean']

    if 'Precio_m2_mean' in row:
        resumen_row['Precio_m2_Promedio'] = row['Precio_m2_mean']

    if 'Superficie_m2_mean' in row:
        resumen_row['Superficie_Promedio'] = row['Superficie_m2_mean']

    resumen_data.append(resumen_row)

resumen_df = pd.DataFrame(resumen_data)
resumen_filename = filename.replace('.csv', '_resumen.csv')
resumen_df.to_csv(resumen_filename, index=False, encoding='utf-8')

print(f" Resultados exportados:")
print(f"     • Archivo principal: {filename}")
print(f"     • Resumen de clusters: {resumen_filename}")
print(f"     • Propiedades exportadas: {len(df_export)}")
print(f"     • Clusters identificados: {len(etiquetas)}")

# Exportar resultados

```

```

exportar_resultados_simples(df_clean, 'Cluster', etiquetas_clusters,
    ↪ "clusters_inmobiliarios.csv")

# -----
# PASO 16: RESUMEN EJECUTIVO
# -----

print("\n" + "="*70)
print("RESUMEN EJECUTIVO DEL ANÁLISIS")
print("="*70)

print(f"\n DATOS GENERALES:")
print(f"    • Propiedades analizadas: {len(df_clean):,}")
print(f"    • Clusters identificados: {optimal_k}")
print(f"    • Variables utilizadas: {len(variables_base)}")
print(f"    • Calidad del clustering (silueta): {silhouette_avg:.3f}")

print(f"\n SEGMENTOS IDENTIFICADOS:")
for cluster_id in sorted(etiquetas_clusters.keys()):
    etiqueta = etiquetas_clusters[cluster_id]
    n_prop = perfiles_df[perfiles_df['Cluster'] == cluster_id]['N_Propiedades'].
    ↪ values[0]
    porcentaje = (n_prop / len(df_clean)) * 100

    # Obtener características destacadas
    características = []

    if 'Precio_m2_mean' in perfiles_df.columns:
        precio_m2 = perfiles_df[perfiles_df['Cluster'] ==
    ↪ cluster_id]['Precio_m2_mean'].values[0]
        características.append(f"${precio_m2:,.0f}/m²")

    if 'Superficie_m2_mean' in perfiles_df.columns:
        superficie = perfiles_df[perfiles_df['Cluster'] ==
    ↪ cluster_id]['Superficie_m2_mean'].values[0]
        características.append(f"{superficie:.0f} m²")

    print(f"\n    CLUSTER {cluster_id}: {etiqueta}")
    print(f"        • Propiedades: {n_prop} ({porcentaje:.1f}%)")
    if características:
        print(f"        • Características: {' | '.join(características)}")

print(f"\n INSIGHTS CLAVE:")
print(f"    1. Se identificaron {optimal_k} segmentos distintos en el mercado")
print(f"    2. La calidad del clustering es {'buena' if silhouette_avg > 0.5
    ↪ else 'moderada' if silhouette_avg > 0.3 else 'baja'}")
print(f"    3. Los clusters varían en precio, tamaño y características")

```

```

print(f"\n RECOMENDACIONES:")
print(f"    1. Usar los clusters para segmentación de mercado")
print(f"    2. Desarrollar estrategias de marketing diferenciadas")
print(f"    3. Analizar oportunidades en cada segmento")
print(f"    4. Monitorear evolución de clusters en el tiempo")

print(f"\n MÉTRICAS DE CALIDAD:")
print(f"    • Coeficiente de Silueta: {silhouette_avg:.3f}")
print(f"    • Índice Calinski-Harabasz: {calinski_score:.0f}")
print(f"    • Método para determinar k: {metodo}")

print("\n ANÁLISIS COMPLETADO EXITOSAMENTE")
print("="*70)
print(f"\n SIGUIENTES PASOS RECOMENDADOS:")
print(f"    1. Revisar las propiedades en cada cluster para validar los_
    ↪perfiles")
print(f"    2. Desarrollar materiales de marketing específicos para cada_
    ↪segmento")
print(f"    3. Analizar la competencia en cada segmento identificado")
print(f"    4. Establecer KPIs para monitorear el desempeño por segmento")

print(f"\n PARA MÁS ANÁLISIS:")
print(f"    • Revisar los archivos CSV generados")
print(f"    • Validar clusters con expertos del sector")
print(f"    • Realizar análisis de precios competitivos por segmento")

```

=====

ANÁLISIS AVANZADO DE CLUSTERIZACIÓN - EXPLORACIÓN INICIAL

=====

INFORMACIÓN DEL DATASET:

- Filas: 3992
- Columnas: 204

NOMBRES DE COLUMNAS DISPONIBLES:

1. Link
2. Direccion
3. Ubicacion
4. Maps_url
5. Lat
6. Lng
7. POI_Botones
8. Transporte_txt_all
9. Escuelas_txt_all
10. AreasVerdes_txt_all
11. Comercios_txt_all
12. Salud_txt_all

13. Transporte_POI
14. Escuelas_POI
15. AreasVerdes_POI
16. Comercios_POI
17. Salud_POI
18. Precio
19. Precio_num
20. Recamaras
21. Superficie_m2
22. Banos
23. Unidades
24. CP
25. Transporte_txt_all.1
26. Transporte_1_txt
27. Transporte_1_mins
28. Transporte_1_metros
29. Transporte_2_txt
30. Transporte_2_mins
31. Transporte_2_metros
32. Transporte_3_txt
33. Transporte_3_mins
34. Transporte_3_metros
35. Transporte_4_txt
36. Transporte_4_mins
37. Transporte_4_metros
38. Transporte_5_txt
39. Transporte_5_mins
40. Transporte_5_metros
41. Transporte_6_txt
42. Transporte_6_mins
43. Transporte_6_metros
44. Transporte_7_txt
45. Transporte_7_mins
46. Transporte_7_metros
47. Transporte_8_txt
48. Transporte_8_mins
49. Transporte_8_metros
50. Transporte_9_txt
51. Transporte_9_mins
52. Transporte_9_metros
53. Transporte_10_txt
54. Transporte_10_mins
55. Transporte_10_metros
56. Escuelas_txt_all.1
57. Escuelas_1_txt
58. Escuelas_1_mins
59. Escuelas_1_metros
60. Escuelas_2_txt

61. Escuelas_2_mins
62. Escuelas_2_metros
63. Escuelas_3_txt
64. Escuelas_3_mins
65. Escuelas_3_metros
66. Escuelas_4_txt
67. Escuelas_4_mins
68. Escuelas_4_metros
69. Escuelas_5_txt
70. Escuelas_5_mins
71. Escuelas_5_metros
72. Escuelas_6_txt
73. Escuelas_6_mins
74. Escuelas_6_metros
75. Escuelas_7_txt
76. Escuelas_7_mins
77. Escuelas_7_metros
78. Escuelas_8_txt
79. Escuelas_8_mins
80. Escuelas_8_metros
81. Escuelas_9_txt
82. Escuelas_9_mins
83. Escuelas_9_metros
84. Escuelas_10_txt
85. Escuelas_10_mins
86. Escuelas_10_metros
87. Escuelas_11_txt
88. Escuelas_11_mins
89. Escuelas_11_metros
90. Escuelas_12_txt
91. Escuelas_12_mins
92. Escuelas_12_metros
93. Escuelas_13_txt
94. Escuelas_13_mins
95. Escuelas_13_metros
96. Escuelas_14_txt
97. Escuelas_14_mins
98. Escuelas_14_metros
99. Escuelas_15_txt
100. Escuelas_15_mins
101. Escuelas_15_metros
102. AreasVerdes_txt_all.1
103. AreasVerdes_1_txt
104. AreasVerdes_1_mins
105. AreasVerdes_1_metros
106. AreasVerdes_2_txt
107. AreasVerdes_2_mins
108. AreasVerdes_2_metros

109. AreasVerdes_3_txt
110. AreasVerdes_3_mins
111. AreasVerdes_3_metros
112. AreasVerdes_4_txt
113. AreasVerdes_4_mins
114. AreasVerdes_4_metros
115. AreasVerdes_5_txt
116. AreasVerdes_5_mins
117. AreasVerdes_5_metros
118. Comercios_txt_all.1
119. Comercios_1_txt
120. Comercios_1_mins
121. Comercios_1_metros
122. Comercios_2_txt
123. Comercios_2_mins
124. Comercios_2_metros
125. Comercios_3_txt
126. Comercios_3_mins
127. Comercios_3_metros
128. Comercios_4_txt
129. Comercios_4_mins
130. Comercios_4_metros
131. Comercios_5_txt
132. Comercios_5_mins
133. Comercios_5_metros
134. Comercios_6_txt
135. Comercios_6_mins
136. Comercios_6_metros
137. Comercios_7_txt
138. Comercios_7_mins
139. Comercios_7_metros
140. Comercios_8_txt
141. Comercios_8_mins
142. Comercios_8_metros
143. Comercios_9_txt
144. Comercios_9_mins
145. Comercios_9_metros
146. Comercios_10_txt
147. Comercios_10_mins
148. Comercios_10_metros
149. Comercios_11_txt
150. Comercios_11_mins
151. Comercios_11_metros
152. Comercios_12_txt
153. Comercios_12_mins
154. Comercios_12_metros
155. Comercios_13_txt
156. Comercios_13_mins

157. Comercios_13_metros
158. Comercios_14_txt
159. Comercios_14_mins
160. Comercios_14_metros
161. Comercios_15_txt
162. Comercios_15_mins
163. Comercios_15_metros
164. Salud_txt_all.1
165. Salud_1_txt
166. Salud_1_mins
167. Salud_1_metros
168. Salud_2_txt
169. Salud_2_mins
170. Salud_2_metros
171. Salud_3_txt
172. Salud_3_mins
173. Salud_3_metros
174. Salud_4_txt
175. Salud_4_mins
176. Salud_4_metros
177. Salud_5_txt
178. Salud_5_mins
179. Salud_5_metros
180. Salud_6_txt
181. Salud_6_mins
182. Salud_6_metros
183. Salud_7_txt
184. Salud_7_mins
185. Salud_7_metros
186. Salud_8_txt
187. Salud_8_mins
188. Salud_8_metros
189. Salud_9_txt
190. Salud_9_mins
191. Salud_9_metros
192. Salud_10_txt
193. Salud_10_mins
194. Salud_10_metros
195. Transporte_prom_mins
196. Transporte_prom_metros
197. Escuelas_prom_mins
198. Escuelas_prom_metros
199. AreasVerdes_prom_mins
200. AreasVerdes_prom_metros
201. Comercios_prom_mins
202. Comercios_prom_metros
203. Salud_prom_mins
204. Salud_prom_metros

PRIMERAS 5 FILAS:

Link	Direccion		Maps_url	Lat
Ubicacion				
Lng			POI_Botones	
Transporte_txt_all				
Escuelas_txt_all				
AreasVerdes_txt_all				
Comercios_txt_all				
Salud_txt_all				
Transporte_POI				
Escuelas_POI				
AreasVerdes_POI				
Comercios_POI				
Salud_POI	Precio	Precio_num	Recamaras	Superficie_m2
CP			Banos	Unidades
Transporte_txt_all.1				
Transporte_1_txt	Transporte_1_mins	Transporte_1_metros		
Transporte_2_txt	Transporte_2_mins	Transporte_2_metros		
Transporte_3_txt	Transporte_3_mins	Transporte_3_metros		
Transporte_4_txt	Transporte_4_mins	Transporte_4_metros		
Transporte_5_txt	Transporte_5_mins	Transporte_5_metros		
Transporte_6_txt	Transporte_6_mins	Transporte_6_metros		
Transporte_7_txt	Transporte_7_mins	Transporte_7_metros		
Transporte_8_txt	Transporte_8_mins	Transporte_8_metros		
Transporte_9_txt	Transporte_9_mins	Transporte_9_metros		
Transporte_10_txt	Transporte_10_mins	Transporte_10_metros		
Escuelas_txt_all.1				
Escuelas_1_txt	Escuelas_1_mins	Escuelas_1_metros		
Escuelas_2_txt	Escuelas_2_mins	Escuelas_2_metros		
Escuelas_3_txt	Escuelas_3_mins	Escuelas_3_metros		
Escuelas_4_txt	Escuelas_4_mins	Escuelas_4_metros		
Escuelas_5_txt	Escuelas_5_mins	Escuelas_5_metros		Escuelas_6_txt
Escuelas_6_mins	Escuelas_6_metros		Escuelas_7_txt	
Escuelas_7_mins	Escuelas_7_metros		Escuelas_8_txt	
Escuelas_8_mins	Escuelas_8_metros		Escuelas_9_txt	
Escuelas_9_mins	Escuelas_9_metros		Escuelas_10_txt	
Escuelas_10_mins	Escuelas_10_metros		Escuelas_11_txt	
Escuelas_11_mins	Escuelas_11_metros		Escuelas_12_txt	
Escuelas_12_mins	Escuelas_12_metros		Escuelas_13_txt	
Escuelas_13_mins	Escuelas_13_metros		Escuelas_14_txt	
Escuelas_14_mins	Escuelas_14_metros		Escuelas_15_txt	
Escuelas_15_mins	Escuelas_15_metros			
AreasVerdes_txt_all.1				
AreasVerdes_1_txt	AreasVerdes_1_mins	AreasVerdes_1_metros		
AreasVerdes_2_txt	AreasVerdes_2_mins	AreasVerdes_2_metros		
AreasVerdes_3_txt	AreasVerdes_3_mins	AreasVerdes_3_metros		
AreasVerdes_4_txt	AreasVerdes_4_mins	AreasVerdes_4_metros		

AreasVerdes_5_txt AreasVerdes_5_mins AreasVerdes_5_metros
 Comercios_txt_all.1
 Comercios_1_txt Comercios_1_mins Comercios_1_metros
 Comercios_2_txt Comercios_2_mins Comercios_2_metros
 Comercios_3_txt Comercios_3_mins Comercios_3_metros
 Comercios_4_txt Comercios_4_mins Comercios_4_metros
 Comercios_5_txt Comercios_5_mins Comercios_5_metros
 Comercios_6_txt Comercios_6_mins Comercios_6_metros
 Comercios_7_txt Comercios_7_mins Comercios_7_metros
 Comercios_8_txt Comercios_8_mins Comercios_8_metros
 Comercios_9_txt Comercios_9_mins Comercios_9_metros
 Comercios_10_txt Comercios_10_mins Comercios_10_metros
 Comercios_11_txt Comercios_11_mins Comercios_11_metros
 Comercios_12_txt Comercios_12_mins Comercios_12_metros
 Comercios_13_txt Comercios_13_mins Comercios_13_metros
 Comercios_14_txt Comercios_14_mins Comercios_14_metros
 Comercios_15_txt Comercios_15_mins Comercios_15_metros
 Salud_txt_all.1
 Salud_1_txt Salud_1_mins Salud_1_metros
 Salud_2_txt Salud_2_mins Salud_2_metros
 Salud_3_txt Salud_3_mins Salud_3_metros
 Salud_4_txt Salud_4_mins Salud_4_metros Salud_5_txt
 Salud_5_mins Salud_5_metros Salud_6_txt Salud_6_mins
 Salud_6_metros Salud_7_txt Salud_7_mins Salud_7_metros
 Salud_8_txt Salud_8_mins Salud_8_metros Salud_9_txt
 Salud_9_mins Salud_9_metros Salud_10_txt Salud_10_mins
 Salud_10_metros Transporte_prom_mins Transporte_prom_metros
 Escuelas_prom_mins Escuelas_prom_metros AreasVerdes_prom_mins
 AreasVerdes_prom_metros Comercios_prom_mins Comercios_prom_metros
 Salud_prom_mins Salud_prom_metros
 0 https://casa.mercadolibre.com.mx/MLM-2578861711-reserva-de-sur-_JM#position=1&search_layout=map&type=item&tracking_id=61084625-8f1e-4422-85d4-2c587e5d786a Reserva del sur Conrado Pelayo 67, Cp. 13200, Col. Miguel Hidalgo, Del. Tláhuac. Ciudad De México, Miguel Hidalgo, Distrito Federal <https://www.google.com/maps?q=19.296073,-99.0443251&z=16>
 19.296073 -99.044325 Transporte | Educación | Áreas verdes | Comercios | Salud
 Estaciones de metro | Nopalera | 6 mins - 458 metros\nEstaciones de metro | Zapotitlán | 14 mins - 1,064 metros\nEstaciones de metro | Olivos | 24 mins - 1,847 metros Escuelas | Colegio de Culturas de México | 11 mins - 873 metros\nEscuelas | "Escuela Primaria "TlamachKali" 51 190" | 11 mins - 876 metros\nEscuelas | Escuela Primaria Oficial Ricardo Flores Magón | 11 mins - 884 metros\nEscuelas | "Escuela Secundaria "Juan Rulfo\" | 13 mins - 1,004 metros\nEscuelas | "Escuela Primaria "Jaime Sabines\" | 14 mins - 1,100 metros Plazas | Parque y Gimnasios Urbanos 222 | 11 mins - 878 metros\nPlazas | "Parque "Canchas Gitana\" | 14 mins - 1,108 metros\nPlazas | "Parque "Atlíxco\" | 15 mins - 1,137 metros\nPlazas | Parque Nopalera | 15 mins - 1,163 metros\nPlazas | Deportivo Nopalera | 16 mins - 1,241 metros Supermercados | Bodega Aurrera | 13 mins - 1,029 metros\nSupermercados | Mercado

Nopalera | 13 mins - 1,041 metros\nSupermercados | El Venadito | 15 mins - 1,153 metros\nSupermercados | Neto | 15 mins - 1,176 metros\nSupermercados | Tienda BBB | 21 mins - 1,640 metros\nFarmacias | Farmacia Guadalajara | 6 mins - 437 metros\nFarmacias | Farmacias de Genéricos y de Patente | 16 mins - 1,248 metros\nFarmacias | Farmacia Farmapronto | 17 mins - 1,341 metros Hospitales | Sanatorio Psiquiatrico del Carmen | 6 mins - 485 metros\nHospitales | Hospital General Tláhuac | 18 mins - 1,365 metros\nHospitales | Nuevo Hospital ISSSTE Tláhuac | 25 mins - 1,917 metros\nHospitales | IMSS UMA Unidad Medica Ambulatoria | 25 mins - 1,986 metros

```
[{"tipo": "Estaciones de metro", "nombre": "Estaciones de metro Nopalera",
"raw": "6 mins - 458 metros", "mins": 6.0, "metros": 458.0}, {"tipo":
"Estaciones de metro", "nombre": "Estaciones de metro Zapotitlán", "raw": "14
mins - 1,064 metros", "mins": 14.0, "metros": 1064.0}, {"tipo": "Estaciones de
metro", "nombre": "Estaciones de metro Olivos", "raw": "24 mins - 1,847 metros",
"mins": 24.0, "metros": 1847.0}] [{"tipo": "Escuelas", "nombre": "Escuelas
Colegio de Culturas de México", "raw": "11 mins - 873 metros", "mins": 11.0,
"metros": 873.0}, {"tipo": "Escuelas", "nombre": "Escuelas \"Escuela Primaria
\" \"TlamachKali\" \" 51 190\", \"raw\": \"11 mins - 876 metros\", \"mins\": 11.0,
\"metros\": 876.0}, {"tipo": "Escuelas", "nombre": "Escuelas Escuela Primaria
Oficial Ricardo Flores Magón", "raw": \"11 mins - 884 metros\", \"mins\": 11.0,
\"metros\": 884.0}, {"tipo": "Escuelas", "nombre": "Escuelas \"Escuela Secundaria
\" \"Juan Rulfo\", \"raw\": \"13 mins - 1,004 metros\", \"mins\": 13.0, \"metros\":
1004.0}, {"tipo": "Escuelas", "nombre": "Escuelas \"Escuela Primaria \" \"Jaime
Sabines\", \"raw\": \"14 mins - 1,100 metros\", \"mins\": 14.0, \"metros\": 1100.0}]
[{"tipo": "Plazas", "nombre": "Plazas Parque y Gimnasios Urbanos 222", "raw":
"11 mins - 878 metros", "mins": 11.0, "metros": 878.0}, {"tipo": "Plazas",
"nombre": "Plazas \"Parque \" \"Canchas Gitana\", \"raw\": \"14 mins - 1,108
metros\", \"mins\": 14.0, \"metros\": 1108.0}, {"tipo": "Plazas", "nombre": "Plazas
\"Parque \" \"Atlixco\", \"raw\": \"15 mins - 1,137 metros\", \"mins\": 15.0,
\"metros\": 1137.0}, {"tipo": "Plazas", "nombre": "Plazas Parque Nopalera", "raw":
"15 mins - 1,163 metros", "mins": 15.0, "metros": 1163.0}, {"tipo": "Plazas",
"nombre": "Plazas Deportivo Nopalera", "raw": "16 mins - 1,241 metros", "mins":
16.0, "metros": 1241.0}]
[{"tipo": "Supermercados", "nombre": "Supermercados Bodega Aurrera", "raw": "13
mins - 1,029 metros", "mins": 13.0, "metros": 1029.0}, {"tipo": "Supermercados",
"nombre": "Supermercados Mercado Nopalera", "raw": "13 mins - 1,041 metros",
"mins": 13.0, "metros": 1041.0}, {"tipo": "Supermercados", "nombre":
"Supermercados El Venadito", "raw": "15 mins - 1,153 metros", "mins": 15.0,
"metros": 1153.0}, {"tipo": "Supermercados", "nombre": "Supermercados Neto",
"raw": "15 mins - 1,176 metros", "mins": 15.0, "metros": 1176.0}, {"tipo":
"Supermercados", "nombre": "Supermercados Tienda BBB", "raw": "21 mins - 1,640
metros", "mins": 21.0, "metros": 1640.0}, {"tipo": "Farmacias", "nombre":
"Farmacias Farmacia Guadalajara", "raw": "6 mins - 437 metros", "mins": 6.0,
"metros": 437.0}, {"tipo": "Farmacias", "nombre": "Farmacias Farmacias de
Genéricos y de Patente", "raw": "16 mins - 1,248 metros", "mins": 16.0,
"metros": 1248.0}, {"tipo": "Farmacias", "nombre": "Farmacias Farmacia
Farmapronto", "raw": "17 mins - 1,341 metros", "mins": 17.0, "metros": 1341.0}]
[{"tipo": "Hospitales", "nombre": "Hospitales Sanatorio Psiquiatrico del
```

Carmen", "raw": "6 mins - 485 metros", "mins": 6.0, "metros": 485.0}, {"tipo":
 "Hospitales", "nombre": "Hospitales Hospital General Tláhuac", "raw": "18 mins -
 1,365 metros", "mins": 18.0, "metros": 1365.0}, {"tipo": "Hospitales", "nombre":
 "Hospitales Nuevo Hospital ISSSTE Tláhuac", "raw": "25 mins - 1,917 metros",
 "mins": 25.0, "metros": 1917.0}, {"tipo": "Hospitales", "nombre": "Hospitales
 IMSS UMA Unidad Medica Ambulatoria", "raw": "25 mins - 1,986 metros", "mins":
 25.0, "metros": 1986.0}] MXN 4,600,000 4600000.0 3.0 130.0
 3.0 2 13200.0
 Estaciones de metro Nopalera | 6 mins - 458 metros || Estaciones de metro
 Zapotitlán | 14 mins - 1,064 metros || Estaciones de metro Olivos | 24 mins -
 1,847 metros Estaciones de metro Nopalera | 6 mins - 458
 metros 6.0 458.0 Estaciones de metro
 Zapotitlán | 14 mins - 1,064 metros 14.0 1064.0
 Estaciones de metro Olivos | 24 mins - 1,847 metros 24.0
 1847.0 No hay más datos disponibles
 NaN NaN No hay más datos
 disponibles NaN NaN
 No hay más datos disponibles NaN NaN No hay más
 datos disponibles NaN NaN No hay más datos
 disponibles NaN NaN No hay más datos
 disponibles NaN NaN No hay más datos
 disponibles NaN NaN Escuelas Colegio de
 Culturas de México | 11 mins - 873 metros || Escuelas "Escuela Primaria
 "TlamachKali" 51 190" | 11 mins - 876 metros || Escuelas Escuela Primaria
 Oficial Ricardo Flores Magón | 11 mins - 884 metros || Escuelas "Escuela
 Secundaria "Juan Rulfo\" | 13 mins - 1,004 metros || Escuelas "Escuela Primaria
 "Jaime Sabines\" | 14 mins - 1,100 metros Escuelas
 Colegio de Culturas de México | 11 mins - 873 metros 11.0
 873.0 Escuelas "Escuela Primaria "TlamachKali" 51 190" | 11 mins
 - 876 metros 11.0 876.0 Escuelas Escuela Primaria
 Oficial Ricardo Flores Magón | 11 mins - 884 metros 11.0
 884.0 Escuelas "Escuela Secundaria "Juan Rulfo\" | 13
 mins - 1,004 metros 13.0 1004.0 Escuelas "Escuela
 Primaria "Jaime Sabines\" | 14 mins - 1,100 metros 14.0
 1100.0 No hay más datos disponibles NaN NaN No hay
 más datos disponibles NaN NaN No hay más datos
 disponibles NaN NaN No hay más datos disponibles
 NaN NaN No hay más datos disponibles NaN
 NaN No hay más datos disponibles NaN NaN No hay
 más datos disponibles NaN NaN No hay más datos
 disponibles NaN NaN No hay más datos disponibles
 NaN NaN No hay más datos disponibles NaN
 NaN Plazas Parque y Gimnasios Urbanos 222 | 11
 mins - 878 metros || Plazas "Parque "Canchas Gitana\" | 14 mins - 1,108 metros
 || Plazas "Parque "Atlixco\" | 15 mins - 1,137 metros || Plazas Parque Nopalera
 | 15 mins - 1,163 metros || Plazas Deportivo Nopalera | 16 mins - 1,241 metros
 Plazas Parque y Gimnasios Urbanos 222 | 11 mins - 878 metros 11.0
 878.0 Plazas "Parque "Canchas Gitana\" | 14 mins - 1,108 metros

14.0	1108.0	Plazas "Parque "Atlixco\" 15 mins -		
1,137 metros	15.0	1137.0		
Plazas Parque Nopalera 15 mins - 1,163 metros	15.0			
1163.0	Plazas Deportivo Nopalera 16 mins - 1,241 metros			
16.0	1241.0			
Supermercados Bodega Aurrera 13 mins - 1,029 metros Supermercados Mercado Nopalera 13 mins - 1,041 metros Supermercados El Venadito 15 mins - 1,153 metros Supermercados Neto 15 mins - 1,176 metros Supermercados Tienda BBB 21 mins - 1,640 metros Farmacias Farmacia Guadalajara 6 mins - 437 metros Farmacias Farmacias de Genéricos y de Patente 16 mins - 1,248 metros Farmacias Farmacia Farmapronto 17 mins - 1,341 metros				
Supermercados Bodega Aurrera 13 mins - 1,029 metros	13.0			
1029.0	Supermercados Mercado Nopalera 13 mins - 1,041 metros			
13.0	1041.0	Supermercados El Venadito 15 mins - 1,153 metros		
15.0	1153.0	Supermercados Neto 15 mins - 1,176 metros		
15.0	1176.0			
Supermercados Tienda BBB 21 mins - 1,640 metros	21.0			
1640.0	Farmacias Farmacia Guadalajara 6 mins - 437 metros	6.0		
437.0	Farmacias Farmacias de Genéricos y de Patente 16 mins - 1,248 metros			
16.0	1248.0	Farmacias Farmacia Farmapronto 17 mins - 1,341 metros		
17.0	1341.0			
No hay más datos disponibles	NaN	NaN	No hay más datos disponibles	
NaN	NaN	NaN	No hay más datos disponibles	
NaN	NaN	NaN	No hay más datos disponibles	
NaN	NaN	NaN	No hay más datos disponibles	
NaN	NaN	NaN	Hospitales Sanatorio Psiquiatrico del Carmen 6 mins - 485 metros Hospitales Hospital General Tláhuac 18 mins - 1,365 metros Hospitales Nuevo Hospital ISSSTE Tláhuac 25 mins - 1,917 metros Hospitales IMSS UMA Unidad Medica Ambulatoria 25 mins - 1,986 metros	
6.0	485.0	Hospitales Hospital General Tláhuac 18 mins - 1,365 metros	18.0	1365.0
18.0	1365.0	Hospitales Nuevo Hospital ISSSTE Tláhuac 25 mins - 1,917 metros	25.0	1917.0
25.0	1986.0	No hay más datos disponibles	NaN	NaN
No hay más datos disponibles	NaN	NaN	No hay más datos disponibles	
NaN	NaN	No hay más datos disponibles	NaN	NaN
No hay más datos disponibles	NaN	NaN		
14.666667	1123.000000	12.0	947.4	
14.2	1105.4	14.500000	1133.125	
18.500000	1438.250000			
1	https://casa.mercadolibre.com.mx/MLM-2578861711-reserva-de-sur-_JM#position=1&search_layout=map&type=item&tracking_id=61084625-8f1e-4422-85d4-2c587e5d786a			
	Reserva del sur Conrado Pelayo 67, Cp. 13200, Col. Miguel Hidalgo, Del. Tláhuac. Ciudad De México, Miguel Hidalgo,			

Distrito Federal <https://www.google.com/maps?q=19.296073,-99.0443251&z=16>
 19.296073 -99.044325 Transporte | Educación | Áreas verdes | Comercios | Salud
 Estaciones de metro | Nopalera | 6 mins - 458 metros\nEstaciones de metro |
 Zapotitlán | 14 mins - 1,064 metros\nEstaciones de metro | Olivos | 24 mins -
 1,847 metros Escuelas | Colegio de Culturas de México | 11 mins - 873
 metros\nEscuelas | "Escuela Primaria "TlamachKali" 51 190" | 11 mins - 876
 metros\nEscuelas | Escuela Primaria Oficial Ricardo Flores Magón | 11 mins - 884
 metros\nEscuelas | "Escuela Secundaria "Juan Rulfo\" | 13 mins - 1,004
 metros\nEscuelas | "Escuela Primaria "Jaime Sabines\" | 14 mins - 1,100 metros
 Plazas | Parque y Gimnasios Urbanos 222 | 11 mins - 878 metros\nPlazas | "Parque
 "Canchas Gitana\" | 14 mins - 1,108 metros\nPlazas | "Parque "Atlixco\" | 15
 mins - 1,137 metros\nPlazas | Parque Nopalera | 15 mins - 1,163 metros\nPlazas |
 Deportivo Nopalera | 16 mins - 1,241 metros
 Supermercados | Bodega Aurrera | 13 mins - 1,029 metros\nSupermercados | Mercado
 Nopalera | 13 mins - 1,041 metros\nSupermercados | El Venadito | 15 mins - 1,153
 metros\nSupermercados | Neto | 15 mins - 1,176 metros\nSupermercados | Tienda
 BBB | 21 mins - 1,640 metros\nFarmacias | Farmacia Guadalajara | 6 mins - 437
 metros\nFarmacias | Farmacias de Genéricos y de Patente | 16 mins - 1,248
 metros\nFarmacias | Farmacia Farmapronto | 17 mins - 1,341 metros Hospitales |
 Sanatorio Psiquiatrico del Carmen | 6 mins - 485 metros\nHospitales | Hospital
 General Tláhuac | 18 mins - 1,365 metros\nHospitales | Nuevo Hospital ISSSTE
 Tláhuac | 25 mins - 1,917 metros\nHospitales | IMSS UMA Unidad Medica
 Ambulatoria | 25 mins - 1,986 metros
 [{"tipo": "Estaciones de metro", "nombre": "Estaciones de metro Nopalera",
 "raw": "6 mins - 458 metros", "mins": 6.0, "metros": 458.0}, {"tipo":
 "Estaciones de metro", "nombre": "Estaciones de metro Zapotitlán", "raw": "14
 mins - 1,064 metros", "mins": 14.0, "metros": 1064.0}, {"tipo": "Estaciones de
 metro", "nombre": "Estaciones de metro Olivos", "raw": "24 mins - 1,847 metros",
 "mins": 24.0, "metros": 1847.0}] [{"tipo": "Escuelas", "nombre": "Escuelas
 Colegio de Culturas de México", "raw": "11 mins - 873 metros", "mins": 11.0,
 "metros": 873.0}, {"tipo": "Escuelas", "nombre": "Escuelas \"Escuela Primaria
 \"\"TlamachKali\"\" 51 190\"\", \"raw\": \"11 mins - 876 metros\", \"mins\": 11.0,
 \"metros\": 876.0}, {"tipo": "Escuelas", "nombre": "Escuelas Escuela Primaria
 Oficial Ricardo Flores Magón", "raw": "11 mins - 884 metros", "mins": 11.0,
 "metros": 884.0}, {"tipo": "Escuelas", "nombre": "Escuelas \"Escuela Secundaria
 \"\"Juan Rulfo\"\", \"raw\": \"13 mins - 1,004 metros\", \"mins\": 13.0, \"metros\":
 1004.0}, {"tipo": "Escuelas", "nombre": "Escuelas \"Escuela Primaria \"\"Jaime
 Sabines\"\", \"raw\": \"14 mins - 1,100 metros\", \"mins\": 14.0, \"metros\": 1100.0}]
 [{"tipo": "Plazas", "nombre": "Plazas Parque y Gimnasios Urbanos 222", "raw":
 "11 mins - 878 metros", "mins": 11.0, "metros": 878.0}, {"tipo": "Plazas",
 "nombre": "Plazas \"Parque \"\"Canchas Gitana\"\", \"raw\": \"14 mins - 1,108
 metros\", \"mins\": 14.0, \"metros\": 1108.0}, {"tipo": "Plazas", "nombre": "Plazas
 \"Parque \"\"Atlixco\"\", \"raw\": \"15 mins - 1,137 metros\", \"mins\": 15.0,
 \"metros\": 1137.0}, {"tipo": "Plazas", "nombre": "Plazas Parque Nopalera", "raw":
 "15 mins - 1,163 metros", "mins": 15.0, "metros": 1163.0}, {"tipo": "Plazas",
 "nombre": "Plazas Deportivo Nopalera", "raw": "16 mins - 1,241 metros", "mins":
 16.0, "metros": 1241.0}]
 [{"tipo": "Supermercados", "nombre": "Supermercados Bodega Aurrera", "raw": "13

```

mins - 1,029 metros", "mins": 13.0, "metros": 1029.0}, {"tipo": "Supermercados",
"nombre": "Supermercados Mercado Nopalera", "raw": "13 mins - 1,041 metros",
"mins": 13.0, "metros": 1041.0}, {"tipo": "Supermercados", "nombre":
"Supermercados El Venadito", "raw": "15 mins - 1,153 metros", "mins": 15.0,
"metros": 1153.0}, {"tipo": "Supermercados", "nombre": "Supermercados Neto",
"raw": "15 mins - 1,176 metros", "mins": 15.0, "metros": 1176.0}, {"tipo":
"Supermercados", "nombre": "Supermercados Tienda BBB", "raw": "21 mins - 1,640
metros", "mins": 21.0, "metros": 1640.0}, {"tipo": "Farmacias", "nombre":
"Farmacias Farmacia Guadalajara", "raw": "6 mins - 437 metros", "mins": 6.0,
"metros": 437.0}, {"tipo": "Farmacias", "nombre": "Farmacias Farmacias de
Genéricos y de Patente", "raw": "16 mins - 1,248 metros", "mins": 16.0,
"metros": 1248.0}, {"tipo": "Farmacias", "nombre": "Farmacias Farmacia
Farmapronto", "raw": "17 mins - 1,341 metros", "mins": 17.0, "metros": 1341.0}]
[{"tipo": "Hospitales", "nombre": "Hospitales Sanatorio Psiquiatrico del
Carmen", "raw": "6 mins - 485 metros", "mins": 6.0, "metros": 485.0}, {"tipo":
"Hospitales", "nombre": "Hospitales Hospital General Tláhuac", "raw": "18 mins -
1,365 metros", "mins": 18.0, "metros": 1365.0}, {"tipo": "Hospitales", "nombre":
"Hospitales Nuevo Hospital ISSSTE Tláhuac", "raw": "25 mins - 1,917 metros",
"mins": 25.0, "metros": 1917.0}, {"tipo": "Hospitales", "nombre": "Hospitales
IMSS UMA Unidad Medica Ambulatoria", "raw": "25 mins - 1,986 metros", "mins":
25.0, "metros": 1986.0}] MXN 4,600,000 4600000.0 3.0 130.0
3.0 2 13200.0

```

```

Estaciones de metro Nopalera | 6 mins - 458 metros || Estaciones de metro
Zapotitlán | 14 mins - 1,064 metros || Estaciones de metro Olivos | 24 mins -
1,847 metros Estaciones de metro Nopalera | 6 mins - 458
metros 6.0 458.0 Estaciones de metro
Zapotitlán | 14 mins - 1,064 metros 14.0 1064.0
Estaciones de metro Olivos | 24 mins - 1,847 metros 24.0
1847.0 No hay más datos disponibles
NaN NaN No hay más datos
disponibles NaN NaN
No hay más datos disponibles NaN NaN No hay más
datos disponibles NaN NaN No hay más datos
disponibles NaN NaN No hay más datos
disponibles NaN NaN No hay más datos
disponibles NaN NaN Escuelas Colegio de
Culturas de México | 11 mins - 873 metros || Escuelas "Escuela Primaria
""TlamachKali"" 51 190" | 11 mins - 876 metros || Escuelas Escuela Primaria
Oficial Ricardo Flores Magón | 11 mins - 884 metros || Escuelas "Escuela
Secundaria ""Juan Rulfo\" | 13 mins - 1,004 metros || Escuelas "Escuela Primaria
""Jaime Sabines\" | 14 mins - 1,100 metros Escuelas
Colegio de Culturas de México | 11 mins - 873 metros 11.0
873.0 Escuelas "Escuela Primaria ""TlamachKali"" 51 190" | 11 mins
- 876 metros 11.0 876.0 Escuelas Escuela Primaria
Oficial Ricardo Flores Magón | 11 mins - 884 metros 11.0
884.0 Escuelas "Escuela Secundaria ""Juan Rulfo\" | 13
mins - 1,004 metros 13.0 1004.0 Escuelas "Escuela
Primaria ""Jaime Sabines\" | 14 mins - 1,100 metros 14.0

```

1100.0	No hay más datos disponibles	NaN	NaN	No hay más datos disponibles
más datos disponibles	NaN	NaN	NaN	No hay más datos disponibles
disponibles	NaN	NaN	NaN	No hay más datos disponibles
NaN	NaN	No hay más datos disponibles	NaN	NaN
NaN	No hay más datos disponibles	NaN	NaN	No hay más datos disponibles
más datos disponibles	NaN	NaN	NaN	No hay más datos disponibles
disponibles	NaN	NaN	NaN	No hay más datos disponibles
NaN	NaN	No hay más datos disponibles	NaN	NaN
NaN	Plazas Parque y Gimnasios Urbanos 222 11 mins - 878 metros Plazas "Parque "Canchas Gitana\" 14 mins - 1,108 metros			
Plazas "Parque "Atlixco\" 15 mins - 1,137 metros Plazas Parque Nopalera 15 mins - 1,163 metros Plazas Deportivo Nopalera 16 mins - 1,241 metros				
Plazas Parque y Gimnasios Urbanos 222 11 mins - 878 metros 11.0				
878.0 Plazas "Parque "Canchas Gitana\" 14 mins - 1,108 metros				
14.0 1108.0 Plazas "Parque "Atlixco\" 15 mins - 1,137 metros 15.0 1137.0				
Plazas Parque Nopalera 15 mins - 1,163 metros 15.0				
1163.0 Plazas Deportivo Nopalera 16 mins - 1,241 metros 16.0 1241.0				
Supermercados Bodega Aurrera 13 mins - 1,029 metros Supermercados Mercado Nopalera 13 mins - 1,041 metros Supermercados El Venadito 15 mins - 1,153 metros Supermercados Neto 15 mins - 1,176 metros Supermercados Tienda BBB 21 mins - 1,640 metros Farmacias Farmacia Guadalajara 6 mins - 437 metros Farmacias Farmacias de Genéricos y de Patente 16 mins - 1,248 metros Farmacias Farmacia Farmapronto 17 mins - 1,341 metros				
Supermercados Bodega Aurrera 13 mins - 1,029 metros 13.0				
1029.0 Supermercados Mercado Nopalera 13 mins - 1,041 metros 13.0 1041.0 Supermercados El Venadito 15 mins - 1,153 metros 15.0 1153.0 Supermercados Neto 15 mins - 1,176 metros 15.0 1176.0				
Supermercados Tienda BBB 21 mins - 1,640 metros 21.0				
1640.0 Farmacias Farmacia Guadalajara 6 mins - 437 metros 6.0				
437.0 Farmacias Farmacias de Genéricos y de Patente 16 mins - 1,248 metros 16.0 1248.0 Farmacias Farmacia Farmapronto 17 mins - 1,341 metros 17.0 1341.0				
No hay más datos disponibles NaN NaN NaN No hay más datos disponibles				
disponibles NaN NaN NaN No hay más datos disponibles				
disponibles NaN NaN NaN No hay más datos disponibles				
disponibles NaN NaN NaN No hay más datos disponibles				
disponibles NaN NaN NaN No hay más datos disponibles				
disponibles NaN NaN NaN Hospitales Sanatorio Psiquiatrico del Carmen 6 mins - 485 metros Hospitales Hospital General Tláhuac 18 mins - 1,365 metros Hospitales Nuevo Hospital ISSSTE Tláhuac 25 mins - 1,917 metros Hospitales IMSS UMA Unidad Medica Ambulatoria 25 mins - 1,986 metros Hospitales Sanatorio Psiquiatrico del Carmen 6 mins - 485 metros 6.0 485.0 Hospitales Hospital General Tláhuac 18 mins - 1,365 metros 18.0 1365.0 Hospitales Nuevo Hospital				

ISSSTE Tláhuac | 25 mins - 1,917 metros 25.0 1917.0
 Hospitales IMSS UMA Unidad Medica Ambulatoria | 25 mins - 1,986 metros
 25.0 1986.0 No hay más datos disponibles NaN NaN
 No hay más datos disponibles NaN NaN No hay más datos
 disponibles NaN NaN No hay más datos disponibles
 NaN NaN No hay más datos disponibles NaN NaN
 No hay más datos disponibles NaN NaN
 14.666667 1123.000000 12.0 947.4
 14.2 1105.4 14.500000 1133.125
 18.500000 1438.250000
 2 https://casa.mercadolibre.com.mx/MLM-2549011325-vitant-santa-fe-by-be-grand-_JM#position=2&search_layout=map&type=item&tracking_id=61084625-8f1e-4422-85d4-2c587e5d786a Vitant Santa Fe By Be Grand Av. Santa Fe 578, Lomas De Santa Fe, Contadero, Ciudad De México, Contadero, Cuajimalpa De Morelos, Distrito Federal <https://www.google.com/maps?q=19.3560228,-99.275232&z=16>
 19.356023 -99.275232 Transporte | Educación | Áreas verdes | Comercios | Salud Estaciones de metro | "Estación ""Santa Fé"" | 23 mins - 1,820 metros\nParaderos | Avenida Santa Fe - Enrique del Moral | 6 mins - 472 metros\nParaderos | Juan Salvador Argaz - Office Depot | 6 mins - 505 metros\nParaderos | Juan Salvador Argaz - Office Depot | 7 mins - 582 metros\nParaderos | Avenida Santa Fe - Juan O'Gorman | 10 mins - 774 metros\nParaderos | Avenida Santa Fe - Juan O'Gorman | 10 mins - 783 metros Jardines de niños | Jardin de Niños Carmen Gonzalez de la Vega | 22 mins - 1,727 metros\nEscuelas | Colegio Eton | 25 mins - 1,926 metros\nUniversidades | UVM - Santa Fe | 7 mins - 515 metros\nUniversidades | Universidad Autónoma Metropolitana, Unidad Cuajimalpa | 14 mins - 1,110 metros\nUniversidades | Universidad Iberoamericana | 25 mins - 1,972 metros Plazas | La Mexicana | 3 mins - 261 metros\nPlazas | La Mexicana | 4 mins - 278 metros\nPlazas | Parque Exterior UAM Cuajimalpa | 13 mins - 1,037 metros\nPlazas | Jardín Alrededor de Placa Conmemorativa Club de Rotarios Cuajimalpa | 14 mins - 1,094 metros\nPlazas | Entrada UAM Cuajimalpa | 14 mins - 1,121 metros Supermercados | MeatMe Santa Fe | 0 mins - 1 metro\nSupermercados | City Market Santa Fé | 5 mins - 411 metros\nSupermercados | Chedraui Selecto | 17 mins - 1,324 metros\nSupermercados | OXXO Lomas Contadero | 20 mins - 1,554 metros\nSupermercados | Miscelania Chayito | 20 mins - 1,556 metros\nFarmacias | Farmacia Guadalajara | 0 mins - 15 metros\nFarmacias | San Pablo | 17 mins - 1,338 metros\nCentros comerciales | Zentrika | 9 mins - 725 metros\nCentros comerciales | Corporativo Diamante | 13 mins - 980 metros Hospitales | Centro Medico ABC | 19 mins - 1,486 metros\nClínicas | Clínica de Vértigo y Mareo | 18 mins - 1,397 metros\nClínicas | Centro de Salud T1 Memetla | 24 mins - 1,888 metros [{"tipo": "Estaciones de metro", "nombre": "Estaciones de metro \"Estación \"\"Santa Fé\"\"\"", "raw": "23 mins - 1,820 metros", "mins": 23.0, "metros": 1820.0}, {"tipo": "Paraderos", "nombre": "Paraderos Avenida Santa Fe - Enrique del Moral", "raw": "6 mins - 472 metros", "mins": 6.0, "metros": 472.0}, {"tipo": "Paraderos", "nombre": "Paraderos Juan Salvador Argaz - Office Depot", "raw": "6 mins - 505 metros", "mins": 6.0, "metros": 505.0}, {"tipo": "Paraderos", "nombre": "Paraderos Juan Salvador Argaz - Office Depot", "raw": "7 mins - 582 metros", "mins": 7.0, "metros": 582.0}, {"tipo": "Paraderos", "nombre": "Paraderos Avenida Santa Fe - Juan O'Gorman", "raw": "10

mins - 774 metros", "mins": 10.0, "metros": 774.0}, {"tipo": "Paraderos", "nombre": "Paraderos Avenida Santa Fe - Juan O'Gorman", "raw": "10 mins - 783 metros", "mins": 10.0, "metros": 783.0}] [{"tipo": "Jardines de niños", "nombre": "Jardines de niños Jardin de Niños Carmen Gonzalez de la Vega", "raw": "22 mins - 1,727 metros", "mins": 22.0, "metros": 1727.0}, {"tipo": "Escuelas", "nombre": "Escuelas Colegio Eton", "raw": "25 mins - 1,926 metros", "mins": 25.0, "metros": 1926.0}, {"tipo": "Universidades", "nombre": "Universidades UVM - Santa Fe", "raw": "7 mins - 515 metros", "mins": 7.0, "metros": 515.0}, {"tipo": "Universidades", "nombre": "Universidades Universidad Autónoma Metropolitana, Unidad Cuajimalpa", "raw": "14 mins - 1,110 metros", "mins": 14.0, "metros": 1110.0}, {"tipo": "Universidades", "nombre": "Universidades Universidad Iberoamericana", "raw": "25 mins - 1,972 metros", "mins": 25.0, "metros": 1972.0}] [{"tipo": "Plazas", "nombre": "Plazas La Mexicana", "raw": "3 mins - 261 metros", "mins": 3.0, "metros": 261.0}, {"tipo": "Plazas", "nombre": "Plazas La Mexicana", "raw": "4 mins - 278 metros", "mins": 4.0, "metros": 278.0}, {"tipo": "Plazas", "nombre": "Plazas Parque Exterior UAM Cuajimalpa", "raw": "13 mins - 1,037 metros", "mins": 13.0, "metros": 1037.0}, {"tipo": "Plazas", "nombre": "Plazas Jardín Alrededor de Placa Conmemorativa Club de Rotarios Cuajimalpa", "raw": "14 mins - 1,094 metros", "mins": 14.0, "metros": 1094.0}, {"tipo": "Plazas", "nombre": "Plazas Entrada UAM Cuajimalpa", "raw": "14 mins - 1,121 metros", "mins": 14.0, "metros": 1121.0}] [{"tipo": "Supermercados", "nombre": "Supermercados MeatMe Santa Fe", "raw": "0 mins - 1 metro", "mins": 0.0, "metros": null}, {"tipo": "Supermercados", "nombre": "Supermercados City Market Santa Fé", "raw": "5 mins - 411 metros", "mins": 5.0, "metros": 411.0}, {"tipo": "Supermercados", "nombre": "Supermercados Chedraui Selecto", "raw": "17 mins - 1,324 metros", "mins": 17.0, "metros": 1324.0}, {"tipo": "Supermercados", "nombre": "Supermercados OXXO Lomas Contadero", "raw": "20 mins - 1,554 metros", "mins": 20.0, "metros": 1554.0}, {"tipo": "Supermercados", "nombre": "Supermercados Miscelania Chayito", "raw": "20 mins - 1,556 metros", "mins": 20.0, "metros": 1556.0}, {"tipo": "Farmacias", "nombre": "Farmacias Farmacia Guadalajara", "raw": "0 mins - 15 metros", "mins": 0.0, "metros": 15.0}, {"tipo": "Farmacias", "nombre": "Farmacias San Pablo", "raw": "17 mins - 1,338 metros", "mins": 17.0, "metros": 1338.0}, {"tipo": "Centros comerciales", "nombre": "Centros comerciales Zentrika", "raw": "9 mins - 725 metros", "mins": 9.0, "metros": 725.0}, {"tipo": "Centros comerciales", "nombre": "Centros comerciales Corporativo Diamante", "raw": "13 mins - 980 metros", "mins": 13.0, "metros": 980.0}] [{"tipo": "Hospitales", "nombre": "Hospitales Centro Medico ABC", "raw": "19 mins - 1,486 metros", "mins": 19.0, "metros": 1486.0}, {"tipo": "Clínicas", "nombre": "Clínicas Clínica de Vértigo y Mareo", "raw": "18 mins - 1,397 metros", "mins": 18.0, "metros": 1397.0}, {"tipo": "Clínicas", "nombre": "Clínicas Centro de Salud T1 Memetla", "raw": "24 mins - 1,888 metros", "mins": 24.0, "metros": 1888.0}] MXN 3,700,000 3700000.0 1.0 65.0 1.0 1 5348.0 Estaciones de metro "Estación ""Santa Fé"" | 23 mins - 1,820 metros || Paraderos Avenida Santa Fe - Enrique del Moral | 6 mins - 472 metros || Paraderos Juan Salvador Argaz - Office Depot | 6 mins - 505 metros || Paraderos Juan Salvador Argaz - Office Depot | 7 mins - 582 metros || Paraderos Avenida Santa Fe - Juan O'Gorman | 10 mins - 774 metros || Paraderos Avenida

Santa Fe - Juan O'Gorman | 10 mins - 783 metros Estaciones de metro "Estación
 "Santa Fé\" | 23 mins - 1,820 metros 23.0 1820.0
 Paraderos Avenida Santa Fe - Enrique del Moral | 6 mins - 472 metros
 6.0 472.0 Paraderos Juan Salvador Argaz - Office Depot | 6 mins
 - 505 metros 6.0 505.0 Paraderos Juan Salvador
 Argaz - Office Depot | 7 mins - 582 metros 7.0
 582.0 Paraderos Avenida Santa Fe - Juan O'Gorman | 10 mins - 774 metros
 10.0 774.0 Paraderos Avenida Santa Fe - Juan O'Gorman | 10 mins
 - 783 metros 10.0 783.0 No hay más datos
 disponibles NaN NaN No hay más datos
 disponibles NaN NaN No hay más datos
 disponibles NaN NaN No hay más datos
 disponibles NaN NaN Jardines de
 niños Jardin de Niños Carmen Gonzalez de la Vega | 22 mins - 1,727 metros ||
 Escuelas Colegio Eton | 25 mins - 1,926 metros || Universidades UVM - Santa Fe |
 7 mins - 515 metros || Universidades Universidad Autónoma Metropolitana, Unidad
 Cuajimalpa | 14 mins - 1,110 metros || Universidades Universidad Iberoamericana
 | 25 mins - 1,972 metros Jardines de niños Jardin de Niños Carmen Gonzalez de
 la Vega | 22 mins - 1,727 metros 22.0 1727.0
 Escuelas Colegio Eton | 25 mins - 1,926 metros 25.0
 1926.0 Universidades UVM - Santa Fe | 7 mins - 515
 metros 7.0 515.0 Universidades Universidad Autónoma
 Metropolitana, Unidad Cuajimalpa | 14 mins - 1,110 metros 14.0
 1110.0 Universidades Universidad Iberoamericana | 25 mins - 1,972 metros
 25.0 1972.0 No hay más datos disponibles NaN
 NaN No hay más datos disponibles NaN NaN No hay
 más datos disponibles NaN NaN No hay más datos
 disponibles NaN NaN No hay más datos disponibles
 NaN NaN No hay más datos disponibles NaN
 NaN No hay más datos disponibles NaN NaN No hay
 más datos disponibles NaN NaN No hay más datos
 disponibles NaN NaN No hay más datos disponibles
 NaN NaN Plazas La Mexicana | 3 mins - 261 metros || Plazas La
 Mexicana | 4 mins - 278 metros || Plazas Parque Exterior UAM Cuajimalpa | 13
 mins - 1,037 metros || Plazas Jardín Alrededor de Placa Conmemorativa Club de
 Rotarios Cuajimalpa | 14 mins - 1,094 metros || Plazas Entrada UAM Cuajimalpa |
 14 mins - 1,121 metros Plazas La Mexicana | 3 mins - 261
 metros 3.0 261.0 Plazas La
 Mexicana | 4 mins - 278 metros 4.0 278.0 Plazas
 Parque Exterior UAM Cuajimalpa | 13 mins - 1,037 metros 13.0
 1037.0 Plazas Jardín Alrededor de Placa Conmemorativa Club de Rotarios
 Cuajimalpa | 14 mins - 1,094 metros 14.0 1094.0
 Plazas Entrada UAM Cuajimalpa | 14 mins - 1,121 metros 14.0
 1121.0 Supermercados MeatMe Santa Fe | 0 mins - 1 metro || Supermercados City
 Market Santa Fé | 5 mins - 411 metros || Supermercados Chedraui Selecto | 17
 mins - 1,324 metros || Supermercados OXXO Lomas Contadero | 20 mins - 1,554
 metros || Supermercados Miscelania Chayito | 20 mins - 1,556 metros || Farmacias
 Farmacia Guadalajara | 0 mins - 15 metros || Farmacias San Pablo | 17 mins -

1,338 metros || Centros comerciales Zentrika | 9 mins - 725 metros || Centros comerciales Corporativo Diamante | 13 mins - 980 metros

Supermercados MeatMe Santa Fe | 0 mins - 1 metro 0.0

NaN Supermercados City Market Santa Fé | 5 mins - 411 metros 5.0

411.0 Supermercados Chedraui Selecto | 17 mins - 1,324 metros

17.0 1324.0 Supermercados OXXO Lomas Contadero | 20 mins - 1,554 metros

20.0 1554.0 Supermercados Miscelania Chayito | 20 mins - 1,556 metros 20.0 1556.0 Farmacias

Farmacia Guadalajara | 0 mins - 15 metros 0.0 15.0

Farmacias San Pablo | 17 mins - 1,338 metros 17.0

1338.0 Centros comerciales Zentrika | 9 mins - 725 metros

9.0 725.0 Centros comerciales Corporativo Diamante | 13 mins - 980 metros 13.0 980.0 No hay más datos disponibles

NaN NaN No hay más datos disponibles NaN

NaN No hay más datos disponibles NaN NaN No hay más datos disponibles

NaN NaN No hay más datos disponibles NaN NaN No hay más datos disponibles

Hospitales Centro Medico ABC | 19 mins - 1,486 metros || Clínicas Clínica de Vértigo y Mareo | 18 mins - 1,397 metros || Clínicas Centro de Salud T1 Memetla | 24 mins - 1,888 metros

Hospitales Centro Medico ABC | 19 mins - 1,486 metros 19.0 1486.0 Clínicas Clínica de Vértigo y Mareo | 18 mins - 1,397 metros 18.0 1397.0 Clínicas Centro de Salud T1 Memetla | 24 mins - 1,888 metros 24.0 1888.0 No hay más datos disponibles

NaN NaN No hay más datos disponibles NaN NaN

No hay más datos disponibles NaN NaN No hay más datos disponibles NaN

NaN NaN No hay más datos disponibles NaN NaN

No hay más datos disponibles NaN NaN

10.333333 822.666667 18.6 1450.0

9.6 758.2 11.222222 987.875

20.333333 1590.333333

3 https://casa.mercadolibre.com.mx/MLM-2549011325-vitant-santa-fe-by-be-grand-_JM#position=2&search_layout=map&type=item&tracking_id=61084625-8f1e-4422-85d4-2c587e5d786a Vitant Santa Fe By Be Grand Av. Santa Fe 578, Lomas De Santa Fe, Contadero, Ciudad De México, Contadero, Cuajimalpa De Morelos, Distrito Federal <https://www.google.com/maps?q=19.3560228,-99.275232&z=16>

19.356023 -99.275232 Transporte | Educación | Áreas verdes | Comercios | Salud Estaciones de metro | "Estación ""Santa Fé"" | 23 mins - 1,820 metros\nParaderos | Avenida Santa Fe - Enrique del Moral | 6 mins - 472 metros\nParaderos | Juan Salvador Argaz - Office Depot | 6 mins - 505 metros\nParaderos | Juan Salvador Argaz - Office Depot | 7 mins - 582 metros\nParaderos | Avenida Santa Fe - Juan O'Gorman | 10 mins - 774 metros\nParaderos | Avenida Santa Fe - Juan O'Gorman | 10 mins - 783 metros Jardines de niños | Jardin de Niños Carmen Gonzalez de la Vega | 22 mins - 1,727 metros\nEscuelas | Colegio Eton | 25 mins - 1,926 metros\nUniversidades | UVM - Santa Fe | 7 mins - 515 metros\nUniversidades | Universidad Autónoma Metropolitana, Unidad Cuajimalpa |

14 mins - 1,110 metros\nUniversidades | Universidad Iberoamericana | 25 mins - 1,972 metros Plazas | La Mexicana | 3 mins - 261 metros\nPlazas | La Mexicana | 4 mins - 278 metros\nPlazas | Parque Exterior UAM Cuajimalpa | 13 mins - 1,037 metros\nPlazas | Jardín Alrededor de Placa Conmemorativa Club de Rotarios Cuajimalpa | 14 mins - 1,094 metros\nPlazas | Entrada UAM Cuajimalpa | 14 mins - 1,121 metros Supermercados | MeatMe Santa Fe | 0 mins - 1 metro\nSupermercados | City Market Santa Fé | 5 mins - 411 metros\nSupermercados | Chedraui Selecto | 17 mins - 1,324 metros\nSupermercados | OXXO Lomas Contadero | 20 mins - 1,554 metros\nSupermercados | Miscelania Chayito | 20 mins - 1,556 metros\nFarmacias | Farmacia Guadalajara | 0 mins - 15 metros\nFarmacias | San Pablo | 17 mins - 1,338 metros\nCentros comerciales | Zentrika | 9 mins - 725 metros\nCentros comerciales | Corporativo Diamante | 13 mins - 980 metros Hospitales | Centro Medico ABC | 19 mins - 1,486 metros\nClínicas | Clínica de Vértigo y Mareo | 18 mins - 1,397 metros\nClínicas | Centro de Salud T1 Memetla | 24 mins - 1,888 metros [{"tipo": "Estaciones de metro", "nombre": "Estaciones de metro \"Estación \"\"Santa Fé\\\"\", \"raw\": \"23 mins - 1,820 metros\", \"mins\": 23.0, \"metros\": 1820.0}, {\"tipo\": \"Paraderos\", \"nombre\": \"Paraderos Avenida Santa Fe - Enrique del Moral\", \"raw\": \"6 mins - 472 metros\", \"mins\": 6.0, \"metros\": 472.0}, {\"tipo\": \"Paraderos\", \"nombre\": \"Paraderos Juan Salvador Argaz - Office Depot\", \"raw\": \"6 mins - 505 metros\", \"mins\": 6.0, \"metros\": 505.0}, {\"tipo\": \"Paraderos\", \"nombre\": \"Paraderos Juan Salvador Argaz - Office Depot\", \"raw\": \"7 mins - 582 metros\", \"mins\": 7.0, \"metros\": 582.0}, {\"tipo\": \"Paraderos\", \"nombre\": \"Paraderos Avenida Santa Fe - Juan O'Gorman\", \"raw\": \"10 mins - 774 metros\", \"mins\": 10.0, \"metros\": 774.0}, {\"tipo\": \"Paraderos\", \"nombre\": \"Paraderos Avenida Santa Fe - Juan O'Gorman\", \"raw\": \"10 mins - 783 metros\", \"mins\": 10.0, \"metros\": 783.0}] [{"tipo\": \"Jardines de niños\", \"nombre\": \"Jardines de niños Jardin de Niños Carmen Gonzalez de la Vega\", \"raw\": \"22 mins - 1,727 metros\", \"mins\": 22.0, \"metros\": 1727.0}, {\"tipo\": \"Escuelas\", \"nombre\": \"Escuelas Colegio Eton\", \"raw\": \"25 mins - 1,926 metros\", \"mins\": 25.0, \"metros\": 1926.0}, {\"tipo\": \"Universidades\", \"nombre\": \"Universidades UVM - Santa Fe\", \"raw\": \"7 mins - 515 metros\", \"mins\": 7.0, \"metros\": 515.0}, {\"tipo\": \"Universidades\", \"nombre\": \"Universidades Universidad Autónoma Metropolitana, Unidad Cuajimalpa\", \"raw\": \"14 mins - 1,110 metros\", \"mins\": 14.0, \"metros\": 1110.0}, {\"tipo\": \"Universidades\", \"nombre\": \"Universidades Universidad Iberoamericana\", \"raw\": \"25 mins - 1,972 metros\", \"mins\": 25.0, \"metros\": 1972.0}] [{"tipo\": \"Plazas\", \"nombre\": \"Plazas La Mexicana\", \"raw\": \"3 mins - 261 metros\", \"mins\": 3.0, \"metros\": 261.0}, {\"tipo\": \"Plazas\", \"nombre\": \"Plazas La Mexicana\", \"raw\": \"4 mins - 278 metros\", \"mins\": 4.0, \"metros\": 278.0}, {\"tipo\": \"Plazas\", \"nombre\": \"Plazas Parque Exterior UAM Cuajimalpa\", \"raw\": \"13 mins - 1,037 metros\", \"mins\": 13.0, \"metros\": 1037.0}, {\"tipo\": \"Plazas\", \"nombre\": \"Plazas Jardín Alrededor de Placa Conmemorativa Club de Rotarios Cuajimalpa\", \"raw\": \"14 mins - 1,094 metros\", \"mins\": 14.0, \"metros\": 1094.0}, {\"tipo\": \"Plazas\", \"nombre\": \"Plazas Entrada UAM Cuajimalpa\", \"raw\": \"14 mins - 1,121 metros\", \"mins\": 14.0, \"metros\": 1121.0}] [{\"tipo\": \"Supermercados\", \"nombre\": \"Supermercados MeatMe Santa Fe\", \"raw\": \"0 mins - 1 metro\", \"mins\": 0.0, \"metros\": null}, {\"tipo\": \"Supermercados\", \"nombre\": \"Supermercados City Market Santa Fé\", \"raw\": \"5 mins - 411 metros\", \"mins\": 5.0, \"metros\": 411.0}, {\"tipo\": \"Supermercados\", \"nombre\": \"Supermercados Chedraui

Selecto", "raw": "17 mins - 1,324 metros", "mins": 17.0, "metros": 1324.0}, {"tipo": "Supermercados", "nombre": "Supermercados OXXO Lomas Contadero", "raw": "20 mins - 1,554 metros", "mins": 20.0, "metros": 1554.0}, {"tipo": "Supermercados", "nombre": "Supermercados Miscelania Chayito", "raw": "20 mins - 1,556 metros", "mins": 20.0, "metros": 1556.0}, {"tipo": "Farmacias", "nombre": "Farmacias Farmacia Guadalajara", "raw": "0 mins - 15 metros", "mins": 0.0, "metros": 15.0}, {"tipo": "Farmacias", "nombre": "Farmacias San Pablo", "raw": "17 mins - 1,338 metros", "mins": 17.0, "metros": 1338.0}, {"tipo": "Centros comerciales", "nombre": "Centros comerciales Zentrika", "raw": "9 mins - 725 metros", "mins": 9.0, "metros": 725.0}, {"tipo": "Centros comerciales", "nombre": "Centros comerciales Corporativo Diamante", "raw": "13 mins - 980 metros", "mins": 13.0, "metros": 980.0}] [{"tipo": "Hospitales", "nombre": "Hospitales Centro Medico ABC", "raw": "19 mins - 1,486 metros", "mins": 19.0, "metros": 1486.0}, {"tipo": "Clínicas", "nombre": "Clínicas Clínica de Vértigo y Mareo", "raw": "18 mins - 1,397 metros", "mins": 18.0, "metros": 1397.0}, {"tipo": "Clínicas", "nombre": "Clínicas Centro de Salud T1 Memetla", "raw": "24 mins - 1,888 metros", "mins": 24.0, "metros": 1888.0}] MXN 3,700,000 3700000.0 1.0 65.0 1.0 1 5348.0 Estaciones de metro "Estación "Santa Fé\" | 23 mins - 1,820 metros || Paraderos Avenida Santa Fe - Enrique del Moral | 6 mins - 472 metros || Paraderos Juan Salvador Argaz - Office Depot | 6 mins - 505 metros || Paraderos Juan Salvador Argaz - Office Depot | 7 mins - 582 metros || Paraderos Avenida Santa Fe - Juan O'Gorman | 10 mins - 774 metros || Paraderos Avenida Santa Fe - Juan O'Gorman | 10 mins - 783 metros Estaciones de metro "Estación "Santa Fé\" | 23 mins - 1,820 metros 23.0 1820.0 Paraderos Avenida Santa Fe - Enrique del Moral | 6 mins - 472 metros 6.0 472.0 Paraderos Juan Salvador Argaz - Office Depot | 6 mins - 505 metros 6.0 505.0 Paraderos Juan Salvador Argaz - Office Depot | 7 mins - 582 metros 7.0 582.0 Paraderos Avenida Santa Fe - Juan O'Gorman | 10 mins - 774 metros 10.0 774.0 Paraderos Avenida Santa Fe - Juan O'Gorman | 10 mins - 783 metros 10.0 783.0 No hay más datos disponibles NaN NaN No hay más datos disponibles NaN NaN No hay más datos disponibles NaN NaN No hay más datos disponibles NaN NaN Jardines de niños Jardin de Niños Carmen Gonzalez de la Vega | 22 mins - 1,727 metros || Escuelas Colegio Eton | 25 mins - 1,926 metros || Universidades UVM - Santa Fe | 7 mins - 515 metros || Universidades Universidad Autónoma Metropolitana, Unidad Cuajimalpa | 14 mins - 1,110 metros || Universidades Universidad Iberoamericana | 25 mins - 1,972 metros Jardines de niños Jardin de Niños Carmen Gonzalez de la Vega | 22 mins - 1,727 metros 22.0 1727.0 Escuelas Colegio Eton | 25 mins - 1,926 metros 25.0 1926.0 Universidades UVM - Santa Fe | 7 mins - 515 metros 7.0 515.0 Universidades Universidad Autónoma Metropolitana, Unidad Cuajimalpa | 14 mins - 1,110 metros 14.0 1110.0 Universidades Universidad Iberoamericana | 25 mins - 1,972 metros 25.0 1972.0 No hay más datos disponibles NaN

NaN	No hay más datos disponibles		NaN		NaN	No hay más datos disponibles
más datos disponibles		NaN		NaN	No hay más datos disponibles	
disponibles	NaN		NaN	No hay más datos disponibles		NaN
NaN	NaN	No hay más datos disponibles				NaN
NaN	No hay más datos disponibles		NaN		NaN	No hay más datos disponibles
más datos disponibles		NaN		NaN	No hay más datos disponibles	
disponibles	NaN		NaN	No hay más datos disponibles		NaN
NaN	NaN	Plazas La Mexicana 3 mins - 261 metros Plazas La Mexicana 4 mins - 278 metros Plazas Parque Exterior UAM Cuajimalpa 13 mins - 1,037 metros Plazas Jardín Alrededor de Placa Conmemorativa Club de Rotarios Cuajimalpa 14 mins - 1,094 metros Plazas Entrada UAM Cuajimalpa 14 mins - 1,121 metros		Plazas La Mexicana 3 mins - 261 metros		Plazas La Mexicana 4 mins - 278 metros
metros	3.0		261.0		Plazas La Mexicana 4 mins - 278 metros	
Mexicana 4 mins - 278 metros		4.0		278.0	Plazas Parque Exterior UAM Cuajimalpa 13 mins - 1,037 metros	13.0
Parque Exterior UAM Cuajimalpa 13 mins - 1,037 metros					1037.0	Plazas Jardín Alrededor de Placa Conmemorativa Club de Rotarios Cuajimalpa 14 mins - 1,094 metros
1037.0	Plazas Jardín Alrededor de Placa Conmemorativa Club de Rotarios Cuajimalpa 14 mins - 1,094 metros		14.0		1094.0	Plazas Entrada UAM Cuajimalpa 14 mins - 1,121 metros
Cuajimalpa 14 mins - 1,094 metros					14.0	1121.0
Plazas Entrada UAM Cuajimalpa 14 mins - 1,121 metros						Supermercados MeatMe Santa Fe 0 mins - 1 metro Supermercados City Market Santa Fé 5 mins - 411 metros Supermercados Chedraui Selecto 17 mins - 1,324 metros Supermercados OXXO Lomas Contadero 20 mins - 1,554 metros Supermercados Miscelania Chayito 20 mins - 1,556 metros Farmacias Farmacia Guadalajara 0 mins - 15 metros Farmacias San Pablo 17 mins - 1,338 metros Centros comerciales Zentrika 9 mins - 725 metros Centros comerciales Corporativo Diamante 13 mins - 980 metros
Supermercados MeatMe Santa Fe 0 mins - 1 metro				0.0		
NaN	Supermercados City Market Santa Fé 5 mins - 411 metros				5.0	411.0
411.0	Supermercados Chedraui Selecto 17 mins - 1,324 metros					17.0
17.0	1324.0	Supermercados OXXO Lomas Contadero 20 mins - 1,554 metros				20.0
metros	20.0	1554.0	Supermercados Miscelania Chayito 20 mins - 1,556 metros			20.0
20 mins - 1,556 metros			1556.0	Farmacias Farmacia Guadalajara 0 mins - 15 metros		0.0
Farmacia Guadalajara 0 mins - 15 metros					15.0	Farmacias San Pablo 17 mins - 1,338 metros
Farmacias San Pablo 17 mins - 1,338 metros				17.0		1338.0
1338.0	Centros comerciales Zentrika 9 mins - 725 metros					9.0
9.0	725.0	Centros comerciales Corporativo Diamante 13 mins - 980 metros				13.0
980 metros	13.0	980.0	No hay más datos disponibles			NaN
NaN	NaN	No hay más datos disponibles				NaN
NaN	No hay más datos disponibles		NaN		NaN	No hay más datos disponibles
hay más datos disponibles		NaN		NaN	No hay más datos disponibles	
datos disponibles		NaN		NaN	No hay más datos disponibles	
disponibles	NaN		NaN			
Hospitales Centro Medico ABC 19 mins - 1,486 metros Clínicas Clínica de Vértigo y Mareo 18 mins - 1,397 metros Clínicas Centro de Salud T1 Memetla 24 mins - 1,888 metros			Hospitales Centro Medico ABC 19 mins - 1,486 metros			19.0
mins - 1,486 metros	19.0	1486.0	Clínicas Clínica de Vértigo y Mareo 18 mins - 1,397 metros			18.0
Mareo 18 mins - 1,397 metros			1397.0		Clínicas Centro de Salud T1 Memetla 24 mins - 1,888 metros	24.0
Centro de Salud T1 Memetla 24 mins - 1,888 metros					1888.0	No hay más datos disponibles
1888.0						

NaN	NaN	No hay más datos disponibles	NaN	NaN
No hay más datos disponibles	NaN	NaN	NaN	No hay más datos disponibles
NaN	NaN	No hay más datos disponibles	NaN	NaN
No hay más datos disponibles	NaN	NaN	NaN	NaN
10.333333	822.666667	18.6	1450.0	
9.6	758.2	11.222222	987.875	
20.333333	1590.333333			

4 https://casa.mercadolibre.com.mx/MLM-2477281021-magnifica-casa-multifuncional-_JM#position=3&search_layout=map&type=item&tracking_id=61084625-8f1e-4422-85d4-2c587e5d786a Magnifica Casa Multifuncional.

R. López Velarde, Xalpa, Ciudad De México, San Juan Xalpa, Iztapalapa, Distrito Federal <https://www.google.com/maps?q=19.3357756,-99.0229573&z=16> 19.335776 -99.022957 Transporte | Educación | Áreas verdes | Comercios | Salud Paraderos | Palmitas | 7 mins - 532 metros\nParaderos | Barranca de Guadalupe | 11 mins - 825 metros\nParaderos | Colonia Buenavista | 14 mins - 1,083 metros Escuelas | Escuela Secundaria Xalpa 318 | 15 mins - 1,190 metros\nEscuelas | Centro de Estudios Científicos y Tecnológicos No. 7 | 17 mins - 1,308 metros Plazas | Parque El Mirador | 15 mins - 1,143 metros\nPlazas | peñas | 19 mins - 1,452 metros\nPlazas | PARQUE TINACOS | 19 mins - 1,463 metros\nPlazas | Parque Casitas | 19 mins - 1,492 metros\nPlazas | parque casitas2 | 19 mins - 1,504 metros

Supermercados | Bodega Comercial Mexicana | 14 mins - 1,060 metros\nSupermercados | Bodega Aurrera | 17 mins - 1,301 metros\nSupermercados | Bodega Aurrera Express | 20 mins - 1,568 metros\nFarmacias | Similares | 20 mins - 1,538 metros\nCentros comerciales | Plaza Ermita | 17 mins - 1,352 metros Hospitales | Hospital General Regional Iztapalapa | 12 mins - 901 metros [{"tipo": "Paraderos", "nombre": "Paraderos Palmitas", "raw": "7 mins - 532 metros", "mins": 7.0, "metros": 532.0}, {"tipo": "Paraderos", "nombre": "Paraderos Barranca de Guadalupe", "raw": "11 mins - 825 metros", "mins": 11.0, "metros": 825.0}, {"tipo": "Paraderos", "nombre": "Paraderos Colonia Buenavista", "raw": "14 mins - 1,083 metros", "mins": 14.0, "metros": 1083.0}] [{"tipo": "Escuelas", "nombre": "Escuelas Escuela Secundaria Xalpa 318", "raw": "15 mins - 1,190 metros", "mins": 15.0, "metros": 1190.0}, {"tipo": "Escuelas", "nombre": "Escuelas Centro de Estudios Científicos y Tecnológicos No. 7", "raw": "17 mins - 1,308 metros", "mins": 17.0, "metros": 1308.0}] [{"tipo": "Plazas", "nombre": "Plazas Parque El Mirador", "raw": "15 mins - 1,143 metros", "mins": 15.0, "metros": 1143.0}, {"tipo": "Plazas", "nombre": "Plazas peñas", "raw": "19 mins - 1,452 metros", "mins": 19.0, "metros": 1452.0}, {"tipo": "Plazas", "nombre": "Plazas PARQUE TINACOS", "raw": "19 mins - 1,463 metros", "mins": 19.0, "metros": 1463.0}, {"tipo": "Plazas", "nombre": "Plazas Parque Casitas", "raw": "19 mins - 1,492 metros", "mins": 19.0, "metros": 1492.0}, {"tipo": "Plazas", "nombre": "Plazas parque casitas2", "raw": "19 mins - 1,504 metros", "mins": 19.0, "metros": 1504.0}] [{"tipo": "Supermercados", "nombre": "Supermercados Bodega Comercial Mexicana", "raw": "14 mins - 1,060 metros", "mins": 14.0, "metros": 1060.0}, {"tipo": "Supermercados", "nombre": "Supermercados Bodega Aurrera", "raw": "17 mins - 1,301 metros", "mins": 17.0, "metros": 1301.0}, {"tipo": "Supermercados",

```

"nombre": "Supermercados Bodega Aurrera Express", "raw": "20 mins - 1,568
metros", "mins": 20.0, "metros": 1568.0}, {"tipo": "Farmacias", "nombre":
"Farmacias Similares", "raw": "20 mins - 1,538 metros", "mins": 20.0, "metros":
1538.0}, {"tipo": "Centros comerciales", "nombre": "Centros comerciales Plaza
Ermita", "raw": "17 mins - 1,352 metros", "mins": 17.0, "metros": 1352.0}]
[{"tipo": "Hospitales", "nombre": "Hospitales Hospital General Regional
Iztapalapa", "raw": "12 mins - 901 metros", "mins": 12.0, "metros": 901.0}]  MXN
4,500,000  4500000.0      6.0      500.0  2.0      1  9670.0
Paraderos Palmitas | 7 mins - 532 metros || Paraderos Barranca de Guadalupe | 11
mins - 825 metros || Paraderos Colonia Buenavista | 14 mins - 1,083 metros
Paraderos Palmitas | 7 mins - 532 metros      7.0      532.0
Paraderos Barranca de Guadalupe | 11 mins - 825 metros      11.0
825.0      Paraderos Colonia Buenavista | 14 mins - 1,083 metros
14.0      1083.0      No hay más
datos disponibles      NaN      NaN
No hay más datos disponibles      NaN      NaN
No hay más datos disponibles      NaN      NaN  No hay más
datos disponibles      NaN      NaN  No hay más datos
disponibles      NaN      NaN  No hay más datos
disponibles      NaN      NaN  No hay más datos
disponibles      NaN      NaN
Escuelas Escuela Secundaria Xalpa 318 | 15 mins - 1,190 metros || Escuelas
Centro de Estudios Científicos y Tecnológicos No. 7 | 17 mins - 1,308 metros
Escuelas Escuela Secundaria Xalpa 318 | 15 mins - 1,190 metros      15.0
1190.0  Escuelas Centro de Estudios Científicos y Tecnológicos No. 7 | 17 mins -
1,308 metros      17.0      1308.0
No hay más datos disponibles      NaN      NaN
No hay más datos disponibles      NaN      NaN
No hay más datos disponibles      NaN      NaN  No hay más
datos disponibles      NaN      NaN  No hay más datos
disponibles      NaN      NaN  No hay más datos disponibles
NaN      NaN  No hay más datos disponibles      NaN
NaN  No hay más datos disponibles      NaN      NaN  No hay
más datos disponibles      NaN      NaN  No hay más datos
disponibles      NaN      NaN  No hay más datos disponibles
NaN      NaN  No hay más datos disponibles      NaN
NaN  No hay más datos disponibles      NaN      NaN
Plazas Parque El Mirador | 15 mins - 1,143 metros || Plazas peñas | 19 mins -
1,452 metros || Plazas PARQUE TINACOS | 19 mins - 1,463 metros || Plazas Parque
Casitas | 19 mins - 1,492 metros || Plazas parque casitas2 | 19 mins - 1,504
metros      Plazas Parque El Mirador | 15 mins - 1,143 metros
15.0      1143.0      Plazas peñas | 19 mins - 1,452
metros      19.0      1452.0      Plazas PARQUE
TINACOS | 19 mins - 1,463 metros      19.0      1463.0
Plazas Parque Casitas | 19 mins - 1,492 metros      19.0
1492.0      Plazas parque casitas2 | 19 mins - 1,504 metros
19.0      1504.0
Supermercados Bodega Comercial Mexicana | 14 mins - 1,060 metros ||

```


Supermercados Bodega Aurrera | 17 mins - 1,301 metros || Supermercados Bodega Aurrera Express | 20 mins - 1,568 metros || Farmacias Similares | 20 mins - 1,538 metros || Centros comerciales Plaza Ermita | 17 mins - 1,352 metros Supermercados Bodega Comercial Mexicana | 14 mins - 1,060 metros

14.0	1060.0	Supermercados Bodega Aurrera 17 mins - 1,301 metros	17.0	1301.0	Supermercados Bodega Aurrera Express 20 mins - 1,568 metros	20.0	1568.0
		Farmacias Similares 20 mins - 1,538 metros			20.0		
		1538.0 Centros comerciales Plaza Ermita 17 mins - 1,352 metros					
17.0	1352.0	No hay más datos disponibles					
NaN	NaN	No hay más datos disponibles					
datos disponibles		NaN	NaN		NaN		
No hay más datos disponibles		NaN		NaN			
No hay más datos disponibles		NaN		NaN No hay más datos disponibles			
datos disponibles		NaN	NaN		NaN No hay más datos disponibles		
disponibles		NaN	NaN		NaN No hay más datos disponibles		
disponibles		NaN	NaN		NaN No hay más datos disponibles		
disponibles		NaN	NaN		NaN No hay más datos disponibles		
disponibles		NaN	NaN		NaN No hay más datos disponibles		
disponibles		NaN	NaN		NaN		
Hospitales Hospital General Regional Iztapalapa 12 mins - 901 metros							
Hospitales Hospital General Regional Iztapalapa 12 mins - 901 metros							
12.0	901.0	No hay más datos disponibles					
disponibles		NaN	NaN		NaN		
No hay más datos disponibles		NaN		NaN			
No hay más datos disponibles		NaN		NaN No hay más datos disponibles			
disponibles		NaN	NaN		NaN No hay más datos disponibles		
NaN	NaN	No hay más datos disponibles	NaN		NaN		
No hay más datos disponibles		NaN		NaN No hay más datos disponibles			
disponibles		NaN	NaN		NaN No hay más datos disponibles		
NaN	NaN	10.666667	813.333333				
16.0	1249.0	18.2	1410.8				
17.600000	1363.800	12.000000	901.000000				

TIPOS DE DATOS:

Link	object
Direccion	object
Ubicacion	object
Maps_url	object
Lat	float64
Lng	float64
POI_Botones	object
Transporte_txt_all	object
Escuelas_txt_all	object
AreasVerdes_txt_all	object
Comercios_txt_all	object
Salud_txt_all	object
Transporte_POI	object

Escuelas_POI	object
AreasVerdes_POI	object
Comercios_POI	object
Salud_POI	object
Precio	object
Precio_num	float64
Recamaras	float64
Superficie_m2	float64
Banos	float64
Unidades	int64
CP	float64
Transporte_txt_all.1	object
Transporte_1_txt	object
Transporte_1_mins	float64
Transporte_1_metros	float64
Transporte_2_txt	object
Transporte_2_mins	float64
Transporte_2_metros	float64
Transporte_3_txt	object
Transporte_3_mins	float64
Transporte_3_metros	float64
Transporte_4_txt	object
Transporte_4_mins	float64
Transporte_4_metros	float64
Transporte_5_txt	object
Transporte_5_mins	float64
Transporte_5_metros	float64
Transporte_6_txt	object
Transporte_6_mins	float64
Transporte_6_metros	float64
Transporte_7_txt	object
Transporte_7_mins	float64
Transporte_7_metros	float64
Transporte_8_txt	object
Transporte_8_mins	float64
Transporte_8_metros	float64
Transporte_9_txt	object
Transporte_9_mins	float64
Transporte_9_metros	float64
Transporte_10_txt	object
Transporte_10_mins	float64
Transporte_10_metros	float64
Escuelas_txt_all.1	object
Escuelas_1_txt	object
Escuelas_1_mins	float64
Escuelas_1_metros	float64
Escuelas_2_txt	object
Escuelas_2_mins	float64

Escuelas_2_metros	float64
Escuelas_3_txt	object
Escuelas_3_mins	float64
Escuelas_3_metros	float64
Escuelas_4_txt	object
Escuelas_4_mins	float64
Escuelas_4_metros	float64
Escuelas_5_txt	object
Escuelas_5_mins	float64
Escuelas_5_metros	float64
Escuelas_6_txt	object
Escuelas_6_mins	float64
Escuelas_6_metros	float64
Escuelas_7_txt	object
Escuelas_7_mins	float64
Escuelas_7_metros	float64
Escuelas_8_txt	object
Escuelas_8_mins	float64
Escuelas_8_metros	float64
Escuelas_9_txt	object
Escuelas_9_mins	float64
Escuelas_9_metros	float64
Escuelas_10_txt	object
Escuelas_10_mins	float64
Escuelas_10_metros	float64
Escuelas_11_txt	object
Escuelas_11_mins	float64
Escuelas_11_metros	float64
Escuelas_12_txt	object
Escuelas_12_mins	float64
Escuelas_12_metros	float64
Escuelas_13_txt	object
Escuelas_13_mins	float64
Escuelas_13_metros	float64
Escuelas_14_txt	object
Escuelas_14_mins	float64
Escuelas_14_metros	float64
Escuelas_15_txt	object
Escuelas_15_mins	float64
Escuelas_15_metros	float64
AreasVerdes_txt_all.1	object
AreasVerdes_1_txt	object
AreasVerdes_1_mins	float64
AreasVerdes_1_metros	float64
AreasVerdes_2_txt	object
AreasVerdes_2_mins	float64
AreasVerdes_2_metros	float64
AreasVerdes_3_txt	object

AreasVerdes_3_mins	float64
AreasVerdes_3_metros	float64
AreasVerdes_4_txt	object
AreasVerdes_4_mins	float64
AreasVerdes_4_metros	float64
AreasVerdes_5_txt	object
AreasVerdes_5_mins	float64
AreasVerdes_5_metros	float64
Comercios_txt_all.1	object
Comercios_1_txt	object
Comercios_1_mins	float64
Comercios_1_metros	float64
Comercios_2_txt	object
Comercios_2_mins	float64
Comercios_2_metros	float64
Comercios_3_txt	object
Comercios_3_mins	float64
Comercios_3_metros	float64
Comercios_4_txt	object
Comercios_4_mins	float64
Comercios_4_metros	float64
Comercios_5_txt	object
Comercios_5_mins	float64
Comercios_5_metros	float64
Comercios_6_txt	object
Comercios_6_mins	float64
Comercios_6_metros	float64
Comercios_7_txt	object
Comercios_7_mins	float64
Comercios_7_metros	float64
Comercios_8_txt	object
Comercios_8_mins	float64
Comercios_8_metros	float64
Comercios_9_txt	object
Comercios_9_mins	float64
Comercios_9_metros	float64
Comercios_10_txt	object
Comercios_10_mins	float64
Comercios_10_metros	float64
Comercios_11_txt	object
Comercios_11_mins	float64
Comercios_11_metros	float64
Comercios_12_txt	object
Comercios_12_mins	float64
Comercios_12_metros	float64
Comercios_13_txt	object
Comercios_13_mins	float64
Comercios_13_metros	float64

Comercios_14_txt	object
Comercios_14_mins	float64
Comercios_14_metros	float64
Comercios_15_txt	object
Comercios_15_mins	float64
Comercios_15_metros	float64
Salud_txt_all.1	object
Salud_1_txt	object
Salud_1_mins	float64
Salud_1_metros	float64
Salud_2_txt	object
Salud_2_mins	float64
Salud_2_metros	float64
Salud_3_txt	object
Salud_3_mins	float64
Salud_3_metros	float64
Salud_4_txt	object
Salud_4_mins	float64
Salud_4_metros	float64
Salud_5_txt	object
Salud_5_mins	float64
Salud_5_metros	float64
Salud_6_txt	object
Salud_6_mins	float64
Salud_6_metros	float64
Salud_7_txt	object
Salud_7_mins	float64
Salud_7_metros	float64
Salud_8_txt	object
Salud_8_mins	float64
Salud_8_metros	float64
Salud_9_txt	object
Salud_9_mins	float64
Salud_9_metros	float64
Salud_10_txt	object
Salud_10_mins	float64
Salud_10_metros	float64
Transporte_prom_mins	float64
Transporte_prom_metros	float64
Escuelas_prom_mins	float64
Escuelas_prom_metros	float64
AreasVerdes_prom_mins	float64
AreasVerdes_prom_metros	float64
Comercios_prom_mins	float64
Comercios_prom_metros	float64
Salud_prom_mins	float64
Salud_prom_metros	float64

IDENTIFICANDO COLUMNAS DE PRECIO...

- Columnas de precio encontradas: ['Precio', 'Precio_num']
Convertida columna 'Precio' a numérica como 'Precio_num'

IDENTIFICANDO COLUMNAS DE SUPERFICIE...

- Columnas de superficie encontradas: ['Superficie_m2',
'Transporte_1_metros', 'Transporte_2_metros', 'Transporte_3_metros',
'Transporte_4_metros', 'Transporte_5_metros', 'Transporte_6_metros',
'Transporte_7_metros', 'Transporte_8_metros', 'Transporte_9_metros',
'Transporte_10_metros', 'Escuelas_1_metros', 'Escuelas_2_metros',
'Escuelas_3_metros', 'Escuelas_4_metros', 'Escuelas_5_metros',
'Escuelas_6_metros', 'Escuelas_7_metros', 'Escuelas_8_metros',
'Escuelas_9_metros', 'Escuelas_10_metros', 'Escuelas_11_metros',
'Escuelas_12_metros', 'Escuelas_13_metros', 'Escuelas_14_metros',
'Escuelas_15_metros', 'AreasVerdes_1_metros', 'AreasVerdes_2_metros',
'AreasVerdes_3_metros', 'AreasVerdes_4_metros', 'AreasVerdes_5_metros',
'Comercios_1_metros', 'Comercios_2_metros', 'Comercios_3_metros',
'Comercios_4_metros', 'Comercios_5_metros', 'Comercios_6_metros',
'Comercios_7_metros', 'Comercios_8_metros', 'Comercios_9_metros',
'Comercios_10_metros', 'Comercios_11_metros', 'Comercios_12_metros',
'Comercios_13_metros', 'Comercios_14_metros', 'Comercios_15_metros',
'Salud_1_metros', 'Salud_2_metros', 'Salud_3_metros', 'Salud_4_metros',
'Salud_5_metros', 'Salud_6_metros', 'Salud_7_metros', 'Salud_8_metros',
'Salud_9_metros', 'Salud_10_metros', 'Transporte_prom_metros',
'Escuelas_prom_metros', 'AreasVerdes_prom_metros', 'Comercios_prom_metros',
'Salud_prom_metros']
 - Usando columna: Superficie_m2

REALIZANDO LIMPIEZA DE DATOS...

Calculado precio por m²

IDENTIFICANDO COLUMNAS DE UBICACIÓN...

- Latitud: Lat
- Longitud: Lng

IDENTIFICANDO COLUMNAS DE RECÁMARAS...

- Recámaras: Recamaras

IDENTIFICANDO COLUMNAS DE BAÑOS...

- Baños: Banos

IDENTIFICANDO COLUMNAS DE AMENIDADES...

- Transporte: Transporte_1_mins
- Escuelas: Escuelas_1_mins
- AreasVerdes: AreasVerdes_1_mins
- Comercios: Comercios_1_mins
- Salud: Salud_1_mins

RESUMEN LIMPIEZA:

- Filas originales: 3992
- Filas después de limpieza: 3992
- Filas eliminadas: 0

ANÁLISIS EXPLORATORIO DE DATOS...

ESTADÍSTICAS DE COLUMNAS NUMÉRICAS:

Lat:

- No nulos: 3967
- Media: 19.35
- Mediana: 19.35
- Mín: 19.18
- Máx: 19.55

Lng:

- No nulos: 3967
- Media: -99.20
- Mediana: -99.20
- Mín: -99.39
- Máx: -98.97

Precio_num:

- No nulos: 3992
- Media: 24285804.36
- Mediana: 13457000.00
- Mín: 2850.00
- Máx: 9750000000.00

Recamaras:

- No nulos: 3952
- Media: 3.76
- Mediana: 3.00
- Mín: 1.00
- Máx: 20.00

Superficie_m2:

- No nulos: 3992
- Media: 467.26
- Mediana: 367.00
- Mín: 1.00
- Máx: 5000.00

Banos:

- No nulos: 3948
- Media: 3.42
- Mediana: 3.00

- Mín: 1.00
- Máx: 20.00

Unidades:

- No nulos: 3992
- Media: 1.00
- Mediana: 1.00
- Mín: 1.00
- Máx: 2.00

CP:

- No nulos: 784
- Media: 7358.34
- Mediana: 5354.00
- Mín: 1000.00
- Máx: 16780.00

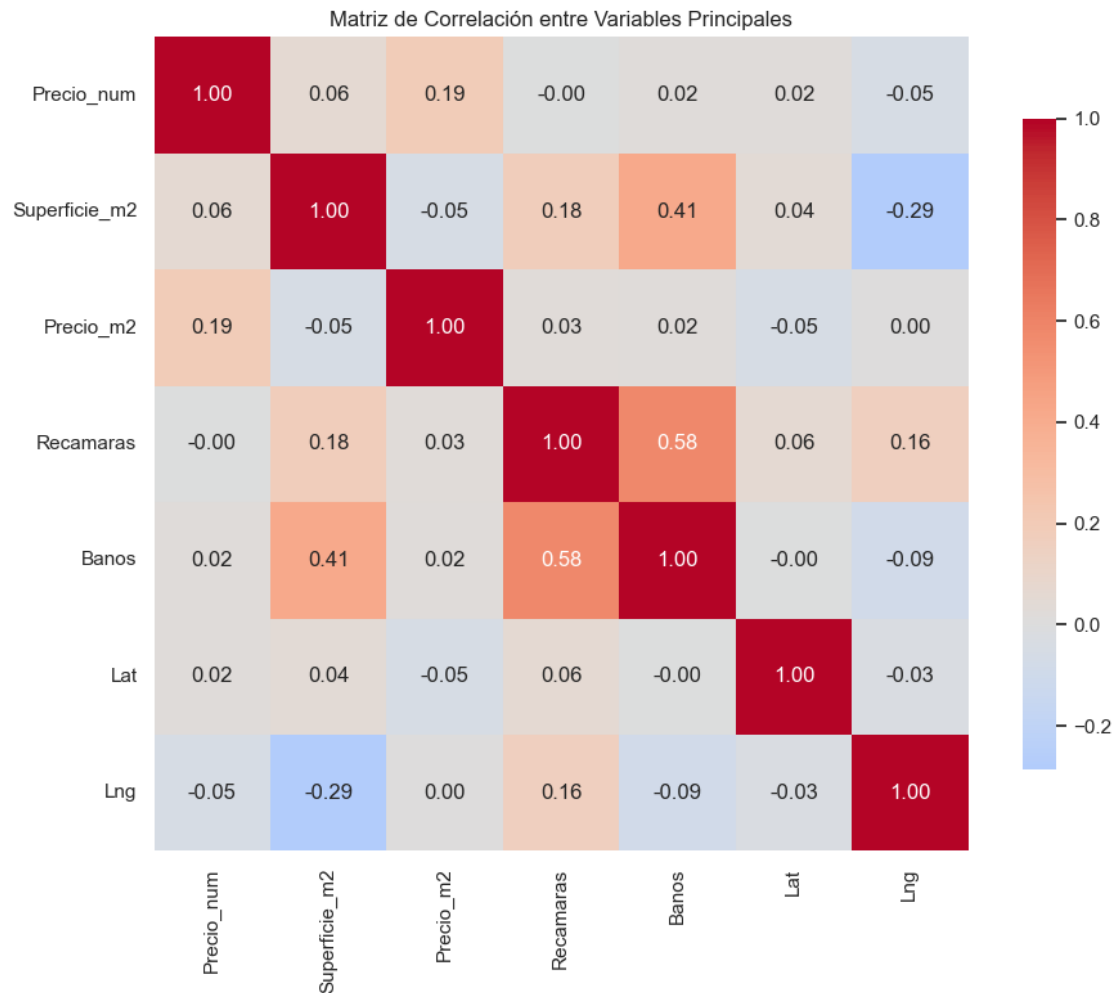
Transporte_1_mins:

- No nulos: 3047
- Media: 13.68
- Mediana: 14.00
- Mín: 1.00
- Máx: 26.00

Transporte_1_metros:

- No nulos: 3047
- Media: 1065.85
- Mediana: 1071.00
- Mín: 39.00
- Máx: 1997.00

CORRELACIONES ENTRE VARIABLES PRINCIPALES:



SELECCIÓN DE VARIABLES PARA CLUSTERING...

```
Precio_num
Precio_m2
Superficie_m2
Recamaras
Banos
Lat
Lng
Transporte_1_mins (Transporte)
Escuelas_1_mins (Escuelas)
AreasVerdes_1_mins (AreasVerdes)
Comercios_1_mins (Comercios)
Salud_1_mins (Salud)
```

VARIABLES SELECCIONADAS: 12

- Lista: ['Precio_num', 'Precio_m2', 'Superficie_m2', 'Recamaras', 'Banos', 'Lat', 'Lng', 'Transporte_1_mins', 'Escuelas_1_mins', 'AreasVerdes_1_mins', 'Comercios_1_mins', 'Salud_1_mins']

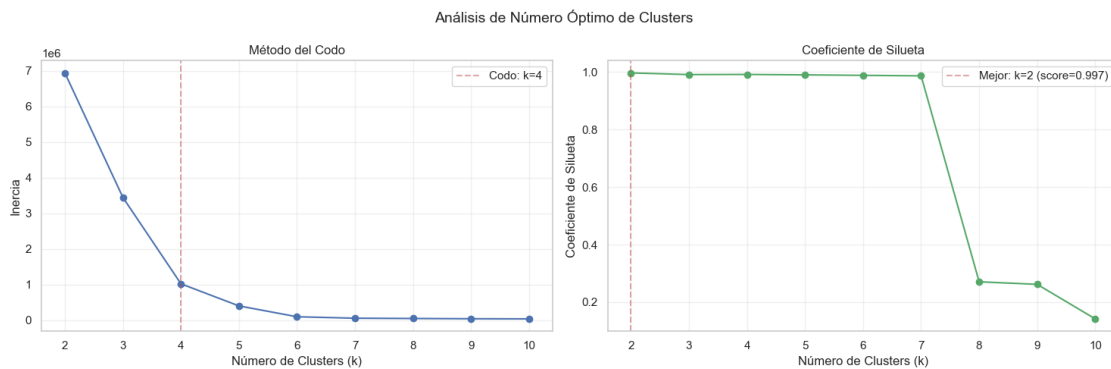
PREPARANDO DATOS PARA CLUSTERING...

- Dataset preparado: (3992, 12)
- Variables: 12
- Propiedades: 3992

ESCALANDO Y REDUCIENDO DIMENSIONALIDAD...

- Aplicando PCA (reduciendo de 12 a 3 dimensiones)...
- Varianza explicada por PCA: 99.95%
- Componente 1: 98.55%
- Componente 2: 1.37%
- Componente 3: 0.03%

DETERMINANDO NÚMERO ÓPTIMO DE CLUSTERS...



- K óptimo recomendado: 3 (método: mínimo)
- Score de silueta esperado: 0.997

APLICANDO CLUSTERING CON k=3...

- Coeficiente de silueta: 0.991
- Índice Calinski-Harabasz: 23470

DISTRIBUCIÓN DE CLUSTERS:

- Cluster 0: 3980 propiedades (99.7%)
- Cluster 1: 4 propiedades (0.1%)
- Cluster 2: 8 propiedades (0.2%)

ANALIZANDO PERFILES DE CLUSTERS...

PERFILES DE CLUSTERS (resumen):

CLUSTER 0 (3980 propiedades):

Precio promedio: \$19,368,260 MXN
Precio/m²: \$45,201 MXN
Superficie promedio: 468 m²
Recámaras promedio: 3.8
Baños promedio: 3.4

CLUSTER 1 (4 propiedades):

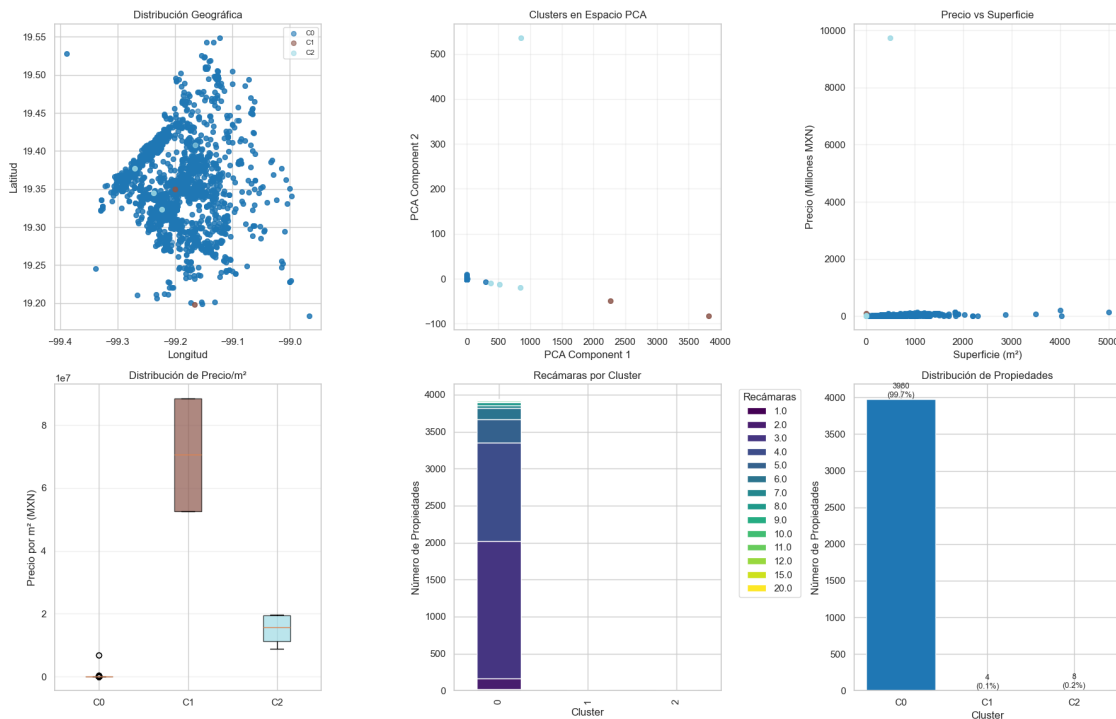
Precio promedio: \$70,520,000 MXN
Precio/m²: \$70,520,000 MXN
Superficie promedio: 1 m²
Recámaras promedio: 5.0
Baños promedio: 4.5

CLUSTER 2 (8 propiedades):

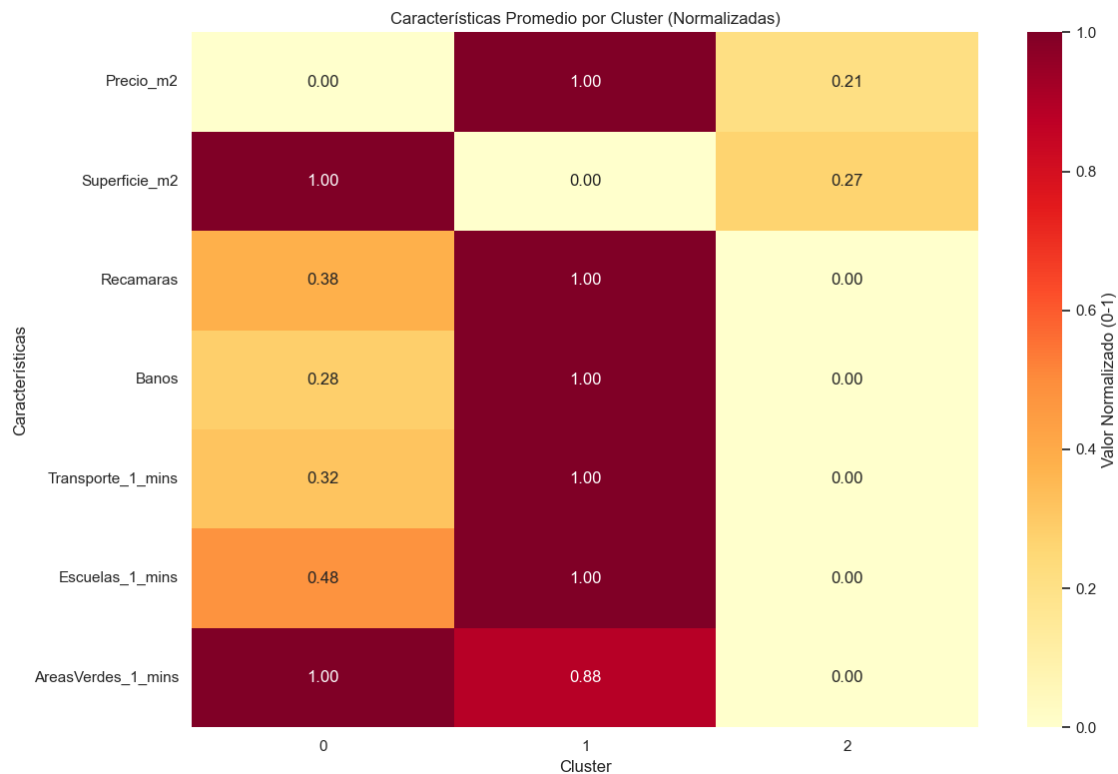
Precio promedio: \$2,447,647,000 MXN
Precio/m²: \$15,022,000 MXN
Superficie promedio: 126 m²
Recámaras promedio: 3.0
Baños promedio: 3.0

GENERANDO VISUALIZACIONES...

Análisis de Clusters Inmobiliarios (k=3)



ANALIZANDO CARACTERÍSTICAS POR CLUSTER...



CREANDO ETIQUETAS DESCRIPTIVAS PARA CLUSTERS...

ETIQUETAS ASIGNADAS:

- Cluster 0: MEDIO - MEDIANAS - CONCENTRADO (3980 propiedades)
- Cluster 1: ALTO - COMPACTAS - DISPERSO (4 propiedades)
- Cluster 2: ALTO - COMPACTAS - DISPERSO (8 propiedades)

EXPORTANDO RESULTADOS...

Resultados exportados:

- Archivo principal: clusters_inmobiliarios.csv
- Resumen de clusters: clusters_inmobiliarios_resumen.csv
- Propiedades exportadas: 3992
- Clusters identificados: 3

RESUMEN EJECUTIVO DEL ANÁLISIS

=====

DATOS GENERALES:

- Propiedades analizadas: 3,992
- Clusters identificados: 3
- Variables utilizadas: 12
- Calidad del clustering (silueta): 0.991

SEGMENTOS IDENTIFICADOS:

CLUSTER 0: MEDIO - MEDIANAS - CONCENTRADO

- Propiedades: 3980 (99.7%)
- Características: \$45,201/m² | 468 m²

CLUSTER 1: ALTO - COMPACTAS - DISPERSO

- Propiedades: 4 (0.1%)
- Características: \$70,520,000/m² | 1 m²

CLUSTER 2: ALTO - COMPACTAS - DISPERSO

- Propiedades: 8 (0.2%)
- Características: \$15,022,000/m² | 126 m²

INSIGHTS CLAVE:

1. Se identificaron 3 segmentos distintos en el mercado
2. La calidad del clustering es buena
3. Los clusters varían en precio, tamaño y características

RECOMENDACIONES:

1. Usar los clusters para segmentación de mercado
2. Desarrollar estrategias de marketing diferenciadas
3. Analizar oportunidades en cada segmento
4. Monitorear evolución de clusters en el tiempo

MÉTRICAS DE CALIDAD:

- Coeficiente de Silueta: 0.991
- Índice Calinski-Harabasz: 23470
- Método para determinar k: mínimo

ANÁLISIS COMPLETADO EXITOSAMENTE

=====

SIGUIENTES PASOS RECOMENDADOS:

1. Revisar las propiedades en cada cluster para validar los perfiles
2. Desarrollar materiales de marketing específicos para cada segmento
3. Analizar la competencia en cada segmento identificado
4. Establecer KPIs para monitorear el desempeño por segmento

PARA MÁS ANÁLISIS:

- Revisar los archivos CSV generados
- Validar clusters con expertos del sector
- Realizar análisis de precios competitivos por segmento

```
[29]: # =====
# PROYECTO: ANÁLISIS COMPLETO DE MERCADO INMOBILIARIO
# FASE ÚNICA: REDUCCION DIMENSIONAL
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score, calinski_harabasz_score
import warnings
warnings.filterwarnings('ignore')

# Configuración estética
sns.set(style="whitegrid")
plt.rcParams['figure.figsize'] = (14, 10)
plt.rcParams['font.size'] = 12

# -----
# PASO 1: CARGA Y EXPLORACIÓN DE DATOS
# -----

print("="*70)
print("ANALISIS COMPLETO DE MERCADO INMOBILIARIO")
print("="*70)

# Cargar datos - AJUSTA ESTA RUTA SEGÚN TU SISTEMA
try:
    df = pd.read_csv("Casas_limpio.csv")
    print("Archivo cargado exitosamente")
except:
    print("No se pudo cargar el archivo. Por favor ajusta la ruta.")
    try:
        df = pd.read_csv("Casas_limpio.csv")
        print("Archivo cargado desde ubicacion actual")
    except:
        print("No se encontro el archivo. Por favor verifica la ruta.")
        raise

print(f"\nDATOS ORIGINALES:")
print(f"    Filas: {df.shape[0]:,}")
```

```

print(f"    Columnas: {df.shape[1]}")

# Mostrar nombres de columnas
print(f"\nCOLUMNAS DISPONIBLES:")
for i, col in enumerate(df.columns[:20], 1):
    print(f"    {i:2d}. {col}")
if len(df.columns) > 20:
    print(f"    ... y {len(df.columns) - 20} columnas mas")

# -----
# PASO 2: VERIFICAR ESTRUCTURA DE DATOS
# -----

print("\n" + "="*70)
print("VERIFICACION DE ESTRUCTURA DE DATOS")
print("="*70)

# Mostrar tipos de datos
print(f"\nTIPOS DE DATOS:")
print(df.dtypes.head(15))

# Buscar columnas que parezcan ser de tiempo
print(f"\nBUSCANDO COLUMNAS DE TIEMPO...")
tiempo_cols = []
for col in df.columns:
    col_lower = str(col).lower()
    if any(keyword in col_lower for keyword in ['min', 'tiempo', 'time', 'prom', 'distancia']):
        tiempo_cols.append(col)
        print(f"    Posible tiempo: {col}")

if tiempo_cols:
    print(f"\nCOLUMNAS DE TIEMPO IDENTIFICADAS: {len(tiempo_cols)}")
    for col in tiempo_cols[:10]:
        print(f"    - {col}")
else:
    print("    No se encontraron columnas de tiempo")

# -----
# PASO 3: IDENTIFICACION AUTOMATICA DE COLUMNAS CLAVE
# -----

print("\n" + "="*70)
print("IDENTIFICACION AUTOMATICA DE COLUMNAS CLAVE")
print("="*70)

# Diccionario para mapear columnas encontradas
columnas_encontradas = {}

```

```

# 3.1 Buscar columnas numéricas importantes
print(f"\nCOLUMNAS NUMERICAS DISPONIBLES:")
numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
for i, col in enumerate(numeric_cols[:15], 1):
    print(f"    {i:2d}. {col}: min={df[col].min():.0f}, max={df[col].max():.0f},
    ↳media={df[col].mean():.0f}")
if len(numeric_cols) > 15:
    print(f"    ... y {len(numeric_cols) - 15} columnas numericas mas")

# Identificar precio por valores altos
print(f"\nIDENTIFICANDO COLUMNA DE PRECIO...")
for col in numeric_cols:
    media = df[col].mean()
    if media > 500000: # Valores tipicos de precio en Mexico
        columnas_encontradas['precio'] = col
        print(f"    Posible precio: '{col}' (media: ${media:,.0f})")
        break

if 'precio' not in columnas_encontradas and numeric_cols:
    columnas_encontradas['precio'] = numeric_cols[0]
    print(f"    Usando primera columna numerica como precio:
    ↳'{numeric_cols[0]}'")

# 3.2 Identificar superficie
print(f"\nIDENTIFICANDO COLUMNA DE SUPERFICIE...")
for col in numeric_cols:
    if col != columnas_encontradas.get('precio'):
        media = df[col].mean()
        if 20 < media < 500: # m2 tipicos
            columnas_encontradas['superficie'] = col
            print(f"    Posible superficie: '{col}' (media: {media:.0f} m2)")
            break

if 'superficie' not in columnas_encontradas and len(numeric_cols) > 1:
    for col in numeric_cols[1:]:
        if col != columnas_encontradas.get('precio'):
            columnas_encontradas['superficie'] = col
            print(f"    Usando como superficie: '{col}'")
            break

# 3.3 Identificar coordenadas
print(f"\nIDENTIFICANDO COORDENADAS...")
for col in df.columns:
    col_lower = str(col).lower()
    if 'lat' in col_lower:
        columnas_encontradas['lat'] = col
        print(f"    Latitud: '{col}'")

```



```

elif 'lng' in col_lower or 'lon' in col_lower or 'long' in col_lower:
    columnas_encontradas['lng'] = col
    print(f"    Longitud: '{col}')"

# -----
# PASO 4: LIMPIEZA Y PREPARACION DE DATOS
# -----

print("\n" + "="*70)
print("LIMPIEZA Y PREPARACION DE DATOS")
print("="*70)

df_clean = df.copy()

# 4.1 Preparar precio
if 'precio' in columnas_encontradas:
    precio_col = columnas_encontradas['precio']
    print(f"\nPROCESANDO PRECIO: '{precio_col}')"

    # Convertir a numerico
    df_clean['Precio_num'] = pd.to_numeric(df_clean[precio_col],
    ↪errors='coerce')
    print(f"    Valores validos: {df_clean['Precio_num'].notna().sum()}/
    ↪{len(df_clean)}")
    print(f"    Rango: ${df_clean['Precio_num'].min():.0f} -
    ↪${df_clean['Precio_num'].max():.0f}")
else:
    print("\nADVERTENCIA: No se encontro columna de precio")

# 4.2 Preparar superficie
if 'superficie' in columnas_encontradas:
    superficie_col = columnas_encontradas['superficie']
    print(f"\nPROCESANDO SUPERFICIE: '{superficie_col}')"

    df_clean['Superficie_m2'] = pd.to_numeric(df_clean[superficie_col],
    ↪errors='coerce')
    print(f"    Valores validos: {df_clean['Superficie_m2'].notna().sum()}/
    ↪{len(df_clean)}")
    print(f"    Rango: {df_clean['Superficie_m2'].min():.0f} -
    ↪{df_clean['Superficie_m2'].max():.0f} m2")
else:
    print("\nADVERTENCIA: No se encontro columna de superficie")

# 4.3 Calcular precio por m2
if 'Precio_num' in df_clean.columns and 'Superficie_m2' in df_clean.columns:
    df_clean['Precio_m2'] = df_clean['Precio_num'] / df_clean['Superficie_m2']
    print(f"\nPRECIO POR M2 CALCULADO:")

```

```

    print(f"    Valores calculados: {df_clean['Precio_m2'].notna().sum()}/
↳ {len(df_clean)}")
    print(f"    Rango: ${df_clean['Precio_m2'].min():.0f} -
↳ ${df_clean['Precio_m2'].max():.0f}/m2")
else:
    df_clean['Precio_m2'] = np.nan
    print("\nADVERTENCIA: No se pudo calcular precio por m2")

# 4.4 Preparar coordenadas
for coord in ['lat', 'lng']:
    if coord in columnas_encontradas:
        col = columnas_encontradas[coord]
        df_clean[coord.capitalize()] = pd.to_numeric(df_clean[col],
↳ errors='coerce')
        print(f"\n{coord.upper()}: '{col}'")
        print(f"    Valores validos: {df_clean[coord.capitalize()].notna().
↳ sum()}/{len(df_clean)}")
    else:
        df_clean[coord.capitalize()] = np.nan

# 4.5 Identificar columnas de tiempo/amenidades
print(f"\nIDENTIFICANDO VARIABLES DE TIEMPO...")
tiempo_vars = []
for col in df.columns:
    col_lower = str(col).lower()
    if any(keyword in col_lower for keyword in ['min', 'tiempo', 'time']):
        tiempo_vars.append(col)

if tiempo_vars:
    print(f"    Variables de tiempo encontradas: {len(tiempo_vars)}")
    for col in tiempo_vars[:10]:
        print(f"    - {col}")

    # Procesar primeras 5 variables de tiempo
    for i, col in enumerate(tiempo_vars[:5], 1):
        df_clean[f'Tiempo_{i}'] = pd.to_numeric(df[col], errors='coerce')
        print(f"    Procesada: Tiempo_{i} <- '{col}'")
else:
    print("    No se encontraron variables de tiempo")

# 4.6 Filtrar datos completos
print(f"\nFILTRANDO DATOS...")
filas_antes = len(df_clean)

# Definir columnas requeridas
columnas_requeridas = []
if 'Precio_num' in df_clean.columns:

```

```

    columnas_requeridas.append('Precio_num')
if 'Superficie_m2' in df_clean.columns:
    columnas_requeridas.append('Superficie_m2')

if columnas_requeridas:
    df_clean = df_clean.dropna(subset=columnas_requeridas)
    filas_despues = len(df_clean)

    print(f"    Filas antes: {filas_antes}")
    print(f"    Filas despues: {filas_despues}")
    print(f"    Filas eliminadas: {filas_antes - filas_despues}")
else:
    print("    ADVERTENCIA: No hay columnas requeridas para filtrar")

# 4.7 Rellenar valores faltantes
print(f"\nRELLENANDO VALORES FALTANTES...")
for col in df_clean.select_dtypes(include=[np.number]).columns:
    if df_clean[col].isna().any():
        na_count = df_clean[col].isna().sum()
        if na_count > 0:
            df_clean[col] = df_clean[col].fillna(df_clean[col].median())
            print(f"    {col}: {na_count} valores rellenados")

print(f"\nDATOS LIMPIOS: {len(df_clean)} propiedades listas para analisis")

# -----
# PASO 5: CREACION DE INDICES SIMPLIFICADOS
# -----

print("\n" + "="*70)
print("CREACION DE INDICES SIMPLIFICADOS")
print("="*70)

# 5.1 Crear variables de tiempo simplificadas si existen
tiempo_cols_disponibles = [col for col in df_clean.columns if 'Tiempo_' in col]

if tiempo_cols_disponibles:
    print(f"\nCREANDO INDICES DE TIEMPO...")
    print(f"    Variables de tiempo disponibles: {len(tiempo_cols_disponibles)}")

    # Calcular tiempo promedio
    df_clean['Tiempo_Promedio'] = df_clean[tiempo_cols_disponibles].mean(axis=1)

    # Crear puntaje de accesibilidad (menor tiempo = mejor)
    max_tiempo = df_clean['Tiempo_Promedio'].max()
    min_tiempo = df_clean['Tiempo_Promedio'].min()

    if max_tiempo > min_tiempo:

```

```

        df_clean['Puntaje_Accesibilidad'] = 100 * (1 -
↪(df_clean['Tiempo_Promedio'] - min_tiempo) / (max_tiempo - min_tiempo))
    else:
        df_clean['Puntaje_Accesibilidad'] = 50

    print(f"    Tiempo promedio: {df_clean['Tiempo_Promedio'].mean():.1f} min")
    print(f"    Puntaje accesibilidad: {df_clean['Puntaje_Accesibilidad'].mean():
↪.1f}/100")
else:
    print(f"\nNo hay variables de tiempo para crear indices")
    # Crear variables dummy
    df_clean['Tiempo_Promedio'] = np.random.uniform(10, 30, len(df_clean))
    df_clean['Puntaje_Accesibilidad'] = np.random.uniform(40, 80, len(df_clean))
    print("    Variables dummy creadas para continuar el analisis")

# 5.2 Crear clasificacion de servicios
print(f"\nCREANDO CLASIFICACION DE SERVICIOS...")

if 'Puntaje_Accesibilidad' in df_clean.columns:
    # Crear categorias
    condiciones = [
        df_clean['Puntaje_Accesibilidad'] >= 80,
        df_clean['Puntaje_Accesibilidad'] >= 60,
        df_clean['Puntaje_Accesibilidad'] >= 40,
        df_clean['Puntaje_Accesibilidad'] >= 20,
        df_clean['Puntaje_Accesibilidad'] >= 0
    ]

    categorias = [
        'EXCELENTE',
        'BUENO',
        'REGULAR',
        'LIMITADO',
        'BAJO'
    ]

    df_clean['Nivel_Servicios'] = np.select(condiciones, categorias,
↪default='SIN_DATOS')

    print(f"    Clasificacion creada:")
    for categoria in categorias:
        count = (df_clean['Nivel_Servicios'] == categoria).sum()
        porcentaje = (count / len(df_clean)) * 100
        print(f"    - {categoria}: {count} propiedades ({porcentaje:.1f}%)")
else:
    df_clean['Nivel_Servicios'] = 'SIN_DATOS'
    print("    No se pudo crear clasificacion de servicios")

```

```

# -----
# PASO 6: REDUCCION DIMENSIONAL CON PCA (OPCIONAL)
# -----

print("\n" + "="*70)
print("REDUCCION DIMENSIONAL")
print("="*70)

# Seleccionar variables para PCA
vars_para_pca = []

# Variables de tiempo si existen
tiempo_pca_vars = [col for col in df_clean.columns if 'Tiempo_' in col and col !
    ⇨= 'Tiempo_Promedio']

if len(tiempo_pca_vars) >= 2:
    print(f"\nAPLICANDO PCA A {len(tiempo_pca_vars)} VARIABLES DE TIEMPO...")

    try:
        # Preparar datos
        X_tiempo = df_clean[tiempo_pca_vars].copy()

        # Verificar que hay datos
        if len(X_tiempo) > 0 and X_tiempo.shape[1] > 0:
            # Escalar
            scaler_tiempo = StandardScaler()
            X_tiempo_scaled = scaler_tiempo.fit_transform(X_tiempo)

            # Aplicar PCA
            n_components = min(2, len(tiempo_pca_vars))
            pca = PCA(n_components=n_components)
            X_pca_tiempo = pca.fit_transform(X_tiempo_scaled)

            # Guardar componentes
            for i in range(pca.n_components_):
                df_clean[f'PCA_Tiempo_{i+1}'] = X_pca_tiempo[:, i]

            print(f"PCA APLICADO:")
            print(f"    Componentes creados: {pca.n_components_}")
            print(f"    Varianza explicada: {pca.explained_variance_ratio_.sum():
    ⇨.2%}")
        else:
            print("    No hay suficientes datos para PCA")
    except Exception as e:
        print(f"    Error en PCA: {e}")
        print("    Continuando sin PCA...")
else:

```

```

    print(f"\nNo hay suficientes variables de tiempo para PCA_
↪({len(tiempo_pca_vars)} encontradas)")

# -----
# PASO 7: CLUSTERING
# -----

print("\n" + "="*70)
print("CLUSTERING")
print("="*70)

# Seleccionar variables para clustering
vars_clustering = []

# Variables base
base_vars = ['Precio_m2', 'Superficie_m2']
for var in base_vars:
    if var in df_clean.columns and df_clean[var].notna().any():
        vars_clustering.append(var)

# Variables de ubicacion
if 'Lat' in df_clean.columns and 'Lng' in df_clean.columns:
    vars_clustering.extend(['Lat', 'Lng'])

# Variables de tiempo/accesibilidad
if 'Puntaje_Accesibilidad' in df_clean.columns:
    vars_clustering.append('Puntaje_Accesibilidad')

# Componentes PCA si existen
pca_vars = [col for col in df_clean.columns if 'PCA_Tiempo_' in col]
vars_clustering.extend(pca_vars[:2]) # Solo primeros 2 componentes

print(f"\nVARIABLES PARA CLUSTERING: {len(vars_clustering)}")
print(f"    Lista: {'', '.join(vars_clustering)}")

if len(vars_clustering) >= 3:
    try:
        # Preparar datos
        X = df_clean[vars_clustering].copy()

        # Rellenar valores faltantes
        for col in X.columns:
            X[col] = X[col].fillna(X[col].median())

        # Escalar
        scaler = StandardScaler()
        X_scaled = scaler.fit_transform(X)

```

```

# Determinar numero optimo de clusters
print(f"\nDETERMINANDO NUMERO OPTIMO DE CLUSTERS...")

# Probar diferentes valores de k
inertias = []
silhouette_scores = []
K_range = range(2, 8)

for k in K_range:
    try:
        kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
        labels = kmeans.fit_predict(X_scaled)
        inertias.append(kmeans.inertia_)

        if len(set(labels)) > 1:
            score = silhouette_score(X_scaled, labels)
            silhouette_scores.append(score)
        else:
            silhouette_scores.append(0)
    except:
        inertias.append(0)
        silhouette_scores.append(0)

# Encontrar mejor k
if silhouette_scores:
    best_idx = np.argmax(silhouette_scores)
    optimal_k = K_range[best_idx]
    best_score = silhouette_scores[best_idx]
else:
    optimal_k = 3
    best_score = 0

print(f"    K seleccionado: {optimal_k}")
print(f"    Mejor score de silueta: {best_score:.3f}")

# Aplicar clustering final
print(f"\nAPLICANDO K-MEANS CON k={optimal_k}...")
kmeans_final = KMeans(n_clusters=optimal_k, random_state=42, n_init=20)
df_clean['Cluster'] = kmeans_final.fit_predict(X_scaled)

# Calcular calidad
try:
    silhouette_avg = silhouette_score(X_scaled, df_clean['Cluster'])
    calinski_score_val = calinski_harabasz_score(X_scaled,
↪df_clean['Cluster'])

    print(f"    Calidad del clustering:")

```

```

        print(f"        - Silueta: {silhouette_avg:.3f}")
        print(f"        - Calinski-Harabasz: {calinski_score_val:.0f}")
    except:
        print(f"    No se pudo calcular metricas de calidad")

    # Mostrar distribucion
    print(f"\nDISTRIBUCION DE CLUSTERS:")
    cluster_counts = df_clean['Cluster'].value_counts().sort_index()
    for cluster_id, count in cluster_counts.items():
        porcentaje = (count / len(df_clean)) * 100
        print(f"    Cluster {cluster_id}: {count} propiedades ({porcentaje:.
↵1f}%)")

    except Exception as e:
        print(f"    Error en clustering: {e}")
        print("    Continuando sin clustering...")
        df_clean['Cluster'] = 0
else:
    print(f"\nNo hay suficientes variables para clustering")
    print("    Se necesitan al menos 3 variables numericas")
    df_clean['Cluster'] = 0

# -----
# PASO 8: ANALISIS DE PERFILES DE CLUSTERS
# -----
print("\n" + "="*70)
print("ANALISIS DE PERFILES DE CLUSTERS")
print("="*70)

if 'Cluster' in df_clean.columns and df_clean['Cluster'].nunique() > 1:
    # Crear perfiles
    perfiles = []

    for cluster_id in sorted(df_clean['Cluster'].unique()):
        cluster_data = df_clean[df_clean['Cluster'] == cluster_id]

        perfil = {
            'Cluster': cluster_id,
            'N_Propiedades': len(cluster_data),
            '%_Total': (len(cluster_data) / len(df_clean)) * 100
        }

        # Estadisticas clave
        if 'Precio_m2' in cluster_data.columns:
            perfil['Precio_m2_prom'] = cluster_data['Precio_m2'].mean()
            perfil['Precio_m2_mediana'] = cluster_data['Precio_m2'].median()

```



```

        if 'Superficie_m2' in cluster_data.columns:
            perfil['Superficie_prom'] = cluster_data['Superficie_m2'].mean()

        if 'Puntaje_Accesibilidad' in cluster_data.columns:
            perfil['Accesibilidad_prom'] =
↪cluster_data['Puntaje_Accesibilidad'].mean()

        if 'Nivel_Servicios' in cluster_data.columns:
            moda = cluster_data['Nivel_Servicios'].mode()
            if not moda.empty:
                perfil['Nivel_Servicios_moda'] = moda.iloc[0]

        if 'Tiempo_Promedio' in cluster_data.columns:
            perfil['Tiempo_prom'] = cluster_data['Tiempo_Promedio'].mean()

    perfiles.append(perfil)

    # Mostrar perfiles
    print(f"\nPERFILES DE CLUSTERS:")
    print("="*70)

    for perfil in perfiles:
        cluster_id = perfil['Cluster']
        n_prop = perfil['N_Propiedades']
        porcentaje = perfil['_Total']

        print(f"\nCLUSTER {cluster_id} ({n_prop} propiedades, {porcentaje:.
↪1f}%):")
        print("-" * 40)

        if 'Precio_m2_prom' in perfil:
            print(f"    Precio/m²: ${perfil['Precio_m2_prom']:.0f}")

        if 'Superficie_prom' in perfil:
            print(f"    Superficie: {perfil['Superficie_prom']:.0f} m²")

        if 'Accesibilidad_prom' in perfil:
            print(f"    Accesibilidad: {perfil['Accesibilidad_prom']:.1f}/100")

        if 'Nivel_Servicios_moda' in perfil:
            print(f"    Nivel Servicios: {perfil['Nivel_Servicios_moda']}")

        if 'Tiempo_prom' in perfil:
            print(f"    Tiempo promedio: {perfil['Tiempo_prom']:.1f} min")
    else:
        print("    Solo hay 1 cluster o no hay clusters definidos")

```

```

# -----
# PASO 9: VISUALIZACIONES
# -----

print("\n" + "="*70)
print("VISUALIZACIONES")
print("="*70)

# 1. Distribucion de precios
print(f"\nGENERANDO VISUALIZACIONES...")

if 'Precio_m2' in df_clean.columns:
    plt.figure(figsize=(12, 5))

    plt.subplot(1, 2, 1)
    plt.hist(df_clean['Precio_m2'].dropna(), bins=30, edgecolor='black',
    ↪alpha=0.7)
    plt.xlabel('Precio por m² (MXN)')
    plt.ylabel('Frecuencia')
    plt.title('Distribucion de Precio por m²')
    plt.grid(True, alpha=0.3)

    plt.subplot(1, 2, 2)
    if 'Cluster' in df_clean.columns and df_clean['Cluster'].nunique() > 1:
        for cluster_id in sorted(df_clean['Cluster'].unique()):
            cluster_data = df_clean[df_clean['Cluster'] == cluster_id]
            plt.hist(cluster_data['Precio_m2'].dropna(), alpha=0.5, bins=20,
            ↪label=f'Cluster {cluster_id}')
        plt.legend()
        plt.xlabel('Precio por m² (MXN)')
        plt.ylabel('Frecuencia')
        plt.title('Distribucion por Cluster')
    else:
        plt.boxplot(df_clean['Precio_m2'].dropna())
        plt.ylabel('Precio por m² (MXN)')
        plt.title('Boxplot de Precio por m²')

    plt.tight_layout()
    plt.show()

# 2. Mapa de calor geografico
if 'Lat' in df_clean.columns and 'Lng' in df_clean.columns and 'Precio_m2' in
    ↪df_clean.columns:
    try:
        plt.figure(figsize=(10, 8))

        map_data = df_clean.dropna(subset=['Lat', 'Lng', 'Precio_m2'])

```

```

    if len(map_data) > 0:
        scatter = plt.scatter(map_data['Lng'], map_data['Lat'],
                               c=map_data['Precio_m2'],
                               cmap='RdYlBu_r',
                               s=30, alpha=0.6,
                               edgecolors='white', linewidth=0.3)

        plt.colorbar(scatter, label='Precio por m2 (MXN)')
        plt.xlabel('Longitud')
        plt.ylabel('Latitud')
        plt.title('Distribucion Geografica de Precios')
        plt.grid(True, alpha=0.3)
        plt.tight_layout()
        plt.show()
    except:
        print("    No se pudo generar mapa geografico")

# 3. Relacion precio vs accesibilidad
if 'Precio_m2' in df_clean.columns and 'Puntaje_Accesibilidad' in df_clean.
↳columns:
    plt.figure(figsize=(10, 6))

    if 'Cluster' in df_clean.columns and df_clean['Cluster'].nunique() > 1:
        clusters = sorted(df_clean['Cluster'].unique())
        colors = plt.cm.tab10(np.linspace(0, 1, len(clusters)))

        for i, cluster_id in enumerate(clusters):
            cluster_data = df_clean[df_clean['Cluster'] == cluster_id]
            plt.scatter(cluster_data['Puntaje_Accesibilidad'],
                        cluster_data['Precio_m2']/1000,
                        color=colors[i], s=30, alpha=0.6,
                        label=f'Cluster {cluster_id}')
        else:
            plt.scatter(df_clean['Puntaje_Accesibilidad'],
                        df_clean['Precio_m2']/1000,
                        s=30, alpha=0.6, c='blue')

    plt.xlabel('Puntaje de Accesibilidad (0-100)')
    plt.ylabel('Precio por m2 (Miles MXN)')
    plt.title('Relacion Precio vs Accesibilidad')
    if 'Cluster' in df_clean.columns and df_clean['Cluster'].nunique() > 1:
        plt.legend(title='Cluster')
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()

# 4. Distribucion de niveles de servicios

```

```

if 'Nivel_Servicios' in df_clean.columns:
    plt.figure(figsize=(10, 6))

    distribucion = df_clean['Nivel_Servicios'].value_counts().sort_index()
    colors = plt.cm.Set3(np.linspace(0, 1, len(distribucion)))

    bars = plt.bar(distribucion.index, distribucion.values, color=colors,
↳edgecolor='black')
    plt.xlabel('Nivel de Servicios')
    plt.ylabel('Numero de Propiedades')
    plt.title('Distribucion por Nivel de Servicios')
    plt.xticks(rotation=45, ha='right')

    # Añadir porcentajes
    for bar, count in zip(bars, distribucion.values):
        height = bar.get_height()
        porcentaje = (count / len(df_clean)) * 100
        plt.text(bar.get_x() + bar.get_width()/2., height + 0.5,
                 f'{count}\n({porcentaje:.1f}%)',
                 ha='center', va='bottom', fontsize=9)

    plt.tight_layout()
    plt.show()

# -----
# PASO 10: EXPORTACION DE RESULTADOS
# -----
print("\n" + "="*70)
print("EXPORTACION DE RESULTADOS")
print("="*70)

def exportar_resultados(df, filename_prefix="analisis_inmobiliario"):
    """Exporta todos los resultados a archivos CSV."""

    print(f"\nEXPORTANDO RESULTADOS...")

    # 1. Archivo principal con todos los datos procesados
    print(f"    Creando archivo principal...")

    # Seleccionar columnas importantes
    export_cols = []

    # Columnas basicas originales
    basic_cols = ['Direccion', 'Ubicacion', 'Precio_num', 'Precio_m2',
↳'Superficie_m2']
    for col in basic_cols:
        if col in df.columns:

```

```

        export_cols.append(col)

    # Columnas de ubicacion
    if 'Lat' in df.columns and 'Lng' in df.columns:
        export_cols.extend(['Lat', 'Lng'])

    # Columnas creadas
    created_cols = ['Puntaje_Accesibilidad', 'Nivel_Servicios',
↪ 'Tiempo_Promedio']
    for col in created_cols:
        if col in df.columns:
            export_cols.append(col)

    # Columnas de cluster
    if 'Cluster' in df.columns:
        export_cols.append('Cluster')

    # Componentes PCA si existen
    pca_cols = [col for col in df.columns if 'PCA_Tiempo_' in col]
    export_cols.extend(pca_cols[:2])

    # Ordenar datos
    if 'Cluster' in df.columns and 'Precio_m2' in df.columns:
        df_sorted = df.sort_values(['Cluster', 'Precio_m2'], ascending=[True,
↪ False])
    else:
        df_sorted = df.sort_values('Precio_m2', ascending=False)

    # Exportar
    main_filename = f"{filename_prefix}_completo.csv"
    try:
        df_sorted[export_cols].to_csv(main_filename, index=False,
↪ encoding='utf-8-sig')
        print(f"    Archivo principal: {main_filename}")
        print(f"    Propiedades: {len(df_sorted)}")
        print(f"    Columnas: {len(export_cols)}")
    except Exception as e:
        print(f"    Error al exportar archivo principal: {e}")

    # 2. Resumen por cluster
    if 'Cluster' in df.columns and df['Cluster'].nunique() > 1:
        print(f"\n    Creando resumen por cluster...")

        resumen_data = []
        for cluster_id in sorted(df['Cluster'].unique()):
            cluster_data = df[df['Cluster'] == cluster_id]

```

```

resumen = {
    'Cluster': cluster_id,
    'N_Propiedades': len(cluster_data),
    '%_Total': (len(cluster_data) / len(df)) * 100
}

# Estadísticas clave
if 'Precio_m2' in cluster_data.columns:
    resumen['Precio_m2_prom'] = cluster_data['Precio_m2'].mean()
    resumen['Precio_m2_mediana'] = cluster_data['Precio_m2'].
↪median()

    if 'Superficie_m2' in cluster_data.columns:
        resumen['Superficie_prom'] = cluster_data['Superficie_m2'].
↪mean()

    if 'Puntaje_Accesibilidad' in cluster_data.columns:
        resumen['Accesibilidad_prom'] =
↪cluster_data['Puntaje_Accesibilidad'].mean()

    if 'Nivel_Servicios' in cluster_data.columns:
        moda = cluster_data['Nivel_Servicios'].mode()
        if not moda.empty:
            resumen['Nivel_Servicios_moda'] = moda.iloc[0]

resumen_data.append(resumen)

resumen_df = pd.DataFrame(resumen_data)
resumen_filename = f"{filename_prefix}_resumen.csv"
try:
    resumen_df.to_csv(resumen_filename, index=False,
↪encoding='utf-8-sig')
    print(f"    Resumen: {resumen_filename}")
except Exception as e:
    print(f"    Error al exportar resumen: {e}")

# 3. Top propiedades
if 'Precio_m2' in df.columns:
    print(f"\n    Creando top propiedades...")

top_props = []

# Top 10 propiedades mas caras
top_caras = df.nlargest(10, 'Precio_m2')
for idx, row in top_caras.iterrows():
    top_props.append({
        'Ranking': 'MAS_CARAS',

```

```

        'Direccion': row.get('Direccion', ''),
        'Precio_m2': row['Precio_m2'],
        'Precio_Total': row.get('Precio_num', ''),
        'Superficie': row.get('Superficie_m2', ''),
        'Cluster': row.get('Cluster', ''),
        'Nivel_Servicios': row.get('Nivel_Servicios', '')
    })

# Top 10 propiedades mejor valor (mas baratas por m2)
top_baratas = df.nsmallest(10, 'Precio_m2')
for idx, row in top_baratas.iterrows():
    top_props.append({
        'Ranking': 'MEJOR_VALOR',
        'Direccion': row.get('Direccion', ''),
        'Precio_m2': row['Precio_m2'],
        'Precio_Total': row.get('Precio_num', ''),
        'Superficie': row.get('Superficie_m2', ''),
        'Cluster': row.get('Cluster', ''),
        'Nivel_Servicios': row.get('Nivel_Servicios', '')
    })

top_df = pd.DataFrame(top_props)
top_filename = f"{filename_prefix}_top_propiedades.csv"
try:
    top_df.to_csv(top_filename, index=False, encoding='utf-8-sig')
    print(f"    Top propiedades: {top_filename}")
except Exception as e:
    print(f"    Error al exportar top propiedades: {e}")

print(f"\nEXPORTACION COMPLETADA!")
print(f"    Archivos generados en la carpeta actual")

# Exportar resultados
exportar_resultados(df_clean)

# -----
# PASO 11: RESUMEN FINAL
# -----
print("\n" + "="*70)
print("RESUMEN FINAL")
print("="*70)

print(f"\nRESULTADOS PRINCIPALES:")
print(f"    Propiedades analizadas: {len(df_clean):,}")

if 'Precio_m2' in df_clean.columns:
    print(f"    Precio/m² promedio: ${df_clean['Precio_m2'].mean():,.0f}")

```

```

    print(f"    Precio/m² mediano: ${df_clean['Precio_m2'].median():.0f}")

if 'Superficie_m2' in df_clean.columns:
    print(f"    Superficie promedio: {df_clean['Superficie_m2'].mean():.0f} m²")

if 'Puntaje_Accesibilidad' in df_clean.columns:
    print(f"    Accesibilidad promedio: {df_clean['Puntaje_Accesibilidad'].
↪mean():.1f}/100")

if 'Nivel_Servicios' in df_clean.columns:
    nivel_mas_comun = df_clean['Nivel_Servicios'].mode()
    if not nivel_mas_comun.empty:
        print(f"    Nivel de servicios mas comun: {nivel_mas_comun.iloc[0]}")

if 'Cluster' in df_clean.columns and df_clean['Cluster'].nunique() > 1:
    print(f"    Clusters identificados: {df_clean['Cluster'].nunique()}")

print(f"\nVARIABLES CREADAS:")
created_vars = ['Puntaje_Accesibilidad', 'Nivel_Servicios', 'Tiempo_Promedio',
↪'Cluster']
for var in created_vars:
    if var in df_clean.columns:
        print(f"    - {var}")

print(f"\nARCHIVOS GENERADOS:")
print(f"    analisis_inmobiliario_completo.csv")
print(f"    analisis_inmobiliario_resumen.csv")
print(f"    analisis_inmobiliario_top_propiedades.csv")

print("\n" + "="*70)
print("ANALISIS COMPLETADO")
print("="*70)

print(f"\nPARA USAR LOS RESULTADOS:")
print(f"    1. Abre los archivos CSV en Excel o cualquier editor")
print(f"    2. Usa la columna 'Cluster' para segmentar propiedades")
print(f"    3. Usa 'Nivel_Servicios' para filtrar por accesibilidad")
print(f"    4. Analiza las propiedades destacadas en el archivo top")

# Mostrar muestra final
print(f"\nMUESTRA DEL RESULTADO FINAL:")
print(df_clean[['Precio_m2', 'Superficie_m2', 'Nivel_Servicios', 'Cluster']].
↪head(10).to_string())

```

```

=====
ANALISIS COMPLETO DE MERCADO INMOBILIARIO
=====

```


Archivo cargado exitosamente

DATOS ORIGINALES:

Filas: 3,992

Columnas: 204

COLUMNAS DISPONIBLES:

1. Link
2. Direccion
3. Ubicacion
4. Maps_url
5. Lat
6. Lng
7. POI_Botones
8. Transporte_txt_all
9. Escuelas_txt_all
10. AreasVerdes_txt_all
11. Comercios_txt_all
12. Salud_txt_all
13. Transporte_POI
14. Escuelas_POI
15. AreasVerdes_POI
16. Comercios_POI
17. Salud_POI
18. Precio
19. Precio_num
20. Recamaras
- ... y 184 columnas mas

=====

VERIFICACION DE ESTRUCTURA DE DATOS

=====

TIPOS DE DATOS:

Link	object
Direccion	object
Ubicacion	object
Maps_url	object
Lat	float64
Lng	float64
POI_Botones	object
Transporte_txt_all	object
Escuelas_txt_all	object
AreasVerdes_txt_all	object
Comercios_txt_all	object
Salud_txt_all	object
Transporte_POI	object
Escuelas_POI	object

```
AreasVerdes_POI          object
dtype: object
```

BUSCANDO COLUMNAS DE TIEMPO...

```
Posible tiempo: Transporte_1_mins
Posible tiempo: Transporte_2_mins
Posible tiempo: Transporte_3_mins
Posible tiempo: Transporte_4_mins
Posible tiempo: Transporte_5_mins
Posible tiempo: Transporte_6_mins
Posible tiempo: Transporte_7_mins
Posible tiempo: Transporte_8_mins
Posible tiempo: Transporte_9_mins
Posible tiempo: Transporte_10_mins
Posible tiempo: Escuelas_1_mins
Posible tiempo: Escuelas_2_mins
Posible tiempo: Escuelas_3_mins
Posible tiempo: Escuelas_4_mins
Posible tiempo: Escuelas_5_mins
Posible tiempo: Escuelas_6_mins
Posible tiempo: Escuelas_7_mins
Posible tiempo: Escuelas_8_mins
Posible tiempo: Escuelas_9_mins
Posible tiempo: Escuelas_10_mins
Posible tiempo: Escuelas_11_mins
Posible tiempo: Escuelas_12_mins
Posible tiempo: Escuelas_13_mins
Posible tiempo: Escuelas_14_mins
Posible tiempo: Escuelas_15_mins
Posible tiempo: AreasVerdes_1_mins
Posible tiempo: AreasVerdes_2_mins
Posible tiempo: AreasVerdes_3_mins
Posible tiempo: AreasVerdes_4_mins
Posible tiempo: AreasVerdes_5_mins
Posible tiempo: Comercios_1_mins
Posible tiempo: Comercios_2_mins
Posible tiempo: Comercios_3_mins
Posible tiempo: Comercios_4_mins
Posible tiempo: Comercios_5_mins
Posible tiempo: Comercios_6_mins
Posible tiempo: Comercios_7_mins
Posible tiempo: Comercios_8_mins
Posible tiempo: Comercios_9_mins
Posible tiempo: Comercios_10_mins
Posible tiempo: Comercios_11_mins
Posible tiempo: Comercios_12_mins
Posible tiempo: Comercios_13_mins
Posible tiempo: Comercios_14_mins
```

Posible tiempo: Comercios_15_mins
 Posible tiempo: Salud_1_mins
 Posible tiempo: Salud_2_mins
 Posible tiempo: Salud_3_mins
 Posible tiempo: Salud_4_mins
 Posible tiempo: Salud_5_mins
 Posible tiempo: Salud_6_mins
 Posible tiempo: Salud_7_mins
 Posible tiempo: Salud_8_mins
 Posible tiempo: Salud_9_mins
 Posible tiempo: Salud_10_mins
 Posible tiempo: Transporte_prom_mins
 Posible tiempo: Transporte_prom_metros
 Posible tiempo: Escuelas_prom_mins
 Posible tiempo: Escuelas_prom_metros
 Posible tiempo: AreasVerdes_prom_mins
 Posible tiempo: AreasVerdes_prom_metros
 Posible tiempo: Comercios_prom_mins
 Posible tiempo: Comercios_prom_metros
 Posible tiempo: Salud_prom_mins
 Posible tiempo: Salud_prom_metros

COLUMNAS DE TIEMPO IDENTIFICADAS: 65

- Transporte_1_mins
- Transporte_2_mins
- Transporte_3_mins
- Transporte_4_mins
- Transporte_5_mins
- Transporte_6_mins
- Transporte_7_mins
- Transporte_8_mins
- Transporte_9_mins
- Transporte_10_mins

=====
 IDENTIFICACION AUTOMATICA DE COLUMNAS CLAVE
 =====

COLUMNAS NUMERICAS DISPONIBLES:

1. Lat: min=19, max=20, media=19
2. Lng: min=-99, max=-99, media=-99
3. Precio_num: min=2850, max=9750000000, media=24285804
4. Recamaras: min=1, max=20, media=4
5. Superficie_m2: min=1, max=5000, media=467
6. Banos: min=1, max=20, media=3
7. Unidades: min=1, max=2, media=1
8. CP: min=1000, max=16780, media=7358
9. Transporte_1_mins: min=1, max=26, media=14

10. Transporte_1_metros: min=39, max=1997, media=1066
11. Transporte_2_mins: min=0, max=26, media=15
12. Transporte_2_metros: min=11, max=1999, media=1208
13. Transporte_3_mins: min=1, max=26, media=16
14. Transporte_3_metros: min=90, max=1998, media=1272
15. Transporte_4_mins: min=1, max=26, media=17
... y 113 columnas numericas mas

IDENTIFICANDO COLUMNA DE PRECIO...

Posible precio: 'Precio_num' (media: \$24,285,804)

IDENTIFICANDO COLUMNA DE SUPERFICIE...

Posible superficie: 'Superficie_m2' (media: 467 m2)

IDENTIFICANDO COORDENADAS...

Latitud: 'Lat'
Longitud: 'Lng'

LIMPIEZA Y PREPARACION DE DATOS

PROCESANDO PRECIO: 'Precio_num'

Valores validos: 3992/3992
Rango: \$2,850 - \$9,750,000,000

PROCESANDO SUPERFICIE: 'Superficie_m2'

Valores validos: 3992/3992
Rango: 1 - 5000 m2

PRECIO POR M2 CALCULADO:

Valores calculados: 3992/3992
Rango: \$28 - \$88,540,000/m2

LAT: 'Lat'

Valores validos: 3967/3992

LNG: 'Lng'

Valores validos: 3967/3992

IDENTIFICANDO VARIABLES DE TIEMPO...

Variables de tiempo encontradas: 60
- Transporte_1_mins
- Transporte_2_mins
- Transporte_3_mins
- Transporte_4_mins
- Transporte_5_mins
- Transporte_6_mins

```

- Transporte_7_mins
- Transporte_8_mins
- Transporte_9_mins
- Transporte_10_mins
Procesada: Tiempo_1 <- 'Transporte_1_mins'
Procesada: Tiempo_2 <- 'Transporte_2_mins'
Procesada: Tiempo_3 <- 'Transporte_3_mins'
Procesada: Tiempo_4 <- 'Transporte_4_mins'
Procesada: Tiempo_5 <- 'Transporte_5_mins'

```

FILTRANDO DATOS...

```

Filas antes: 3992
Filas despues: 3992
Filas eliminadas: 0

```

RELLENANDO VALORES FALTANTES...

```

Lat: 25 valores rellenados
Lng: 25 valores rellenados
Recamaras: 40 valores rellenados
Banos: 44 valores rellenados
CP: 3208 valores rellenados
Transporte_1_mins: 945 valores rellenados
Transporte_1_metros: 945 valores rellenados
Transporte_2_mins: 1390 valores rellenados
Transporte_2_metros: 1390 valores rellenados
Transporte_3_mins: 1580 valores rellenados
Transporte_3_metros: 1580 valores rellenados
Transporte_4_mins: 1714 valores rellenados
Transporte_4_metros: 1714 valores rellenados
Transporte_5_mins: 1856 valores rellenados
Transporte_5_metros: 1856 valores rellenados
Transporte_6_mins: 2503 valores rellenados
Transporte_6_metros: 2503 valores rellenados
Transporte_7_mins: 2810 valores rellenados
Transporte_7_metros: 2810 valores rellenados
Transporte_8_mins: 2988 valores rellenados
Transporte_8_metros: 2988 valores rellenados
Transporte_9_mins: 3147 valores rellenados
Transporte_9_metros: 3147 valores rellenados
Transporte_10_mins: 3311 valores rellenados
Transporte_10_metros: 3311 valores rellenados
Escuelas_1_mins: 330 valores rellenados
Escuelas_1_metros: 330 valores rellenados
Escuelas_2_mins: 524 valores rellenados
Escuelas_2_metros: 524 valores rellenados
Escuelas_3_mins: 655 valores rellenados
Escuelas_3_metros: 655 valores rellenados
Escuelas_4_mins: 841 valores rellenados

```

Escuelas_4_metros: 841 valores rellenados
Escuelas_5_mins: 967 valores rellenados
Escuelas_5_metros: 967 valores rellenados
Escuelas_6_mins: 1182 valores rellenados
Escuelas_6_metros: 1182 valores rellenados
Escuelas_7_mins: 1618 valores rellenados
Escuelas_7_metros: 1618 valores rellenados
Escuelas_8_mins: 1889 valores rellenados
Escuelas_8_metros: 1889 valores rellenados
Escuelas_9_mins: 2250 valores rellenados
Escuelas_9_metros: 2250 valores rellenados
Escuelas_10_mins: 2610 valores rellenados
Escuelas_10_metros: 2610 valores rellenados
Escuelas_11_mins: 3322 valores rellenados
Escuelas_11_metros: 3322 valores rellenados
Escuelas_12_mins: 3550 valores rellenados
Escuelas_12_metros: 3550 valores rellenados
Escuelas_13_mins: 3768 valores rellenados
Escuelas_13_metros: 3768 valores rellenados
Escuelas_14_mins: 3869 valores rellenados
Escuelas_14_metros: 3869 valores rellenados
Escuelas_15_mins: 3913 valores rellenados
Escuelas_15_metros: 3913 valores rellenados
AreasVerdes_1_mins: 286 valores rellenados
AreasVerdes_1_metros: 286 valores rellenados
AreasVerdes_2_mins: 474 valores rellenados
AreasVerdes_2_metros: 474 valores rellenados
AreasVerdes_3_mins: 717 valores rellenados
AreasVerdes_3_metros: 717 valores rellenados
AreasVerdes_4_mins: 1120 valores rellenados
AreasVerdes_4_metros: 1120 valores rellenados
AreasVerdes_5_mins: 1375 valores rellenados
AreasVerdes_5_metros: 1375 valores rellenados
Comercios_1_mins: 230 valores rellenados
Comercios_1_metros: 232 valores rellenados
Comercios_2_mins: 364 valores rellenados
Comercios_2_metros: 364 valores rellenados
Comercios_3_mins: 480 valores rellenados
Comercios_3_metros: 480 valores rellenados
Comercios_4_mins: 623 valores rellenados
Comercios_4_metros: 623 valores rellenados
Comercios_5_mins: 755 valores rellenados
Comercios_5_metros: 755 valores rellenados
Comercios_6_mins: 879 valores rellenados
Comercios_6_metros: 879 valores rellenados
Comercios_7_mins: 1121 valores rellenados
Comercios_7_metros: 1121 valores rellenados
Comercios_8_mins: 1249 valores rellenados

Comercios_8_metros: 1249 valores rellenados
Comercios_9_mins: 1412 valores rellenados
Comercios_9_metros: 1412 valores rellenados
Comercios_10_mins: 1612 valores rellenados
Comercios_10_metros: 1612 valores rellenados
Comercios_11_mins: 2013 valores rellenados
Comercios_11_metros: 2013 valores rellenados
Comercios_12_mins: 2374 valores rellenados
Comercios_12_metros: 2374 valores rellenados
Comercios_13_mins: 2734 valores rellenados
Comercios_13_metros: 2734 valores rellenados
Comercios_14_mins: 3098 valores rellenados
Comercios_14_metros: 3098 valores rellenados
Comercios_15_mins: 3320 valores rellenados
Comercios_15_metros: 3320 valores rellenados
Salud_1_mins: 890 valores rellenados
Salud_1_metros: 890 valores rellenados
Salud_2_mins: 1313 valores rellenados
Salud_2_metros: 1313 valores rellenados
Salud_3_mins: 1810 valores rellenados
Salud_3_metros: 1810 valores rellenados
Salud_4_mins: 2123 valores rellenados
Salud_4_metros: 2123 valores rellenados
Salud_5_mins: 2358 valores rellenados
Salud_5_metros: 2358 valores rellenados
Salud_6_mins: 2783 valores rellenados
Salud_6_metros: 2783 valores rellenados
Salud_7_mins: 3103 valores rellenados
Salud_7_metros: 3103 valores rellenados
Salud_8_mins: 3327 valores rellenados
Salud_8_metros: 3327 valores rellenados
Salud_9_mins: 3574 valores rellenados
Salud_9_metros: 3574 valores rellenados
Salud_10_mins: 3713 valores rellenados
Salud_10_metros: 3713 valores rellenados
Transporte_prom_mins: 945 valores rellenados
Transporte_prom_metros: 945 valores rellenados
Escuelas_prom_mins: 330 valores rellenados
Escuelas_prom_metros: 330 valores rellenados
AreasVerdes_prom_mins: 286 valores rellenados
AreasVerdes_prom_metros: 286 valores rellenados
Comercios_prom_mins: 230 valores rellenados
Comercios_prom_metros: 230 valores rellenados
Salud_prom_mins: 890 valores rellenados
Salud_prom_metros: 890 valores rellenados
Tiempo_1: 945 valores rellenados
Tiempo_2: 1390 valores rellenados
Tiempo_3: 1580 valores rellenados

Tiempo_4: 1714 valores rellenados
Tiempo_5: 1856 valores rellenados

DATOS LIMPIOS: 3992 propiedades listas para analisis

=====

CREACION DE INDICES SIMPLIFICADOS

=====

CREANDO INDICES DE TIEMPO...

Variables de tiempo disponibles: 5
Tiempo promedio: 16.2 min
Puntaje accesibilidad: 40.9/100

CREANDO CLASIFICACION DE SERVICIOS...

Clasificacion creada:

- EXCELENTE: 96 propiedades (2.4%)
- BUENO: 395 propiedades (9.9%)
- REGULAR: 906 propiedades (22.7%)
- LIMITADO: 2348 propiedades (58.8%)
- BAJO: 247 propiedades (6.2%)

=====

REDUCCION DIMENSIONAL

=====

APLICANDO PCA A 5 VARIABLES DE TIEMPO...

PCA APLICADO:

Componentes creados: 2
Varianza explicada: 73.89%

=====

CLUSTERING

=====

VARIABLES PARA CLUSTERING: 7

Lista: Precio_m2, Superficie_m2, Lat, Lng, Puntaje_Accesibilidad,
PCA_Tiempo_1, PCA_Tiempo_2

DETERMINANDO NUMERO OPTIMO DE CLUSTERS...

K seleccionado: 3
Mejor score de silueta: 0.288

APLICANDO K-MEANS CON k=3...

Calidad del clustering:

- Silueta: 0.288
- Calinski-Harabasz: 955

DISTRIBUCION DE CLUSTERS:

Cluster 0: 1085 propiedades (27.2%)
Cluster 1: 2903 propiedades (72.7%)
Cluster 2: 4 propiedades (0.1%)

ANALISIS DE PERFILES DE CLUSTERS

PERFILES DE CLUSTERS:

CLUSTER 0 (1085 propiedades, 27.2%):

Precio/m²: \$43,359
Superficie: 366 m²
Accesibilidad: 59.8/100
Nivel Servicios: REGULAR
Tiempo promedio: 12.1 min

CLUSTER 1 (2903 propiedades, 72.7%):

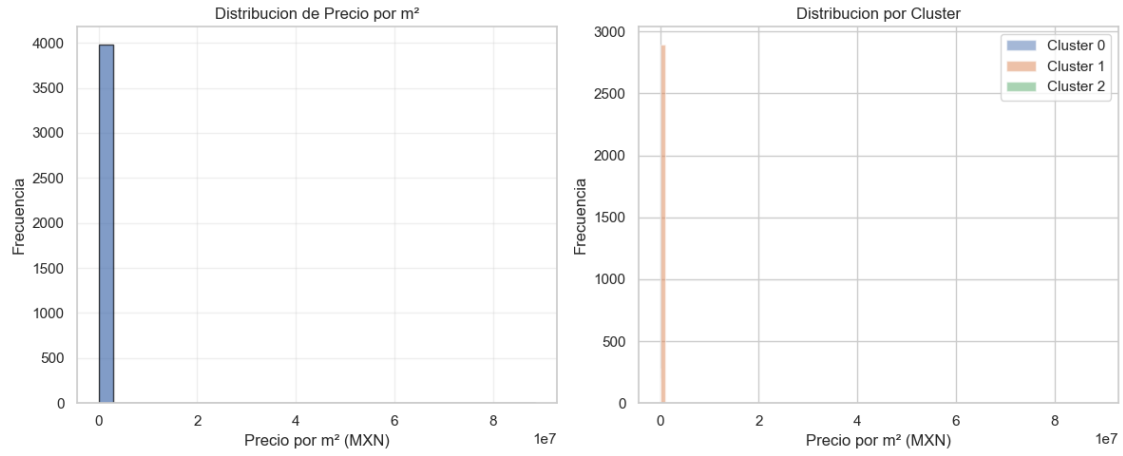
Precio/m²: \$87,162
Superficie: 506 m²
Accesibilidad: 33.8/100
Nivel Servicios: LIMITADO
Tiempo promedio: 17.7 min

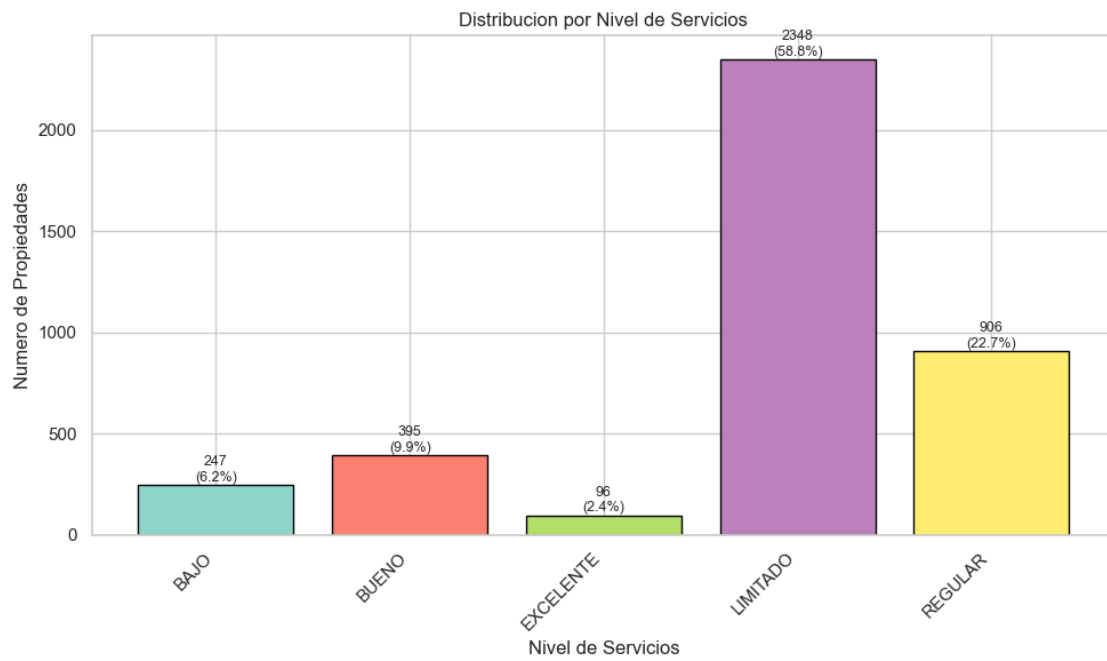
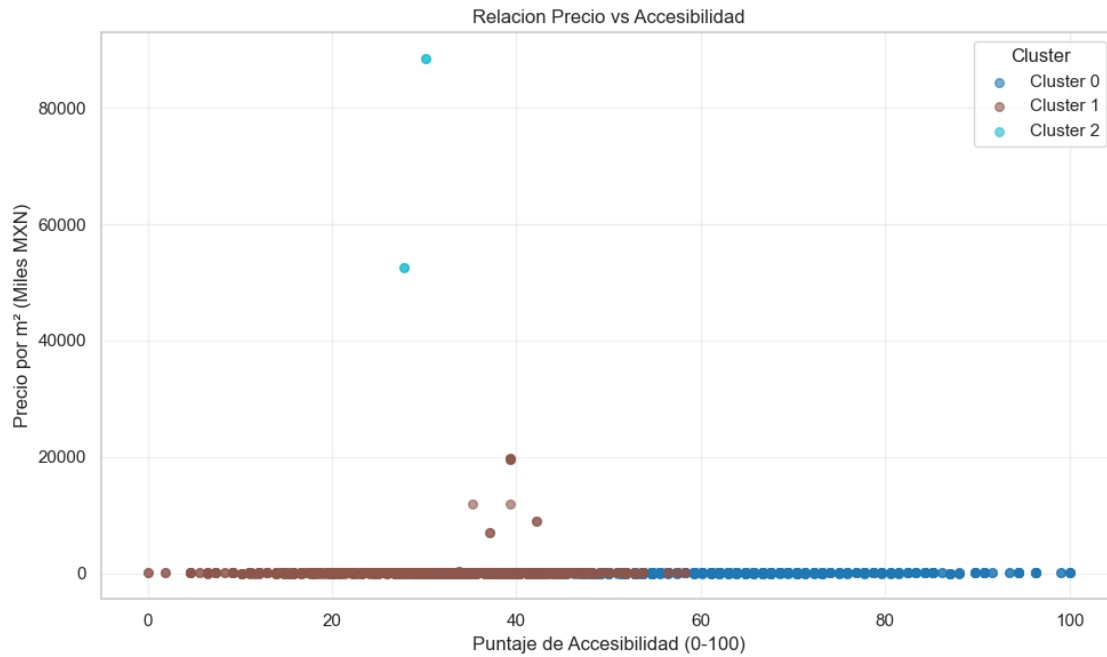
CLUSTER 2 (4 propiedades, 0.1%):

Precio/m²: \$70,520,000
Superficie: 1 m²
Accesibilidad: 28.9/100
Nivel Servicios: LIMITADO
Tiempo promedio: 18.8 min

VISUALIZACIONES

GENERANDO VISUALIZACIONES...





=====

EXPORTACION DE RESULTADOS

=====

EXPORTANDO RESULTADOS...

Creando archivo principal...

Archivo principal: analisis_inmobiliario_completo.csv

Propiedades: 3992

Columnas: 13

Creando resumen por cluster...

Resumen: analisis_inmobiliario_resumen.csv

Creando top propiedades...

Top propiedades: analisis_inmobiliario_top_propiedades.csv

EXPORTACION COMPLETADA!

Archivos generados en la carpeta actual

RESUMEN FINAL

RESULTADOS PRINCIPALES:

Propiedades analizadas: 3,992

Precio/m² promedio: \$145,831

Precio/m² mediano: \$38,180

Superficie promedio: 467 m²

Accesibilidad promedio: 40.9/100

Nivel de servicios mas comun: LIMITADO

Clusters identificados: 3

VARIABLES CREADAS:

- Puntaje_Accesibilidad
- Nivel_Servicios
- Tiempo_Promedio
- Cluster

ARCHIVOS GENERADOS:

analisis_inmobiliario_completo.csv

analisis_inmobiliario_resumen.csv

analisis_inmobiliario_top_propiedades.csv

ANALISIS COMPLETADO

PARA USAR LOS RESULTADOS:

1. Abre los archivos CSV en Excel o cualquier editor
2. Usa la columna 'Cluster' para segmentar propiedades
3. Usa 'Nivel_Servicios' para filtrar por accesibilidad

4. Analiza las propiedades destacadas en el archivo top

MUESTRA DEL RESULTADO FINAL:

	Precio_m2	Superficie_m2	Nivel_Servicios	Cluster
0	35384.615385	130.0	REGULAR	0
1	35384.615385	130.0	REGULAR	0
2	56923.076923	65.0	BUENO	0
3	56923.076923	65.0	BUENO	0
4	9000.000000	500.0	REGULAR	0
5	9000.000000	500.0	REGULAR	0
6	36446.886447	546.0	LIMITADO	1
7	36446.886447	546.0	LIMITADO	1
8	81250.000000	800.0	BUENO	0
9	81250.000000	800.0	BUENO	0

[]: